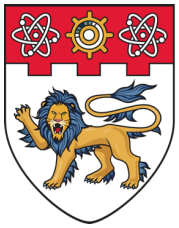
Enhancing Software Evolution Reliability through Neurosymbolic Reasoning



NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

Xiufeng Xu

College of Computing and Data Science

A thesis submitted to the Nanyang Technological University
in partial fulfillment of the requirements for the degree of
Doctor of Philosophy

2026

Statement of Originality

I hereby certify that the work embodied in this thesis is the result of original research, is free of plagiarised materials, and has not been submitted for a higher degree to any other University or Institution.

Disclosure on AI Use:

I hereby declare that Claude (Anthropic), version Opus 4.6, was used to improve the clarity and readability of this thesis. I also declare that I have abided by NTU's Policy on the Use of GenAI in Research, and I have reviewed and edited the content as needed after using this tool, and I take full responsibility for the content of this thesis.

08/04/2026

Date

Xiufeng Xu

Xiufeng Xu

Supervisor Declaration Statement

I have reviewed the content and presentation style of this thesis and declare it is free of plagiarism and of sufficient grammatical clarity to be examined. To the best of my knowledge, the research and writing are those of the candidate except as acknowledged in the Author Attribution Statement. I confirm that the investigations were conducted in accord with the ethics policies and integrity standards of Nanyang Technological University and that the research data are presented honestly and without prejudice.

08/04/2026

Date

Prof. Yi Li

Authorship Attribution Statement

This thesis contains material from three published peer-reviewed papers and one completed manuscript under review, on all of which I am listed as an author.

Chapter 3 contains material from two publications. The first is published as Xiufeng Xu, Chenguang Zhu, and Yi Li. Compsuite: A dataset of java library upgrade incompatibility issues. In International Conference on Automated Software Engineering (ASE). 2023. DOI: 10.1109/ASE56229.2023.00127.

The contributions of the co-authors are as follows:

- Prof. Yi Li and Dr. Chenguang Zhu proposed the topic and provided insightful comments on the methodology and experiment design.
- I constructed the dataset, designed and implemented the tool, performed the experiments, analyzed the data, and drafted the manuscript.
- Prof. Yi Li supervised the project and revised the manuscript.

The second is published as Chenguang Zhu, Mengshi Zhang, Xiuheng Wu, Xiufeng Xu, and Yi Li. Client-specific upgrade compatibility checking via knowledge-guided discovery. ACM Transactions on Software Engineering and Methodology (TOSEM). 2023. DOI: 10.1145/3582569. This work is partially related to the content of this chapter.

The contributions of the co-authors are as follows:

- Dr. Chenguang Zhu proposed the topic, designed the methodology, led the project, and drafted the manuscript.
- Dr. Xiuheng Wu and I jointly constructed the dataset and assisted in conducting the experiments.
- Prof. Mengshi Zhang provided insightful comments and suggestions.
- Prof. Yi Li supervised the project, provided guidance, and revised the manuscript.

Chapter 4 is published as Xiufeng Xu, Fuman Xie, Chenguang Zhu, Guangdong Bai, Sarfraz Khurshid, and Yi Li. Identifying Multi-parameter Constraint Errors in Python Data Science Library API Documentation. In Proceedings of the ACM on Software Engineering (ISSTA). 2025. DOI: 10.1145/3728945.

The contributions of the co-authors are as follows:

- Prof. Yi Li and Dr. Chenguang Zhu proposed the topic and revised the manuscript.

- I co-designed the methodology, implemented and evaluated the approach, and drafted the manuscript.
- Fuman Xie contributed to the implementation of the tool, assisted in conducting the evaluation, and refined the manuscript.
- Prof. Sarfraz Khurshid and Prof. Guangdong Bai provided insightful comments on the methodology design.

The contributions of the co-authors are as follows:

- Prof. Yi Li proposed the topic, supported the research, provided guidance on the methodology design, and revised the manuscript.
- I co-designed the methodology, implemented the tool, conducted the experiments, analyzed the data, and drafted the manuscript.
- Dr. Xiuheng Wu contributed to the implementation of the tool and assisted in conducting the experiments.
- Dr. Zejun Zhang provided insightful comments and helped to write the related work section.

08/04/2026

Date



Xiufeng Xu

Acknowledgements

This PhD journey has been one of the most surprising and rewarding experiences of my life, leading me to where I never expected to reach. It began with a young man standing lost at a crossroads, and it ends beneath the bright sun of the equator. At this moment, I want to express my heartfelt gratitude to everyone who offered their kind support and guidance, both in my academic research and in life beyond it.

First and foremost, I would like to express my deepest gratitude to my advisor, Prof. Yi Li, for his steadfast guidance, patience, and support throughout my PhD studies. His insightful vision and rigorous standards have shaped both this thesis and my growth as a researcher. He granted me the freedom to explore and to stumble, yet was always there with timely guidance whenever I felt lost or disoriented. Looking back over these years, I am struck by how many moments proved formative. He showed me what good research taste truly means, taught me how to think and work as a respected researcher, and, in doing so, helped me become a better person. When my work was scooped, he reminded me that learning to take it in stride is an essential lesson for every researcher. When I hit a wall trying to derive a complex formula, he sat down and spent an entire afternoon working through it with me. He urged me to rehearse my presentations in advance, nudged me to step out of my comfort zone and network at conferences, and even found time to polish the details of my papers while waiting between flights on his busy travels. I may not be his best student, but he is, without question, the best advisor I could have wished for.

It has also been my great privilege to work with Dr. Chenguang Zhu. Many of the early ideas in this thesis grew out of our discussions. Although we have never once met face to face, he provided tremendous help and guidance throughout the conceptualization and refinement of my research. He is an exceptional researcher and collaborator, and in many ways served as a second mentor to me.

I am grateful to Prof. Lei Ma, who stands at the very origin of my PhD journey. It was his recommendation that opened the door for me to join NTU and meet my advisor, Prof. Yi Li, and thus to begin this doctoral chapter of my life. I also wish to give special thanks to Dr. Jianbo Gao, my academic guide before I embarked on this path. His mentorship during my undergraduate years laid the foundation for everything that followed. I further wish to thank my co-authors and collaborators, Xiuheng Wu, Fuman Xie, Prof. Sarfraz Khurshid, Prof. Guangdong Bai, and Zejun Zhang, whose collaboration and support were indispensable to the completion of this thesis.

I would like to thank my fellow lab members, Chengwei Liu, Yushi Cao, Zhiming Li, Bozhi Wu, Tianlin Li, Ye Liu, Chengxuan Zhang, Yixuan Liu, Mengmeng Ge, Jian Wang, Xiaoxuan Lou, Wenbo Guo, Yiran Zhang, Yuqiang Sun, Lida Zhao, Kairan Sun, Chengyue Liu, and Zhengzi Xu, for the stimulating discussions, shared laughter, and camaraderie that made the long days in the lab feel less long. I also appreciate the warm communities at the Peking University Alumni Association (Singapore) and the Konzert Singers Choir, which brought balance and joy to my life far from home.

Another piece of good fortune in Singapore was meeting a group of wonderful young minds: Eric Lei, Kyle Liu, Adam Xin, Grace Zhang, Danial Jiang, Abby Chan, Winston Wang, and Serena Wang. Together we explored the world of programming and AI, and their curiosity, enthusiasm, and creativity continually inspired me to learn new things and to see my own research from a fresh perspective. I am grateful to them and to their parents, Ming Lei, Liyang Zhang, Zhidong Zhang, Hao Wang, Jie Cao, Roger Chan, and others, for the trust they placed in me. I owe special thanks to Ming Lei; our many conversations on topics far beyond research have been a source of profound insight and personal growth.

I owe a deep debt of gratitude to Weijun Wu, who has supported me as if I were his own family. He has a remarkably clear mind, and during my most disoriented moments, he offered illuminating advice that helped me untangle my thoughts and find a way forward.

I am deeply thankful to my girlfriend, Heyang Xu. Her love, patience, and companionship have been the steadiest pillar of support throughout this journey. Though we are separated by national borders and cannot always be together, she has stood

firmly by my side, celebrating with me in moments of progress and comforting me through setbacks. I also wish to thank her parents, Xueming Xu and Lin Yi, who not only understood and supported my research endeavors but also showed me great kindness and care.

Finally, I wish to express my most heartfelt gratitude to my father, Jian Xu, and my mother, Fuyin Xu. While other families gathered together, I was always away studying, and they bore my absence with quiet grace. Their encouragement, selfless support, and unconditional love carried me through the most difficult passages of my life. To be able to share this achievement with them is my greatest honor and deepest joy. This thesis is dedicated to them.

Xiufeng Xu
Singapore
March 2026

To my family

Abstract

Modern software systems are deeply embedded in interconnected dependency networks and must evolve continuously to remain viable. While Large Language Models (LLMs) have emerged as powerful tools for automating software evolution tasks, their probabilistic nature introduces fundamental reliability concerns: inevitable hallucinations and a lack of genuine logical reasoning make fully AI-driven evolution insufficient for industrial-grade demands. This thesis addresses this challenge by advocating a neurosymbolic approach that anchors the generative intelligence of LLMs within formal logical frameworks, systematically enhancing reliability across the three core phases of software evolution: environment perception, logic implementation, and artifact delivery.

For environment perception, this thesis presents CompSuite, a reproducible dataset comprising 123 real-world Java client-library pairs that manifest incompatibility issues upon library upgrades. Sourced from 88 clients and 104 libraries, and accompanied by an automated command-line interface for single-command reproduction, CompSuite provides a high-fidelity empirical foundation for AI systems to perceive and reason about the risks propagated through dependency chains during software evolution.

For artifact delivery, this thesis presents MPChecker, an automated technique for detecting multi-parameter code-documentation inconsistencies. MPChecker employs symbolic execution to extract inter-parameter constraints from code and leverages LLMs to parse corresponding constraints from documentation. A customized fuzzy constraint satisfaction framework reconciles the inherent uncertainty of LLM outputs and enables logical inconsistency detection. Evaluated on four widely used Python data science libraries, MPChecker discovered 14 real-world inconsistencies, of which 11 have been confirmed by the developers.

For logic implementation, this thesis identifies the *Keyword Shortcut* in issue-based code localization and formalizes Keyword-Agnostic Logical Code Localization (KA-LCL). The diagnostic benchmark KA-LogicQuery comprises 225 logic-intensive

queries across 9 projects, while KA-LogicQuery-Neg evaluates abstention when no valid location exists. The proposed DataLoc framework extracts program facts from repositories and synthesizes Datalog programs using an LLM, delegating structural reasoning to a deterministic engine. Parser-gated validation and conservative mutation-based intermediate-rule diagnostics support iterative query refinement. On KA-LogicQuery, DataLoc achieves function-level precision of 66.98% and a success rate of 58.67%, compared with best baseline values of 5.87% and 11.73%, respectively. It returns *no match found* for over 70% of KA-LogicQuery-Neg queries and remains competitive on SWE-bench Lite while returning an average of two candidate locations per issue.

Collectively, this thesis demonstrates that reliable software evolution in the age of AI requires a synthesis of neural flexibility and symbolic rigor. By embedding neurosymbolic reasoning into each phase of the evolution lifecycle, this work provides both theoretical insights and practical tools toward building autonomous systems that are intelligent, verifiable, and trustworthy.

Contents

Acknowledgements ix

Abstract xv

List of Figures xxi

List of Tables xxiii

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Contributions . . . 6
- 1.3 Organization . . . 8

2 Background 11
- 2.1 LLMs and Agentic Systems . . . 11
  - 2.1.1 Large Language Models and Prompt Engineering . . . 11
  - 2.1.2 Automated Workflows and Autonomous Agents . . . 14
- 2.2 Formal Methods in Software Analysis . . . 15
  - 2.2.1 Symbolic Execution and SMT Solver . . . 15
  - 2.2.2 Program Facts and Datalog . . . 17
    - 2.2.2.1 Datalog . . . 17
    - 2.2.2.2 Program Facts in Datalog . . . 18
- 2.3 Neuro-Symbolic AI . . . 19

3 Library Upgrade Incompatibility Issues 21
- 3.1 Introduction . . . 21
  - Contribution. . . . 23
- 3.2 Client-specific Upgrade Incompatibility . . . 23
- 3.3 Dataset Creation . . . 26
  - 3.3.1 Subject Selection . . . 28
  - 3.3.2 Data Collection . . . 31
- 3.4 Dataset Usage . . . 32
  - 3.4.1 Exploring an Incompatibility Issue . . . 32

3.4.2 CompRunner: An Automated Tool for Reproducing Incompatibility Issues . . . 33
3.5 Application Scenarios . . . 34
3.6 Related Work . . . 36
3.7 Summary . . . 37

4 Identifying Multi-parameter Constraint Errors in Documentation 39
4.1 Introduction . . . 39
Contributions. . . . 41
4.2 Background . . . 42
4.2.1 Multi-Parameter Constraints . . . 42
4.2.1.1 Example 1: Explicit Constraint . . . 42
4.2.1.2 Example 2: Implicit Constraint . . . 42
4.2.2 Fuzzy Logic and Fuzzy Constraint Satisfaction . . . 44
4.3 Methodology . . . 45
4.3.1 Preprocessing . . . 46
4.3.2 Constraint Extraction . . . 48
4.3.2.1 Code Constraint Expression Extraction . . . 48
4.3.2.2 Documentation Constraint Expression Extraction . . . 51
Model Selection. . . . 51
Prompt Design. . . . 52
Document Input Prompt. . . . 52
Constraint Extraction Prompt. . . . 52
4.3.3 Inconsistency Detection . . . 55
4.3.3.1 Fuzzy Words . . . 56
4.3.3.2 Fuzzy Constraint Satisfaction . . . 57
4.3.3.3 Constraint Similarity Threshold. . . . 61
4.4 Evaluation . . . 61
4.4.1 Experiment Setup . . . 62
4.4.1.1 Dataset. . . . 62
Constraint Dataset. . . . 62
Inconsistency Dataset. . . . 63
4.4.1.2 Subjects . . . 65
4.4.2 Results . . . 65
4.4.2.1 Accuracy of LLM in extracting constraints from API documentation. . . . 65
4.4.2.2 MPChecker's effectiveness in detecting multi-parameter API documentation errors . . . 67
Comparative Study. . . . 68
4.4.2.3 Practical effect of MPChecker . . . 69
4.5 Discussions . . . 71
4.5.1 Threats to Validity . . . 71
Internal Validity. . . . 71

External Validity. . . . 71
4.5.2 Application Prospects . . . 71
4.6 Related Work . . . 72
API Documentation Analysis. . . . 72
LLM-based Program Analysis. . . . 73

5 Repo-level Code Localization 75
5.1 Introduction . . . 75
5.2 Motivating Example . . . 79
5.3 Methodology . . . 81
5.3.1 Program Facts Extraction . . . 84
5.3.2 Agentic Workflow . . . 86
5.3.2.1 Query Analysis and Information Resolution . . . 86
5.3.2.2 Synthesize-Check-Refine Loop . . . 86
5.3.2.3 Context Retrieval and Final Verification . . . 87
5.3.3 Repairing LLM-Generated Datalog . . . 87
5.3.4 Diagnosing Intermediate Rules via Conservative Mutation Analysis . . . 89
Mutation catalog. . . . 89
Feedback construction. . . . 90
5.3.5 End-to-End Localization Example . . . 92
5.4 Evaluation . . . 96
5.4.1 Benchmarks and Baselines . . . 96
5.4.2 Metrics . . . 98
5.4.3 Results . . . 100
5.4.3.1 Effectiveness on Logic Queries . . . 100
5.4.3.2 Effectiveness on Issue-Based Localization . . . 103
5.4.3.3 Efficiency . . . 104
Temporal efficiency. . . . 104
Token economy. . . . 104
5.4.3.4 Ablation Study . . . 107
Configurations . . . 107
Metrics . . . 107
5.5 Threats to Validity . . . 109
Internal Validity. . . . 109
External Validity. . . . 109
5.6 Related Work . . . 109
LLM for Issue Resolution and Question Answering. . . . 109
Code Localization. . . . 110
5.7 Summary . . . 110

6 Conclusion and Future Work 113
6.1 Conclusion . . . 113
6.2 Implications of Advances in Large Language Models . . . 115

6.2.1 CompSuite: Reproducible Evaluation 115
6.2.2 MPChecker: Simplifying Constraint Extraction 116
6.2.3 DataLoc: Fewer Repairs with Reusable Program Facts . . . 117
6.3 Future Work 118
6.3.1 Benchmarking AI-Assisted Self-Healing Under Dependency Evolution 118
6.3.2 Automated Code-Documentation Consistency Repair 119
6.3.3 Technical Debt and Dependency Fundamentalism 119
6.3.4 Human-AI Interaction: from Intention to Code 120

Bibliography **123**

List of Figures

1.1 Dependency network. 2
1.2 The Structure of the Thesis. 9

2.1 The Transformer model architecture [1]. 12
2.2 Augmented LLM-based agentic system [2]. 14
2.3 A simple C program. 16
2.4 Integrative and hybrid neurosymbolic approaches [3]. 20

3.1 A client-specific incompatibility issue [4]. 25
3.2 The architecture of CompSuite. 30
3.3 The data schema of CompSuite. 32

4.1 Examples of an explicit constraint from `Statsmodels`. 43
4.2 Examples of an implicit constraint from `Scikit-learn`. 44
4.3 The architecture of MPChecker. 45
4.4 Examples of two docstring styles. 48
4.5 Source code transformation for constraint extraction. 50
4.6 Path constraints corresponding to the code in Figure 4.5. 51
4.7 Prompt structure for constraint extraction. 54
4.8 Extended Backus-Naur form for multi-parameter constraints. 58
4.9 Example of the fixed documentation from `Scikit-learn`. 70

5.1 A motivating example of a logic query. 80
5.2 Overview of the DataLoc framework. 82
5.3 Intermediate-rule mutation and feedback. 91
5.4 Relevant program facts from the Astropy repository. 93
5.5 Initial and complete final queries from the Astropy interaction, with original identifiers and constraints. 95
5.6 Average execution time and token consumption of KA-LogicQuery. 106

List of Tables

3.1 Details of clients and libraries included in CompSuite. 27

4.1 Data science libraries used in the experiments. 64
4.2 Results of MPChecker on constraint extraction. 66
4.3 Results of LLM and MPChecker on detecting multi-parameter Code-Documentation Inconsistencies (CDIs). 68

5.1 Effect of keyword shortcuts on localization performance. 76
5.2 Python program-fact relations and their arguments. 85
5.3 Taxonomy of Python Code Dimensions and Representative Elements. 97
5.4 Evaluation results on KA-LogicQuery. 101
5.5 Comparison of different methods and models across various localization granularities. 104
5.6 Ablation Results under Three Configurations. 108

Chapter 1

Introduction

> "To live is to change, and to be perfect is to have changed often."
>
> — St. John Henry Newman

1.1 Motivation

Real software systems are rarely built "all at once" [5]. Instead, software development is inherently an incremental and continuous iterative process. As articulated by Lehman's Laws of Software Evolution [6], a program that is used must be continually adapted; otherwise, it becomes progressively less satisfactory. This evolutionary drive stems from multi-dimensional requirements, ranging from adapting to emerging technology stacks and fixing latent logical defects to responding to ever-evolving security threats. However, this persistent cycle of modification, optimization, and expansion, collectively referred to as software evolution, is a double-edged sword. As features accumulate, the internal entropy of the system inevitably increases, leading to a non-linear growth in structural complexity. Without proactive engineering interventions, costs can eventually outweigh the value delivered by the system, eventually resulting in the accumulation of technical debt and rendering the system unmaintainable.

With the acceleration of global digital transformation, the scenarios encountered by software systems are becoming increasingly complex, naturally driving their

scale to expand to millions or even tens of millions of lines of code. Consequently, the volume of changes required for proactive maintenance of a codebase has become formidable. Furthermore, as illustrated in Figure 1.1, modern software is no longer an isolated node but is deeply embedded in a highly interconnected and interdependent global software supply chain network. Research on the PyPI ecosystem [7], encompassing approximately 380,000 packages, indicates that each package contains an average of 2.6 direct dependencies and 129.6 transitive (non-unique) dependencies. While such interconnection accelerates development through extensive code reuse, it poses significant challenges to software evolution. A minor modification in a single node can propagate along complex dependency chains, potentially triggering unexpected compatibility issues, performance degradation, or security vulnerabilities across numerous downstream projects. For example, the Log4j vulnerability (Log4Shell) [8] demonstrated how a flaw in a widely reused component can propagate through dependency chains and affect numerous downstream systems. In this context, reliable software evolution becomes more difficult because it involves more than just local code modifications.

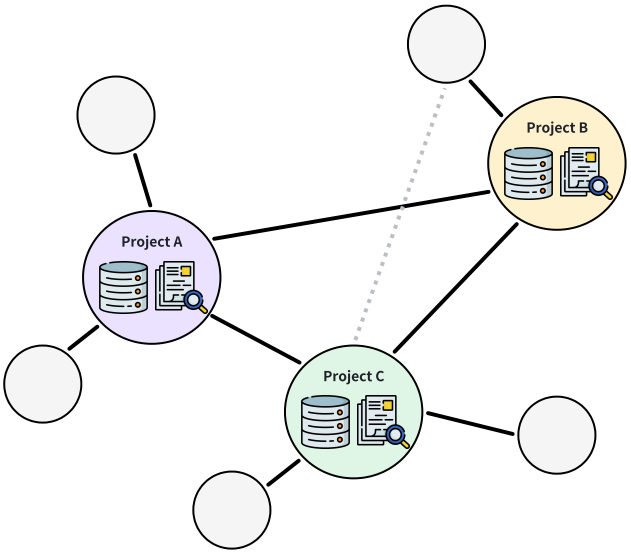


FIGURE 1.1: Dependency network.

Software evolution is a dynamic and continuous modification process to adapt to the ever-changing environment. This is especially evident when software operates as a node within a dependency network, where long-term maintainability is inseparable from the capacity for sustained evolution. Broadly, this constitutes the two primary driving forces: on one hand, **reactive adaptation**, which responds to changes in the external ecosystem such as third-party library upgrades; on the other hand,

proactive improvement, which involves optimizing and refactoring code to meet evolving functional and quality requirements. In general, the execution of software evolution can be abstracted into the following three core phases:

- **Environment perception and requirement identification.** This initial phase entails clearly identifying evolutionary imperatives, including how to manage risks from external changes such as API deprecations or security vulnerabilities, and how to enhance internal quality such as fixing bugs or adding new features. This phase requires a comprehensive understanding of the software's operational context and the ability to accurately identify and prioritize evolution needs.
- **Logic implementation and behavioral verification.** This central phase requires locating relevant code segments and implementing logic modifications according to new environmental constraints and interface contracts. Verification is then conducted to ensure the evolved system satisfies the requirements, confirming the modifications behave as expected without introducing new issues.
- **Artifact delivery and knowledge consolidation.** This final phase involves releasing the evolved software and updating the documentation comprehensively. While documentation is often overlooked, it is a critical component for describing interfaces and functionalities. It plays a crucial role in bridging the gap between internal implementation and external understanding. For large-scale complex systems, documentation is often the only low-cost channel for developers and increasingly AI agents to interpret software behavior and achieve expected usage.

Traditionally, software evolution has been predominantly driven by professional developers, who rely on accumulated domain expertise and deep historical insights into the codebase to guide modifications. To assist developers and support reliable evolution as systems grow in scale and complexity, various software engineering techniques, such as program analysis, automated testing, and formal verification, have been widely adopted to identify potential defects and validate system behaviors. While these methodologies provide essential guarantees of robustness, their practical efficacy in the face of frequent and large-scale software evolution is increasingly constrained. The prohibitive modeling overhead, scalability bottlenecks,

and the well-known state-space explosion problem often prevent these traditional paradigms from keeping pace with the rapid and complex nature of modern codebases.

The integration of Artificial Intelligence (AI), particularly Large Language Models (LLMs), into the software development lifecycle (SDLC) marks a paradigm shift from human-centric to AI-centric autonomous evolution, moving increasingly toward autonomous evolution. AI-powered coding assistants like Cursor [9], Claude Code [10], and Gemini CLI [11] have become indispensable parts of daily development workflows, turning the concept of "Vibe Coding" from a vision into reality. Empowered by advances in LLMs, intelligent development tools and autonomous agents are increasingly capable of understanding and processing large-scale codebases alongside natural language requirements, while progressively demonstrating higher-level reasoning abilities such as cross-context analysis and complicated problem decomposition. Leveraging these capabilities, AI systems can automate a wide range of labor-intensive and high-complexity tasks, such as code generation, bug fixing, documentation synchronization, and system maintenance, thereby reshaping critical stages of software evolution and enabling complex software systems to achieve continuous iteration and scalable growth at unprecedented speed.

Despite these advantages, empirical evidence suggests significant practical limitations. The 2025 Stack Overflow Developer Survey [12] reveals a concerning reality: 66% of developers are frustrated by "AI solutions that are almost right, but not quite", and 45% report that "debugging AI-generated code is more time-consuming". Performance data from the SWE-bench leaderboard [13] further underscores this bottleneck. SWE-bench serves as a mainstream standard for evaluating AI agents against real-world software engineering challenges. Even on SWE-bench Verified, a human-filtered subset of 500 high-quality instances, the most sophisticated integrations of state-of-the-art models and agentic frameworks struggle to achieve a resolution rate exceeding 80%. More critically, these metrics may still be overly optimistic. Since leading LLMs are trained on massive open-source repositories that likely include the benchmark instances themselves, data leakage remains a significant concern. This implies that when confronted with unseen complex software evolution tasks, the actual reliability of current AI systems still falls short of industrial-grade requirements.

There are two core limitations that make reliable fully AI-driven software evolution a challenging task: ❶ the inevitable hallucination problem. It is not an engineering flaw but an inherent theoretical limitation of the generative paradigm. Since LLMs generate outputs based on learned statistical patterns rather than deterministic logic, their behavior is probabilistic in nature. By formalizing hallucination as an inconsistency between computable LLMs and ground truth functions, Xu et al. [14] leverage results from learning theory to demonstrate that such hallucination is a theoretical inevitability. ❷ Lack of genuine logical reasoning capabilities. While the emergent effects brought by large-scale parameterization can mimic reasoning-like processes, and techniques such as Chain-of-Thought (CoT) prompting and Reinforcement Learning (RL) based on verifiable rewards have been employed to enhance reasoning capabilities, these methods still operate within the statistical generation framework. They essentially optimize the statistical probability of producing probabilistically likely sequences rather than adhering to formal deterministic logic. Hallucinations introduce latent vulnerabilities and "plausible but incorrect" solutions, while the absence of genuine reasoning precludes the formal verification and deterministic guarantees required for reliable system evolution.

Addressing these challenges necessitates a dual-track strategy. First, the model's intrinsic capabilities need to be strengthened by providing high-fidelity data, specifically focusing on edge cases in software evolution that are prone to failure and difficult to detect. Second, deterministic mechanisms must be introduced to constrain, guide, and verify the decision-making process of AI. The former is vital for deepening the model's grasp of complex evolutionary contexts, while the latter ensures the reliability and security of AI-driven actions. This balance is best realized through Neurosymbolic AI. By integrating neural methods (e.g., neural networks and deep learning) with symbolic methods (e.g., formal logic, knowledge representation), it combines the superior semantic understanding and efficient learning power of neural methods with the logical rigor and verifiable interpretability of symbolic methods, ensuring that AI-driven software evolution is both intelligent and trustworthy.

This thesis is dedicated to exploring the reliability and security of AI-driven software evolution, aiming to pave a potential path toward a fully autonomous and trustworthy future. The subsequent chapters explore methodologies to optimize

AI performance at each phase of software evolution and provide neurosymbolic frameworks to restrain unpredictable outputs.

1.2 Contributions

AI plays distinct roles and faces unique challenges at each stage of the software evolution lifecycle. In the requirements identification stage, the most common obstacle is the inability to reliably manage external evolution risks. Due to a scarcity of high-quality real-world data, especially for complicated and hard-to-detect scenarios like client-specific compatibility issues triggered by third-party library upgrades, AI lacks the capacity to assess and mitigate the impact and risks introduced by such changes. In the logic implementation stage, code localization is the most critical prerequisite for any reliable code modification. However, current AI systems remain over-reliant on shallow lexical matching rather than deep structural reasoning, failing to accurately identify the specific code segments. This leads to excessive resource overhead and a higher probability of downstream task failure. In the artifact delivery stage, a pervasive problem is that documentation typically lags behind code changes, leading to semantic drift. As documentation serves as a vital interface between the project and its consumers, outdated or inaccurate descriptions directly amplify the risk of external misuse. AI's ability to detect code-documentation inconsistencies remains limited, particularly in complex scenarios involving multi-parameter constraints, making it unable to ensure that documentation faithfully reflects the codebase.

To address these multifaceted challenges, this dissertation advocates for a systematic set of neurosymbolic solutions. For requirements identification, this thesis prioritizes empirical grounding by constructing high-quality datasets that capture real-world breaking change patterns, thereby providing a verifiable benchmark for AI to perceive and reason about external evolutionary risks. For logic implementation, this thesis first formalizes the keyword shortcut bias in current research and proposes a neurosymbolic agent framework. By offloading structural reasoning to a logic engine, the framework transcends the limitations of textual context to achieve precise, repository-level code localization. For artifact delivery, this thesis reports the development of a multi-parameter inconsistency detection framework

that combines formal methods with LLMs, ensuring that the semantic integrity between code and documentation is maintained even under complex constraints.

Central to these solutions is the paradigm of neurosymbolic AI, which serves as the methodological foundation of this research. By synthesizing the formal rigor of symbolic logic, such as symbolic execution and Datalog, with the adaptive learning power of Large Language Models, this thesis provides a deterministic anchor for AI-driven evolution, ensuring that complex software modifications are not only contextually aware but also logically verifiable. Figure 1.2 illustrates the structure of this thesis, which is organized into six chapters. The core contributions of this dissertation are summarized as follows:

- **Reproducible real-world incompatibility dataset.** The study presented in Chapter 3 constructs a dataset, CompSuite, including 123 reproducible, real-world client-library pairs that manifest incompatibility issues when upgrading the library. These data points originate from 88 clients and 104 libraries. An automated command-line interface was created for the dataset. With this interface, users are able to programmatically replicate an incompatibility issue from the dataset with a single command. The interface also offers separate commands for each step involved in the reproduction of incompatibility issues.

- **Multi-parameter code–documentation inconsistency detection.** The study presented in Chapter 4 introduces an automated multi-parameter code-documentation inconsistency detection technique, implemented as an end-to-end command-line tool called MPChecker. Existing techniques in the same area are only designed to handle single parameter inconsistencies, without considering inter-parameter constraints. A customized fuzzy constraint satisfaction framework was introduced to mitigate the uncertainties introduced by LLM outputs. A theoretical derivation of the membership function based on constraint similarity is provided. The study constructed a documentation constraint dataset comprising 72 real-world constraints sourced from widely used data science libraries, and derived a mutation-based inconsistency dataset with 216 constraints. The dataset and tool implementation are made available online: `https://github.com/ParsifalXu/MPChecker`. MPChecker was evaluated on four real-world popular data science libraries.

14 inconsistency issues discovered by MPChecker were reported to the developers, who have confirmed 11 inconsistencies at the time of writing.

- **Keyword Shortcut analysis and diagnostic benchmarks.** The study presented in Chapter 5 identifies the *Keyword Shortcut* in issue-based code localization. In the filtered SWE-bench Lite set, 146 of 274 instances explicitly mention identifiers associated with ground-truth locations. The study introduces KA-LogicQuery for Keyword-Agnostic Logical Code Localization (KA-LCL), comprising 225 logic-intensive queries instantiated from 25 representative combinations across 9 projects, together with KA-LogicQuery-Neg for evaluating abstention when no valid location satisfies a query.

- **Neurosymbolic agent framework for repository-level code localization.** The study presented in Chapter 5 proposes DataLoc, which represents repositories as program facts and translates natural-language constraints into Datalog programs. A synthesize-check-refine loop combines parser-gated validation and repair with conservative mutation-based intermediate-rule diagnostics; diagnostic probe outputs are never returned as final answers. The framework is implemented as an automated end-to-end command-line tool, and the tool and benchmarks are available at `https://zenodo.org/records/22844061`. On KA-LogicQuery, DataLoc achieves function-level precision of 66.98% and a success rate of 58.67%, compared with best baseline values of 5.87% and 11.73%, respectively. It returns *no match found* for over 70% of KA-LogicQuery-Neg queries and remains competitive on SWE-bench Lite while returning an average of two candidate locations per issue.

1.3 Organization

The rest of the thesis is organized as follows:

- **Chapter 2** provides essential background and key concepts necessary for understanding the thesis.

- **Chapter 3** presents CompSuite, a dataset of real-world library upgrade incompatibilities, detailing its construction, characteristics, and potential applications.

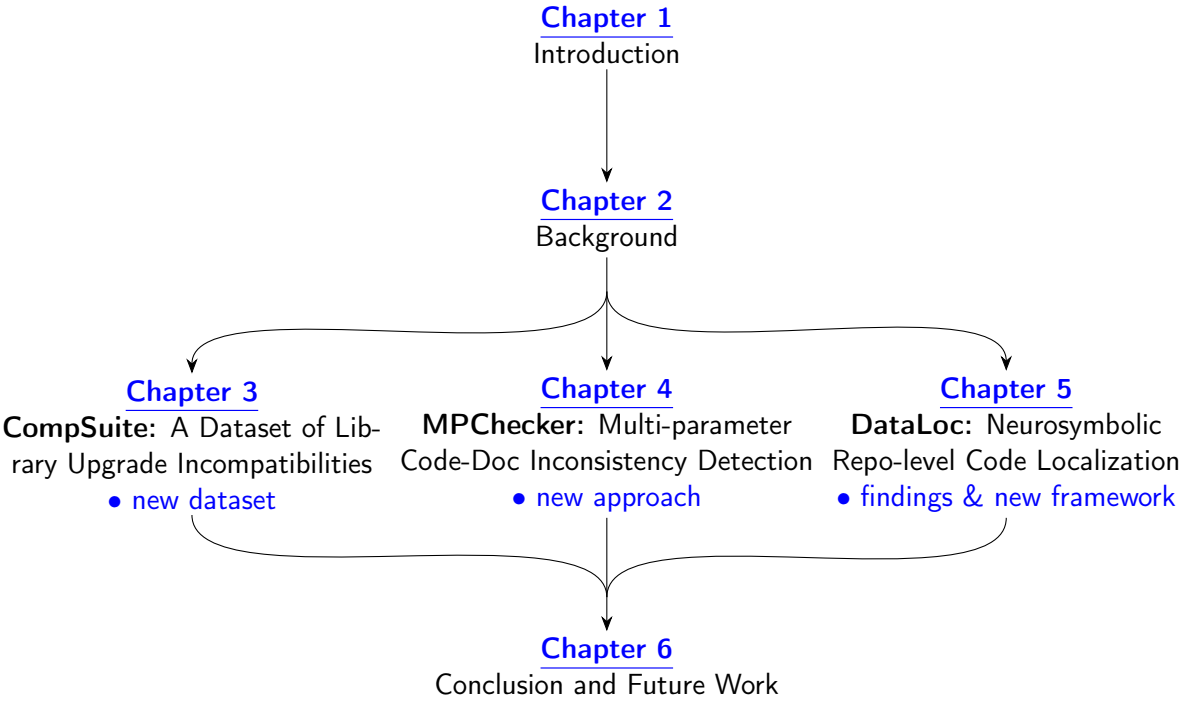


FIGURE 1.2: The Structure of the Thesis.

- **Chapter 4** introduces MPCHECKER, a multi-parameter code-documentation inconsistency detection tool, describing its design, implementation, and evaluation on real-world libraries.
- **Chapter 5** formalizes the keyword shortcut bias in code localization, introduces diagnostic benchmarks, KA-LOGICQUERY and KA-LOGICQUERY-NEG, and presents DATALOC, a neurosymbolic agent framework for repository-level code localization, along with its evaluation results.
- **Chapter 6** summarizes the contributions, discusses how advances in LLMs may simplify the proposed systems while preserving the benefits of explicit analysis and verification, and outlines directions for future research.

Chapter 2

Background

This chapter presents a number of essential concepts and techniques required for describing the contributions in subsequent chapters.

2.1 LLMs and Agentic Systems

2.1.1 Large Language Models and Prompt Engineering

Vaswani et al. [1] introduced the *Transformer* architecture, which marked a pivotal milestone in deep learning and paved the way for the era of large language models (LLMs). Figure 2.1 illustrates the architecture of the Transformer, which employs an encoder-decoder framework, where the encoder transforms an input sequence of symbolic representations $(x_1, x_2, \ldots, x_n)$ into a corresponding sequence of continuous representations $(z_1, z_2, \ldots, z_n)$, upon which the decoder autoregressively produces an output symbol sequence $(y_1, y_2, \ldots, y_m)$. The core mechanism of the Transformer is *Self-Attention*, which allows the model to compute attention scores for each word in a sentence or document in parallel, modeling the influence of each word on every other word.

Given an input sequence of vectors $(x_1, x_2, \ldots, x_n)$, where x_i represents the embedding of the i-th token, each x_i is linearly projected into query (q), key (k), and value (v) vectors by three trained weight matrices. In general, the attention function is computed efficiently by packing the queries, keys, and values into matrices Q, K,

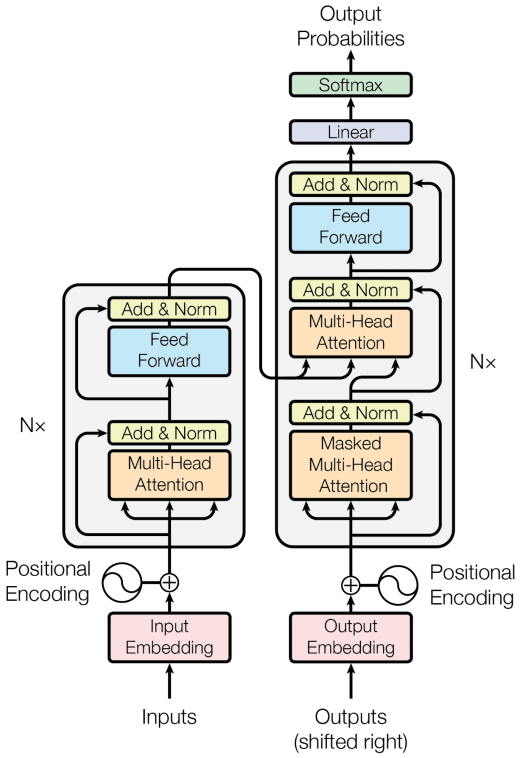


FIGURE 2.1: The Transformer model architecture [1].

and V. The resulting output matrix is defined by the scaled dot-product attention mechanism:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.1}$$

where d_k denotes the dimension of the key vectors.

In practice, the Transformer adopts a *Multi-Head Attention* mechanism rather than a single attention function. The input is split into multiple heads that perform attention functions in parallel, the results of which are concatenated and linearly combined to produce the final output. This design enables the model to simultaneously capture diverse relationships and patterns, allowing it to better understand the context and capture long-range dependencies.

Notably, the parallelizable nature of the self-attention allows it to outperform recurrent neural networks (RNNs) in terms of training efficiency, enabling the efficient pre-training of large-scale models on GPUs. Once trained, these pre-trained

language models (PLMs) can achieve remarkable performance by fine-tuning on specific downstream tasks with relatively small amounts of task-specific data.

Based on the architecture of the Transformer, PLMs have branched into three main categories: encoder-only (e.g., BERT), decoder-only (e.g., GPT), and encoder-decoder (e.g., T5). Among them, the decoder-only architecture has emerged as the dominant choice in recent years due to its effectiveness in natural language generation tasks and superior generalization capabilities, and representative models such as GPT, Claude, Qwen, and DeepSeek have all leveraged this architecture.

Guided by scaling laws [15], model performance improves consistently as model parameters, computational budgets, and data size increase. This growth facilitates the emergence of advanced capabilities, such as zero-shot and few-shot learning, which are typically absent in smaller models below a certain scale. Brown et al. [16] observed that few-shot performance improves more rapidly as model size increases compared to zero-shot performance, highlighting that larger models possess stronger in-context learning capabilities.

To effectively harness the emergent capabilities of LLMs, *Prompt Engineering* has become a critical discipline to guide the model's output distribution toward desired outcomes. Much of its efficacy is rooted in *In-context learning (ICL)* [17], which allows the model to learn from examples or instructions embedded in the prompt without updating weights.

Many prompting techniques have been proposed to enhance the performance of LLMs [18]: (1) *Few-shot prompting* provides the model with a limited number of input-output examples as context, helping it induce an understanding of the given task, recognize task-relevant patterns and respond in appropriate formats. (2) *Zero-shot Prompting* relies solely on task descriptions or instructions without any examples, requiring the model to generalize from its pre-trained knowledge. Common strategies, such as *Role-playing* (e.g., *You are a senior software engineer.*) and *Emotion Prompting* (e.g., *It is really important to my career.*), are also widely used. (3) *Chain-of-Thought (CoT) prompting* encourages the model to generate intermediate reasoning steps, improving performance on complicated tasks that require multi-step reasoning. Related techniques such as *Tree-of-Thoughts (ToT)* and *Graph-of-Thoughts (GoT)* extend this idea by structuring the reasoning process, enabling systematic exploration of multiple reasoning paths. Together, all

of these techniques have been shown to mitigate hallucinations and improve the accuracy of complex tasks, and have substantially broadened the applicability of LLMs, enabling them to tackle increasingly complex tasks more reliably.

2.1.2 Automated Workflows and Autonomous Agents

LLMs' comprehension and reasoning skills make them compelling candidates for the central "brain" or general-purpose controllers within intelligent agentic systems. Based on their architectural design, these systems can be broadly categorized into two types [2]:

- **Workflows** coordinate LLMs and tools through a sequence of predefined, explicit steps. This design offers predictability and consistency, making it suitable for well-defined tasks.
- **Agents** allow the LLM to determine how to accomplish tasks by organizing the processes and tool usage on its own. This design provides flexibility to revise strategies in response to real-time feedback and adapt to complex and dynamic environments.

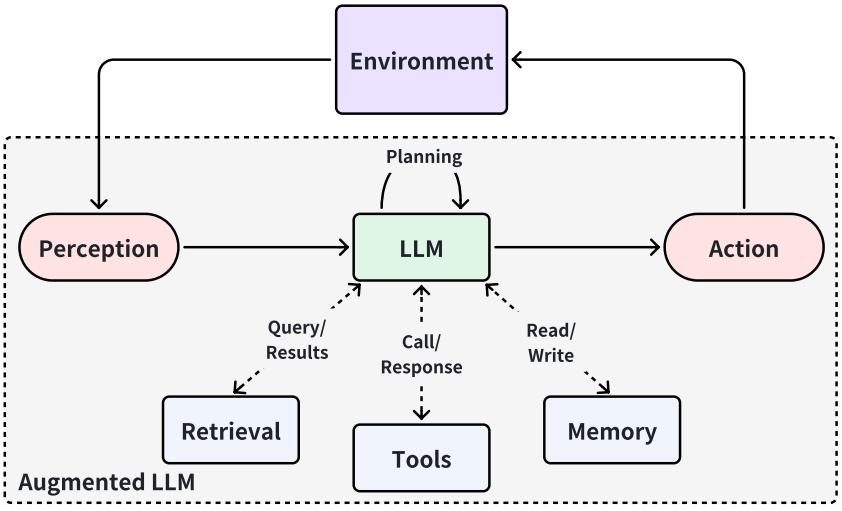


FIGURE 2.2: Augmented LLM-based agentic system [2].

As depicted in Figure 2.2, the core block of an agentic system is an augmented LLM equipped with capabilities such as retrieval, tools, and memory. Retrieval (e.g.,

Retrieval Augmented Generation [19]) helps the LLM access external knowledge bases to reduce hallucinations. Tool Use (e.g., *Function Calling* [20] and *Model Context Protocol* [21]) acts as the "hand" of the agentic system, enabling the LLM to perform actions and interact with the real world. Memory (e.g., short-term context and long-term storage [22]) allows the LLM to persist, recall and reason over historical information.

This agentic system can perceive changes from the environment and execute actions to influence it in return. Within this loop, the LLM serves as the central intelligent engine that understands the state of the environment, queries necessary information, selects appropriate tools, and determines what information to retain in memory at each step. By leveraging frameworks like *ReAct* [23] to integrate reasoning and acting or *Reflexion* [24] to enable self-reflection with dynamic memory, LLMs gain stronger reasoning and planning abilities, enabling more autonomous decision-making.

2.2 Formal Methods in Software Analysis

2.2.1 Symbolic Execution and SMT Solver

Symbolic execution is a formal program analysis technique powered by constraint solving [25]. It explores multiple execution paths of a program by treating program inputs as symbolic representations rather than concrete values. Formally, symbolic execution maintains a pair of *symbolic state* and *path conditions* $< \sigma, \pi >$, where $\sigma : \text{Var} \rightarrow \text{Expr}$ is the mapping from variables to symbolic representations, and π is a conjunction of constraints that must hold for execution to follow the current path. At the beginning, σ is initialized as an empty map and π is set to true. Every time a conditional branch (e.g., an if-statement *if(e)*) is encountered, the engine forks: one adds $\sigma(e)$ to π (true branch) and the other adds $\sigma(\neg e)$ to π (false branch). An execution path is considered feasible if and only if its accumulated path condition π is satisfiable. When a path terminates or reaches a target assertion, a constraint solver is invoked to determine the satisfiability. If π is satisfiable, the solver provides a concrete set of input assignments that guarantees the program will follow that exact execution path.

```
1  void foo(int a, int b) {
2      int x = a + b;
3      if (x > 11) {
4          if (a > 3) {
5              assert(false);  // bug
6          }
7      }
8  }
```

FIGURE 2.3: A simple C program.

Consider the simple C program in Figure 2.3. Symbolic execution begins with symbolic inputs α and β for a and b respectively, and maps x to $\alpha + \beta$. Upon reaching the first if-statement, it forks into two paths: the true branch records $\pi_1 = (\alpha + \beta > 11)$ and the false branch records $\pi_2 = \neg(\alpha + \beta > 11)$. Continuing along π_1, the execution forks again at the second if-statement. The true sub-path accumulates $\pi_{11} = (\alpha+\beta > 11) \wedge (\alpha > 3)$, which is then submitted to the constraint solver upon reaching the assertion. Since π_{11} is satisfiable (e.g., $\alpha = 4$ and $\beta = 8$), symbolic execution reports a feasible bug-triggering path along with a concrete counterexample. Similarly, the remaining paths π_2 and π_{12} are checked and found to be safe.

While symbolic execution is theoretically robust, its practical application is often hindered by challenges such as *path explosion*, modeling of complex data structures, and environmental interaction problems. To address these limitations, *concolic execution* was introduced as a hybrid approach that utilizes concrete execution to drive symbolic execution. Rather than forking at every branch, concolic execution never forks. It executes the program with concrete inputs while simultaneously tracking symbolic constraints. After a concrete run is completed, the engine identifies a branch condition along the path, negates it, and asks the SMT solver for a new set of concrete inputs that satisfy this modified constraint. Tools such as DART [26], CUTE [27], and KLEE [28] have demonstrated the effectiveness of this approach across large real-world codebases.

The efficiency of symbolic execution heavily depends on the underlying constraint solver, which is typically an SMT (Satisfiability Modulo Theories) solver such as Z3 [29] and CVC5 [30]. SMT solving is the process of determining the satisfiability of first-order formulas. It generalizes the Boolean Satisfiability (SAT) problem, which concerns the satisfiability of propositional logic formulas, by introducing background theories such as linear arithmetic, bit-vectors, and arrays. During

symbolic execution, each path constraint is represented as a conjunctive formula, and submitted to the SMT solver as a satisfiability query. If it is satisfiable (SAT), the solver returns a model that assigns concrete values to the symbolic variables; if it is unsatisfiable (UNSAT), it indicates that the execution path is unreachable and can be pruned safely.

2.2.2 Program Facts and Datalog

Program facts are a structured representation of information extracted from source code for the purpose of automated reasoning and analysis. They encode observable properties of a program, such as the existence of entities (e.g., functions, classes, variables), their attributes (e.g., names, locations, modifiers), and relations between them (e.g., containment, calls, inheritance), in a form suitable for systematic querying and inference.

Program facts are derived mechanically from source code through language-specific frontends, typically by parsing the code and traversing intermediate representations such as abstract syntax trees or control-flow graphs. Each extracted fact captures a single, well-defined aspect of the program, and together they provide a precise, machine-readable abstraction of the program's structure and behavior. Importantly, program facts are *descriptive*: they record what is present in the program, rather than how analyses should be performed.

2.2.2.1 Datalog

Datalog is a declarative logic programming language rooted in first-order logic and database theory. A Datalog specification consists of a set of rules that describe how new facts can be derived from existing ones.

Definition 2.1 (Datalog Rule)**.** A Datalog rule has the form:

$$R_0(t_1, \ldots, t_k) \leftarrow R_1(u_1^{(1)}, \ldots, u_{m_1}^{(1)}), \ldots, R_n(u_1^{(n)}, \ldots, u_{m_n}^{(n)}) \tag{2.2}$$

where each R_i is a predicate symbol. The atom on the left-hand side is called the *head*, and the atoms on the right-hand side form the *body*. Each argument t_j or $u_j^{(i)}$ is either a constant or a variable.

The rule is interpreted as follows: for any assignment of variables to constants that makes all body atoms simultaneously true, the corresponding instantiated head atom is also true. Variables thus serve as placeholders that allow a rule to match and relate multiple facts.

Definition 2.2 (Datalog Program)**.** A Datalog program is a finite set of Datalog rules evaluated over a given set of ground facts. Its semantics is defined as the least fixpoint of rule application: rules are repeatedly applied to derive new ground facts until no further facts can be inferred.

Datalog supports recursion and operates under a monotonic, set-based semantics, making it well suited for expressing transitive and structural properties such as reachability, dependency propagation, and hierarchical relations in program analysis.

2.2.2.2 Program Facts in Datalog

In program analysis, program facts are represented in Datalog as *ground predicate instances*. Each predicate schema corresponds to a specific kind of program entity or relation, while each extracted program fact instantiates that schema with concrete constants derived from the source code.

For example, a predicate describing function definitions may be declared as:

```
1  .decl function_definition(file_path: symbol, function_name: symbol,
   ↪  start_line: number, end_line: number, param_count: number, is_async:
   ↪  symbol, containing_class: symbol)
```

An extracted function definition in the source code gives rise to a ground fact of this predicate, with all arguments bound to concrete values such as file paths, names, and line numbers.

Datalog rules operate over these ground program facts using variables to range over matching predicate instances. The body of a rule specifies patterns over existing program facts, while the head defines a new fact to be derived whenever the body is satisfied under some variable assignment. Through repeated rule application, Datalog derives higher-level program properties—such as reachability, dependency

relations, or structural patterns—from the underlying set of extracted program facts.

This representation cleanly separates *fact extraction*, which records concrete observations about the program, from *logical inference*, which declaratively specifies how additional properties are derived.

2.3 Neuro-Symbolic AI

The pursuit of artificial intelligence has been a relentless human endeavor, historically divided into two main categories: *Symbolic AI* and *Sub-symbolic AI*. Symbolic AI represents knowledge through formal, declarative symbols based on rules and logic, and leverages these symbolic representations to reason about and solve complex problems. While this paradigm offers strong interpretability and verifiability, it struggles with scalability and flexibility when confronted with the complexity of the real world, as exhaustively encoding all relevant domain knowledge into hand-crafted rules proves intractable in practice.

Subsequently, sub-symbolic AI emerged, placing artificial neural networks at its core. Rather than explicit rules, this paradigm relies on algorithms to extract statistical patterns from data, identify latent relationships, and make predictions through learned representations. Exemplified by machine learning, deep neural networks, and generative models, sub-symbolic AI has achieved remarkable success in areas such as pattern recognition and natural language processing. Although large-scale models have been observed to exhibit reasoning-like behaviors brought by emergent ability, they still lack explicit logical structure, making rigorous reasoning and formal verification fundamentally difficult.

Neuro-symbolic AI draws inspiration from the System 1 and System 2 theory [31]. System 1 is fast, intuitive, and parallel, akin to the capabilities of deep learning, while System 2 is slow, deliberate, and sequential, resembling symbolic reasoning. Neuro-symbolic AI seeks to integrate these two approaches to create systems that leverage the strengths of both and ultimately produce a superior hybrid AI model possessing reasoning abilities. In the context of software engineering, as illustrated in Figure 2.4, there are two main neurosymbolic approaches [3] to improve logical reasoning:

1. **Integrative approach.** Symbolic reasoning is embedded within the neural network.

2. **Hybrid approach.** Symbolic reasoning is performed by a separate symbolic solver, distinct from the neural network.

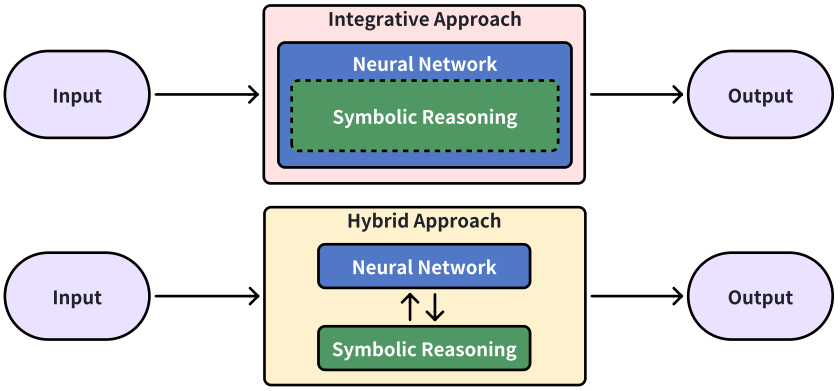


FIGURE 2.4: Integrative and hybrid neurosymbolic approaches [3].

The hybrid approach demonstrates greater potential for advancing general logical reasoning, as it preserves the capabilities and advantages of existing LLMs while providing a more interpretable reasoning chain.

Chapter 3

Library Upgrade Incompatibility Issues

3.1 Introduction

Modern software systems are becoming increasingly complex due to the need for integrating various components developed by different teams or organizations. These components are often subject to continuous evolution, and as a result, ensuring that new upgrades to third-party libraries do not cause any compatibility issues with the existing software system is a challenging task. The complexity of these systems and the number of dependencies involved make it difficult to anticipate and identify incompatibilities that may arise from updates to external components. Incompatibility issues resulting from upgrades to external components can compromise the reliability of software systems, potentially leading to significant financial losses for the organizations that rely on these systems.

Many techniques have been proposed to address third-party library compatibility issues, including regression testing [32, 33], static analysis [34], dependency conflict detection [35], and client-specific compatibility checking [36, 37]. These techniques address library compatibility issues in different dimensions and have been evaluated with their own isolated datasets.

In recent years, Large Language Models (LLMs) have achieved remarkable success across numerous automated software engineering tasks. At the heart of this

progress lie high-quality open-source datasets, which serve as the cornerstone of model capabilities. These datasets provide essential domain-specific training material for learning real-world complex software patterns. Furthermore, the precision of their annotations and the reproducibility of their instances directly impact the performance upper bound of AI tools on various software engineering tasks. For example, *Defects4J* [38] contains over 800 reproducible bugs drawn from 17 open-source Java projects, enabling evaluation of whether AI can generate patches that pass test suites; *CoRe* [39] includes more than 10,000 task instances covering data dependency, control dependency, and information flow across multiple programming languages for evaluating the ability of fundamental static analysis; *SWE-bench* [13] provides an end-to-end benchmark comprising a large collection of real GitHub issues, designed to assess the end-to-end capability of AI in resolving software engineering problems.

However, these datasets mainly focus on internal problems of projects, leaving largely uncovered risks introduced by external changes. Although existing LLM-based tools have demonstrated impressive performance in code comprehension, they face substantial challenges in understanding and reasoning about compatibility issues induced by library upgrades, severely limiting their capacity to manage external evolutionary risks. This limitation stems from two interrelated factors. First, library upgrade compatibility issues are inherently complex, involving multiple components and intricate dependency relationships. Current LLM reasoning is often insufficient to model cross-component interactions or infer the impact of changes. Second, such issues are difficult to detect and some of them may only be triggered in specific environments and test cases. Additionally, constructing reproducible problem instances requires non-trivial configuration, which further exacerbates the scarcity of data. Consequently, a structured, high-quality, and fully reproducible dataset of real-world library upgrade incompatibilities is essential for enhancing and evaluating LLM-based tools in this domain.

An excellent dataset has the potential to serve as a valuable reference for future research in this field. However, composing the dataset requires intricate manual validation, e.g., confirming whether the cause of a test failure is due to a runtime exception, an assertion violation, or other reasons. Therefore, this chapter presents CompSuite, the first incompatibility issue dataset focusing on library behavioral

incompatibility with concrete reproducible test cases. Each test case is isolated and validated, enabling the direct manifestation of the incompatibilities.

CompSuite comprises 123 real-world Java client-library pairs such that upgrading any library results in incompatibility issues for the corresponding client. Every incompatibility issue in CompSuite contains a test case created by developers, allowing for the reproduction of the issue. On top of this dataset, an automated command-line interface was also developed, which streamlines all processes of the reproduction, such as downloading and compiling a project, running target tests and rerunning the tests after a library upgrade. With this infrastructure, users may reproduce an incompatibility issue programmatically with minimal effort.

Contribution. The contributions of this chapter are as follows:

1. A dataset, CompSuite, is constructed, including 123 reproducible, real-world client-library pairs that manifest incompatibility issues when upgrading the library. These data points originate from 88 clients and 104 libraries.
2. An automated command-line interface was created for the dataset. With this interface, users are able to programmatically replicate an incompatibility issue from the dataset with a single command. The interface also offers separate commands for each step involved in the reproduction of incompatibility issues.

CompSuite is intended to support the evaluation of various program analysis techniques, including compatibility checking, module-level regression testing selection, and debugging techniques. Furthermore, this is also very high-quality and valuable data for AI, which can help models better understand this type of issue. More detailed information can be found in Section 3.5.

The dataset and tool are available at: `https://github.com/compsuite-team/compsuite`.

3.2 Client-specific Upgrade Incompatibility

This section provides a more in-depth examination of the issue of client-specific upgrade incompatibilities.

Ensuring compatibility during library upgrades is challenging because third-party component specifications are generally limited to descriptions of their Application Programming Interfaces (APIs). Even when an upgrade does not modify APIs, it may still introduce behavioral changes that compromise the stability of dependent client applications [40]. Such behavioral incompatibilities are particularly problematic because they are often undetectable through static analysis.

Empirical studies highlight both the prevalence and severity of this problem. A study examining 408 sampled open-source Java projects reported that 202 of them experienced failures following library upgrades, with 41.3% of these failures manifesting as runtime test failures that could not be caught by the compiler [41]. These incompatibilities are hard to identify statically, and there is no straightforward and easy way to evaluate their ripple effects on the entire system. Consequently, managing third-party software package upgrades is a daunting task, and many developers avoid it as much as possible due to the associated risks and costs. Another empirical study indicates that 81.5% of surveyed systems retain outdated dependencies [42]. With respect to security patches, 69% of interviewed developers reported being unaware of vulnerabilities in the libraries they use. Furthermore, due to the additional efforts and responsibilities associated with upgrading, developers are often reluctant to perform upgrades. This hesitation raises serious security concerns and may leave their systems vulnerable to zero-day attacks. In the context of expanding reliance on AI-assisted development tools, these issues may intensify and become progressively harder to manage.

The decision of whether to upgrade thus becomes a practical dilemma. Fundamentally, an *upgrade incompatibility issue* arises from a mismatch between two independently evolved software entities: the client and the library. Compatibility checking has been extensively studied from multiple perspectives. Existing approaches can be categorized based on two dimensions: (1) whether they target client developers or not (i.e., *client-oriented*), and (2) whether they treat different client applications separately when considering the impacts from a library upgrade (i.e., *client-specific*).

As shown in Figure 3.1, the example illustrates a client-specific incompatibility issue. The function `lib(s1, s2)` is a library function that takes two input parameters (`s1` and `s2`) and returns their longest common substring. In the upgraded version (v1), `lib(s1, s2)` returns `null` when the two inputs share no common

substring, whereas in the original version (v0), it returns an empty string (`""`) under the same condition. Such a change may cause a behavioral incompatibility in Client A (potentially triggering a `NullPointerException`), while Client B is not affected.

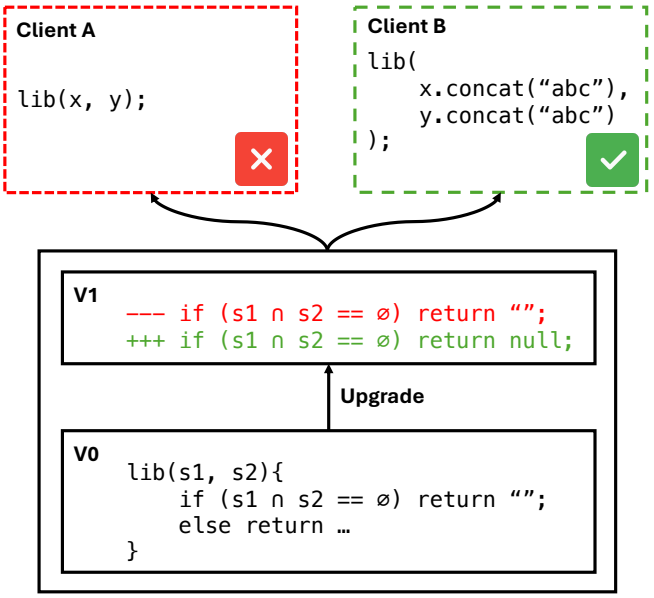


FIGURE 3.1: A client-specific incompatibility issue [4].

Client-oriented techniques are designed from the perspective of client developers and aim to provide information and actionable insights regarding detected incompatibilities. For instance, the smart alert mechanism [43] raises an alarm when a library upgrade may affect existing functionality. Framework and API migration techniques, such as AURA [44], generate transformation rules that map target API methods in earlier versions to their corresponding replacement methods in newer versions. In contrast, library-oriented techniques, such as DeBBI [45], focus on assisting library developers in improving testing and code quality, thereby reducing the likelihood of introducing breaking changes that affect downstream clients upon release.

It is worth noting that not all client-oriented techniques are truly client-specific. For instance, package compatibility analysis tools such as Dependabot [46] and GemChecker [47] provide upgrade recommendations based on historical records of dependency failures. While these approaches benefit from empirical insights derived from prior upgrade incidents, their definitions of compatibility are too

generic. They do not incorporate information about how specific client projects actually interact with the upgraded libraries.

By contrast, regression testing [48] operates within the specific context of a given client application and can be used to validate modifications to client-side code. Nevertheless, regression testing frequently fails to identify incompatibilities that arise from library upgrades. This shortcoming stems from the fact that regression testing suites are constructed by client developers, who often lack comprehensive knowledge about library modifications.

3.3 Dataset Creation

This section outlines the methodology and process employed to create the CompSuite dataset.

Table 3.1: Details of clients and libraries included in CompSuite.

Client	**#LoC**	**#Star**
retrofit	29.7K	41.5K
apollo	61.3K	28K
druid	441.9K	26.8K
webmagic	17.4K	10.8K
languagetool	171.2K	8.5K
Other 83 clients (mean)	371.6K	1.3K
All clients (mean)	358.7K	2.5K

Library	**#Maven Usage**
org.slf4j:slf4j-api	62.5K
com.google.guava:guava	34.4K
org.scala-lang:scala-library	34K
com.fasterxml.jackson.core:jackson-databind	25.8K
ch.qos.logback:logback-classic	25.5K
Other 99 libraries (mean)	3.2K
All libraries (mean)	4.8K

3.3.1 Subject Selection

To ensure the representativeness and reproducibility of the CompSuite dataset, the selection process focused on high-quality and popular client projects and libraries. The selection of client projects was sourced from GitHub [49], a widely recognized online community for hosting open-source codebases. To ensure the inclusion of the most popular projects, all the available projects were systematically sorted in descending order based on their number of stars on GitHub and the target clients were selected from the top of the list. The selection of libraries was sourced from Maven Central [50], which hosts 33.5M Java libraries and their associated binaries, making it a widely used repository of libraries for Java API and library research [51–54]. A library was included in the dataset only if it had more than 100 usages (i.e., clients) on Maven Central. The selection criteria aimed to ensure the inclusion of popular and widely used client projects and libraries in the dataset, thereby maximizing its relevance and usefulness to the research community.

Among the highly-rated client projects, the selection criteria focused on those that use Maven [55] as their build systems, given its widespread adoption and maturity. Maven provides a standardized approach to managing Java projects and their dependencies, where each library dependency in a Maven client project is represented as an item in a `pom.xml` file, making it easy to identify and edit library versions programmatically. Furthermore, Maven offers built-in functionality for running unit tests and generating test reports, which simplifies the identification and diagnosis of incompatibility issues arising from test executions. Since Maven projects typically rely on Maven Central as their centralized repository for hosting and downloading libraries, the process of obtaining and managing libraries in the dataset is simplified.

Table 3.1 presents the top 5 client projects and libraries in the CompSuite dataset, ranked by popularity. For each client project, the table provides its lines of code (LoC) and number of GitHub stars; for each library, it lists its number of usages by other projects from Maven Central.

In total, CompSuite comprises 123 incompatible client-library pairs. These pairs encompass 88 distinct clients and 104 libraries altogether. On average, the affected clients have 2.5K stars on GitHub and 358.7K lines of code, while incompatible libraries have 4.8K usages on Maven Central. These statistics suggest that the

incompatibility issues present in the CompSuite dataset have a significant impact on a large number of codebases and can affect many users of the libraries, either directly or indirectly.

To ensure that all client projects in the dataset are executable and the runs are reproducible, each project underwent a series of checks. First, the project was checked out to the version (SHA) at the time of the dataset creation, referred to as the *base version*. Next, the standard Maven project compilation command was executed to verify whether the project could be compiled successfully. Projects that failed to compile were excluded from the dataset. Subsequently, the standard Maven test command was executed to run all the tests in the project, ensuring that all tests passed on the base version. Any project that failed to pass tests at this stage was excluded. Finally, only client projects that successfully compiled and passed all tests on the base version were included, thereby ensuring that the dataset consists only of projects which can be executed and whose executions can be reproduced.

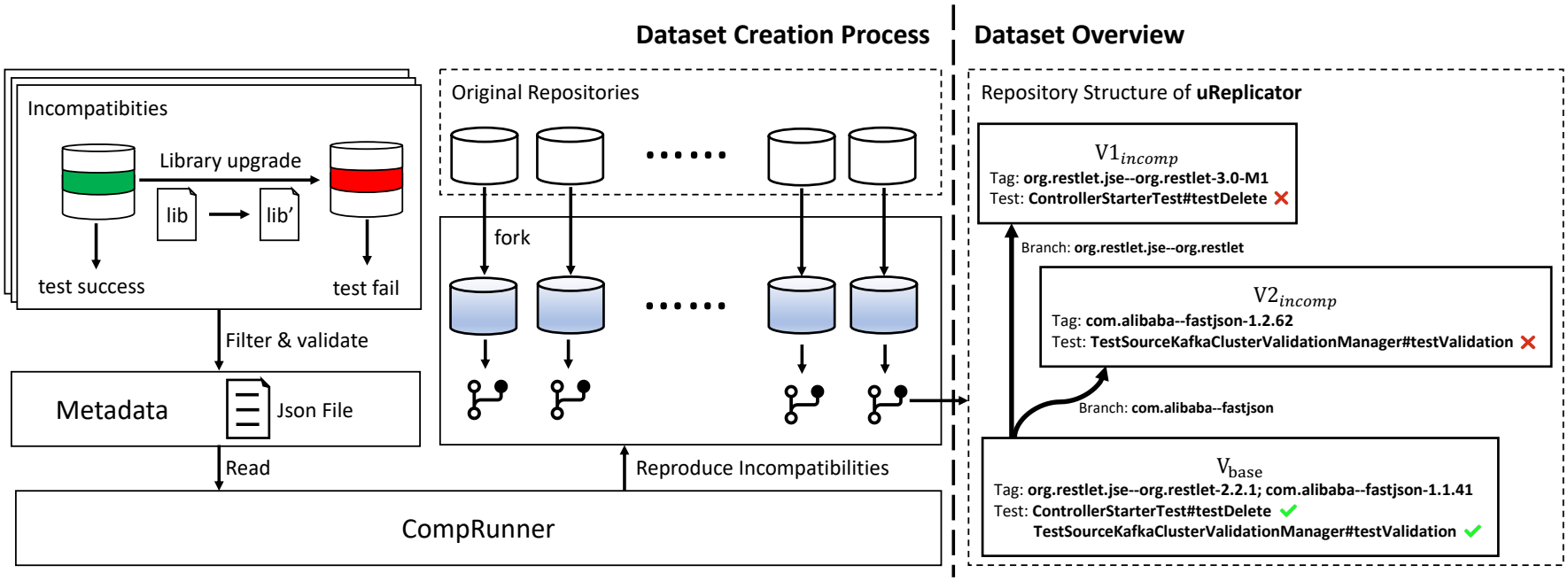


FIGURE 3.2: The architecture of COMPSUITE.

3.3.2 Data Collection

The data were collected using the following procedures. Figure 3.2 visualizes the overall architecture of CompSuite. The upper left portion of Figure 3.2 illustrates the approach taken by CompSuite to identify incompatibilities between a client project and its dependent libraries. Specifically, for each client project on its base version, each dependent library was upgraded individually, and tests were run to determine whether the upgrade caused any failures. The intuition behind this approach is that since all the tests in the client passed on the base version, if upgrading any library causes a test failure, that library upgrade must have introduced incompatibility issues. A test that flips from passing to failing is termed an *incompatibility-revealing test*.

To automatically upgrade the libraries and run the tests, the data collection process used the Maven Versions Plugin [56]. For a given client project, its dependency list was scanned using this plugin to identify all the libraries that had newer versions available on Maven Central. If a library had a newer version, it was marked as upgradable. Next, for each upgradable library, the plugin was used to upgrade it by updating the `pom.xml` file to the most recent version on Maven Central. The test suite of the client was then re-executed. If any tests failed during this run, the client-library pair was marked as having an incompatibility issue, and the test was marked as an incompatibility-revealing test of this issue. It is crucial to note that only one library was upgraded at a time to isolate failures caused by different libraries. To ensure the accuracy and dependability of the dataset, each identified incompatibility issue was manually verified. In particular, the test failure messages and reports were carefully examined to confirm that they were indeed caused by the upgraded library. For each incompatible client-library pair, a single incompatibility-revealing test was selected for inclusion in the final dataset. In cases where a client-library pair had multiple incompatibility issues, the issue judged most representative and easy to comprehend was selected.

Finally, the metadata of all the selected incompatibility issues were persisted in a collection of `json` files. Figure 3.3 presents the metadata of an incompatibility issue in the CompSuite dataset. The data schema includes the ID of the issue, client project name, SHA of the client base version, URL of the client project, library name, versions of the old and new libraries, the name of the incompatibility-revealing test, the submodule containing the incompatibility-revealing test, and

```
{
  "id": "i-49",
  "client": "wasabi",
  "sha": "9f2aa5f92e49c3844d787320e2d22e15317aa8e2",
  "url": "https://github.com/intuit/wasabi",
  "lib": "org.apache.httpcomponents:httpclient",
  "old": "4.5.1",
  "new": "4.5.10",
  "test": "DefaultRestEndPointTest#testGetRestEndPointURI",
  "submodule": "modules/export",
  "test_cmd": "mvn
  ↪  org.apache.maven.plugins:maven-surefire-plugin:2.20:test -fn
  ↪  -Drat.ignoreErrors=true -DtrimStackTrace=false
  ↪  -Dtest=DefaultRestEndPointTest#testGetRestEndPointURI"
}
```

FIGURE 3.3: The data schema of COMPSUITE.

the command to run the test. The majority of the information is self-explanatory. However, it is worth noting that the old version of the library is the one utilized at the base version of the client, while the new version is the most recent version found on Maven Central that triggers the incompatibility when upgrading, as described in Section 3.3.2.

3.4 Dataset Usage

This section provides instructions on the usage of the dataset.

3.4.1 Exploring an Incompatibility Issue

To ensure the reproducibility of incompatibility issues and to facilitate the demonstration of such issues, the dataset includes annotated checkpoints in the version histories of the client projects and tags that guide users to explore any incompatibility issues present in the COMPSUITE dataset.

As illustrated on the right-hand side of Figure 3.2, the approach to handling incompatible client-library pairs involved creating a fork [57] of the original client project for each identified pair, while preserving all code and version history information. To mark the base version of the project, the `gittag` [58] command was used to

designate the base version as V_{base}. Subsequently, a patch was developed to upgrade the library from its old version to its new version, a simple process that can be accomplished with a single line change in the `pom.xml` file for Maven projects. This patch was then applied to the V_{base} version, resulting in a new version identified as V_{incomp}. Notably, the only difference between V_{base} and V_{incomp} lies in the library version used: the old (compatible) version is utilized on V_{base} while the new (incompatible) version is utilized on V_{incomp}. For instance, in Figure 3.2, the client project employs version 2.2.1 of the `org.restlet.jse` – `org.restlet` library on its V_{base} and version 3.0-M1 on its V_{incomp}. In cases where multiple libraries exhibit incompatibility issues in the client project, a separate branch named after each library is created, along with a V_{incomp} version tag for each, with accompanying annotations that denote the corresponding library name and version, as depicted in Figure 3.2.

The V_{incomp} tag for each client-library pair also specifies the specific test that can reveal the incompatibility issue during its run. Following Maven's convention, the test name is formatted as `TestClassName#testMethodName`. By simply copying the text from the tag, users can easily run the incompatibility-revealing test on the V_{incomp} version and observe the incompatibility issue. On the V_{base} version, all tests should pass. This design aims to simplify the usage of CompSuite and make it more accessible and user-friendly.

Using the forked client repositories and version tags provided in the CompSuite dataset, users can easily reproduce any incompatibility issue by checking out to V_{incomp} and running the corresponding incompatibility-revealing test. To compare the behaviors of the client with compatible and incompatible library versions, users can run the incompatibility-revealing test on both V_{base} and V_{incomp} and compare the test outcomes. This allows for a clear understanding of the impact of the library upgrade on the client behaviors.

3.4.2 CompRunner: An Automated Tool for Reproducing Incompatibility Issues

An automated tool, named CompRunner, was also developed as part of CompSuite (shown at the bottom of Figure 3.2). With CompRunner, users can easily

reproduce and investigate any incompatibility issue in a one-click manner by providing the issue ID as input.

CompRunner offers an option which enables users to reproduce an incompatibility issue end-to-end with a single command as is shown below. The command outputs and saves all intermediate results and logs for future reference.

```
1 python main.py --incompat i-56
```

When CompRunner runs, it clones the client project from the forked code repository and saves it in the output directory (which is configurable). Then, it checks out to the base version, compiles the code, and runs the incompatibility-revealing test. Next, it upgrades the library to the new version, reruns the incompatibility-revealing test, and reports any failure information to the user.

CompRunner also provides a set of commands that break down the entire cycle of incompatibility exploration into separate steps:

```
1 python main.py --download i-56
2 python main.py --compile i-56
3 python main.py --testold i-56
4 python main.py --testnew i-56
```

Several other CompRunner commands are available for users to inspect different aspects of the incompatibility issues from the CompSuite dataset. A complete list of these commands can be found on CompSuite's website at `https://github.com/compsuite-team/compsuite`.

3.5 Application Scenarios

It is anticipated that both researchers and practitioners can benefit from CompSuite to facilitate their investigations and research on errors and test failures induced by library upgrades. CompSuite supports the evaluation of various program analysis techniques, such as software upgrade compatibility checking, debugging, and module-level regression test selection techniques.

As an overview, authors of compatibility checkers and detectors may use CompSuite as a benchmark to evaluate the performance of their techniques against other baseline approaches. Furthermore, authors of debugging techniques can utilize CompSuite as a dataset of compatibility bugs, where each bug corresponds to a test case that verifies the existence or absence of the bug. Finally, authors of module-level regression test selection techniques can use CompSuite to assess the safety of their approaches. A safe module-level RTS technique should select all the corresponding incompatibility-revealing test cases when the library changes.

The usage scenarios are detailed as follows.

- **Compatibility Checkers and Detectors.** The existing techniques for compatibility checking and detection in Java can be categorized into three groups: i) Techniques for detecting API incompatibility that focus on detecting API-breaking changes, such as renaming of code entities and changes in parameter types [59–63]. ii) Techniques for detecting behavioral incompatibility that focus on identifying behavioral differences that cause test failures when a library is upgraded in a client, such as changes in program states [4, 36]. iii) Techniques for detecting dependency conflicts [35, 64, 65], which aim to identify library APIs that exhibit inconsistent semantics between libraries due to class path shading. This study takes the view that developers of techniques in the first two categories can use CompSuite as a benchmark to evaluate their tools' performance, such as precision and recall. They can run their tools on the CompSuite dataset and compare the results with the incompatibility issues present in the dataset. On the other hand, developers of techniques for detecting dependency conflicts can slightly modify CompSuite's dataset by placing both old and new libraries on the class path, running library conflict detection, and checking if the issues can be detected.

- **Module-Level Regression Test Selection.** Regression test selection (RTS) is a technique that aims to reduce the cost of regression testing by selecting a subset of tests that may change the behavior due to code changes on each program version [37, 66–68]. Module-level RTS focuses on selecting the affected client tests when a dependent library is updated [41]. The developers of module-level RTS techniques can evaluate the safety of their tools using CompSuite. For each client-library pair in CompSuite, a module-level RTS tool should select

all the corresponding incompatibility-revealing tests when upgrading the library from the old version to the new version.

- **Debugging.** The existing debugging techniques for Java include symbolic execution [25], delta debugging [69], fault localization [70], etc. These techniques aim to identify the root cause of errors or failures in software. Developers of debugging techniques can use CompSuite as a dataset of compatibility bugs, where each compatibility bug corresponds to a test case that checks the presence or absence of the bug. They can use CompSuite to evaluate their techniques' ability to perform root cause analysis by trying to identify the corresponding library change that caused the compatibility issue.

- **Benchmark.** With the growing integration of AI in software engineering, CompSuite is also envisioned as a robust benchmark to evaluate the performance of advancing AI-driven techniques in diagnosing, locating, and resolving compatibility issues. For example, developers of AI tools can utilize CompSuite to quantify the detection rate of LLM-based compatibility checkers or the pass rate of generated patches.

3.6 Related Work

To cater to the requirements of various research endeavors, numerous outstanding datasets have been made available to date. Just et al. [38] introduced Defects4J, a database that supplies actual bugs, fixed program versions, and corresponding test suites. Bui et al. [71] introduced Vul4J focusing on Java vulnerabilities. Jezek et al. [72] released their dataset of compatibility issues arising from program evolution. There are also many datasets that cater to other research domains and ecosystems [73–75].

Distinct from the previously discussed datasets, CompSuite is the first dataset that emphasizes the incompatibility issues caused by Java library behavior changes. These issues are prevalent and difficult to detect. Additionally, the automated tools developed in this study also have the capability to assist researchers in swiftly reproducing issues. A dataset targeting the library upgrade incompatibility issue is expected to contribute to the advancement of the associated technologies.

3.7 Summary

This chapter presents CompSuite, a dataset containing 123 real-world Java client-library pairs where library upgrades cause compatibility issues in the corresponding clients. On top of it, a command-line interface, CompRunner, was developed, which allows users to quickly check incompatibility issues with a single command or reproduce an incompatibility programmatically for in-depth analysis. Various program analysis techniques like library compatibility checking, debugging, and regression test selection may benefit from the dataset.

Chapter 4

Identifying Multi-parameter Constraint Errors in Documentation

4.1 Introduction

Machine learning (ML) and Artificial Intelligence (AI) have consistently garnered widespread attention, achieving remarkable breakthroughs in diverse domains including natural language processing, recommendation systems, autonomous vehicles, and robotics. Behind the rapid advancement of these transformative technologies, data science and machine learning libraries play a crucial role in AI and ML development. By providing extensive APIs for complex mathematical operations and algorithmic implementations, these libraries enable researchers and practitioners to focus on solving domain-specific problems rather than reimplementing fundamental algorithms.

Well-designed API documentation not only provides detailed descriptions of interfaces, including the purpose and range of parameters or attributes, returns, and exceptions thrown, but may also specify logical constraints or dependencies among multiple parameters. For data science and machine learning libraries, multi-parameter constraints are commonly mentioned in their API documentation and users of these libraries are expected to follow them closely when using the APIs. However, frequent version updates may cause the documentation to become out of sync with the corresponding code, known as the **C**ode-**D**ocumentation **I**nconsistency (CDI) issue [76, 77]. Such CDI issues are particularly pronounced

in data science libraries. On one hand, the underlying mathematical models of DS/ML libraries inherently come with various constraints, such as *"a model X can only be chosen when a parameter Y is provided"*, and incorrect parameter configurations not satisfying their constraints may lead to unexpected outcomes. On the other hand, the number of parameters/attributes in the APIs of these libraries can be significantly greater than that in a typical library API, sometimes over a few dozen. Therefore, it is unrealistic to track all parameter constraints manually.

Detecting errors in multi-parameter constraints from Python API documentation is challenging for several reasons. (1) API documentation varies in quality and often does not follow standardized writing guidelines. Some API documentation uses ambiguous language, contains typos, and may not follow a consistent styling guide. This makes simple rule-based pattern-matching approaches ineffective. (2) Existing approaches [78, 79] for detecting documentation errors focus on a single parameter only: e.g., checking whether the information provided on parameter ranges, nullness, and identifier names is correct. It is more challenging to extract multi-parameter constraints precisely from free-style descriptions written in natural languages. (3) For the same reason, a semantics-aware code analysis approach is essential, as logical relations among multiple parameters cannot be easily identified through purely syntactic analysis. The challenge is further compounded by Python's dynamic nature, where variable types, attributes, and behaviors can change at runtime.

To detect multi-parameter constraint inconsistencies from data science library documentation, this chapter proposes an automated tool, MPChecker. MPChecker identifies inconsistencies between API documentation and the corresponding library code by combining symbolic execution-based program analysis techniques with constraint extraction methods powered by large language models (LLMs). MPChecker first extracts multi-parameter constraints from documentation (a.k.a. *doc-constraints*), leveraging the powerful natural language understanding capability of LLMs. The extraction process incorporates a few optimizations, such as Chain of Thought (CoT) [80] and few-shot learning to improve the accuracy of constraint extraction. Then MPChecker uses dynamic symbolic execution to collect all path constraints from the corresponding Python source code. The symbolic path constraints (a.k.a. *code-constraints*) capture the real constraints that

the parameters have to follow according to the library code, which are then used to evaluate the correctness of the *doc-constraints*.

Then, in order to mitigate minor discrepancies that may arise from the *doc-constraints* extracted by LLMs, a *Fuzzy Constraint Logic* (FCL) framework is designed and implemented to estimate how logically consistent a *doc-constraint* is with a set of given *code-constraints*. Intuitively, in the absence of LLM-induced unpredictability, a *doc-constraint* must be evaluated as true under the assumption of *code-constraints*. Through fuzzy constraint satisfaction, MPChecker can accommodate many *nearly-correct* constraints produced by LLMs and thus improve the accuracy of the overall approach.

Contributions. This chapter aims to integrate precise symbolic reasoning with the inherently fuzzy outputs of large language models. Specifically, this chapter makes the following contributions.

1. An automated multi-parameter code-documentation inconsistency detection technique was proposed, and an end-to-end command-line tool called MPChecker was developed. Existing techniques in the same area are only designed to handle single parameter inconsistencies, without considering inter-parameter constraints.

2. A customized fuzzy constraint satisfaction framework was introduced to mitigate the uncertainties introduced by LLM outputs. A theoretical derivation of the membership function based on constraint similarity is provided.

3. The study constructed a documentation constraint dataset comprising 72 real-world constraints sourced from widely used data science libraries, and derived a mutation-based inconsistency dataset with 216 constraints. The dataset and tool implementation are made available online: `https://github.com/ParsifalXu/MPChecker`.

4. MPChecker was evaluated on four real-world popular data science libraries. Fourteen inconsistency issues discovered by MPChecker were reported to the developers, who have confirmed 11 inconsistencies at the time of writing.

4.2 Background

This section reviews the essential terminology and background necessary for understanding the subsequent chapters.

4.2.1 Multi-Parameter Constraints

Two examples illustrate inconsistencies between API documentation and the corresponding code caused by multi-parameter interdependence. Both of them come from open-source Python data science libraries and were successfully detected by MPChecker. In general, there are two types of constraints found in the API documentation. (1) An *explicit constraint* clearly specifies the logical relationship among two or more interrelated parameters. (2) An *implicit constraint* is an unstated or indirectly implied relationship among two or more interrelated parameters, where the constraint is inferred through contexts or convention rather than explicitly specified.

4.2.1.1 Example 1: Explicit Constraint

The first example comes from `statsmodels` [81], which provides a complement to `scipy` for statistical computations including descriptive statistics and estimation and inference for statistical models. `Statsmodels` has more than 10K stars on GitHub and is actively maintained. Figure 4.1 illustrates an inconsistency caused by an explicit constraint from the class *AutoReg*. The relevant portions for the *doc-* and *code-constraints* are highlighted. As mentioned in the documentation of `deterministic`, the trigger condition for the warning is that "trend is not n, **and** seasonal is not False". However, it is apparent that the *code-constraint* for `trend` and `seasonal` (to be used together correctly and avoid any warning) implemented is **or** instead of **and**. One way to fix the documentation is to change "and" to "or".

4.2.1.2 Example 2: Implicit Constraint

The second example comes from `scikit-learn` [82], which is a widely-used (more than 60K stars on GitHub) open-source ML library in Python, designed to offer

Constraint description of `trend` and `seasonal` in class `AutoReg`

> **deterministic:** DeterministicProcess
> A deterministic process. If provided, trend and seasonal are ignored. A warning is raised if trend is not "n" and seasonal is not False.

Corresponding code snippet in class `AutoReg`

```
class AutoReg(tsa_model.TimeSeriesModel):
    def __init__(...):
        if deterministic is not None and
    (self.trend != "n" or self.seasonal):
            warnings.warn('When using deterministic, trend must be "n"
                and seasonal must be False.', SpecificationWarning,
    stacklevel=2)
```

FIGURE 4.1: Examples of an explicit constraint from `Statsmodels`.

simple and efficient tools for data mining and data analysis. Figure 4.2 displays an inconsistency caused by an implicit constraint from the class *SpectralClustering*. It is evident that the highlighted part of the documentation only explicitly mentions one parameter *affinity*, omitting the subject "gamma". More importantly, "ignore" is not a specific identifier or value but rather a description of the program logic—if the parameter *affinity* is set to *nearest_neighbors*, then the parameter *gamma* will not be used. However, the above constraint does not faithfully reflect the behavior implemented in code. According to the code snippet, "gamma" is not only ignored within the *nearest_neighbors* branch, but also ignored within the *precomputed_nearest_neighbors* and *precomputed* branches. This reveals that the documentation does not describe all branches in which `gamma` is ignored. The stated condition itself can still be sufficient, so the distinction between incompleteness and a violated implication matters when checking this example (Section 4.3.3.1).

For this type of implicit constraint, traditional pattern-based approaches are not able to extract the *doc-constraint* correctly and thus fail to detect the inconsistencies. To solve this issue, a customized constraint incorporating fuzzy words is designed, and few-shot learning is adopted to teach LLMs how to generate such constraints (details in Section 4.3.2.2). In this case, the *doc-constraint* should be *"(affinity = "nearest_neighbors") → (ignore(gamma))"*, where a special predicate *"ignore(x)"* is used to indicate that a parameter x is ignored (see Section 4.3.3.1).

Constraint description of `gamma` and `affinity` in class `SpectralClustering`

> **gamma** : float, default=10
> Kernel coefficient for rbf, poly, sigmoid, laplacian and chi2 kernels. **Ignored for affinity="nearest_neighbors".**

Corresponding code snippet in class `SpectralClustering`

```
class SpectralClustering(ClusterMixin, BaseEstimator):
    def fit(self, X, y=None):
        if self.affinity == "nearest_neighbors":
            ...
        elif self.affinity == "precomputed_nearest_neighbors":
            ...
        elif self.affinity == "precomputed":
            ...
        else:
            params = self.kernel_params
            if params is None:
                params = {}
            if not callable(self.affinity):
                params["gamma"] = self.gamma
                params["degree"] = self.degree
                params["coef0"] = self.coef0
```

FIGURE 4.2: Examples of an implicit constraint from `Scikit-learn`.

4.2.2 Fuzzy Logic and Fuzzy Constraint Satisfaction

Unlike traditional Boolean logic, fuzzy logic [83] is a multi-valued logic that allows for values between 0 and 1 to represent varying degrees of truth, where 0 represents complete falsity, and 1 represents absolute truth. The human brain can process vague statements or claims that involve uncertainties or subjective judgments, such as "the weather is hot", "that man runs so fast", or "she is beautiful". Unlike computers, humans possess common sense, allowing them to reason effectively in situations where things are only partially true. Fuzzy logic is primarily used to model uncertainty and vagueness, making it highly applicable in real-world scenarios where precision may be difficult or impossible to achieve.

A traditional constraint satisfaction problem (CSP) [84] requires all constraints to be fully satisfied. Constraints are either completely satisfied or unsatisfied, which is why these strict, non-fuzzy constraints are referred to as "*crisp constraints*". An

extension of CSP, known as soft CSP [85, 86], introduces a distinction between hard constraints and soft constraints. Hard constraints must be absolutely satisfied, while soft constraints are typically assigned a weight or priority, allowing for lower-weighted constraints to be only partially satisfied or even unsatisfied under certain conditions during problem-solving. Another extension is Fuzzy CSP [87], which differs from soft constraints in that it incorporates fuzzy logic and allows each constraint to be "partially satisfied" to a degree, quantified by a "satisfaction degree". This satisfaction degree usually ranges from 0 to 1, indicating the extent to which a constraint is fulfilled. The goal in fuzzy constraint satisfaction is to find a solution that maximizes satisfaction, rather than strictly satisfying all constraints.

4.3 Methodology

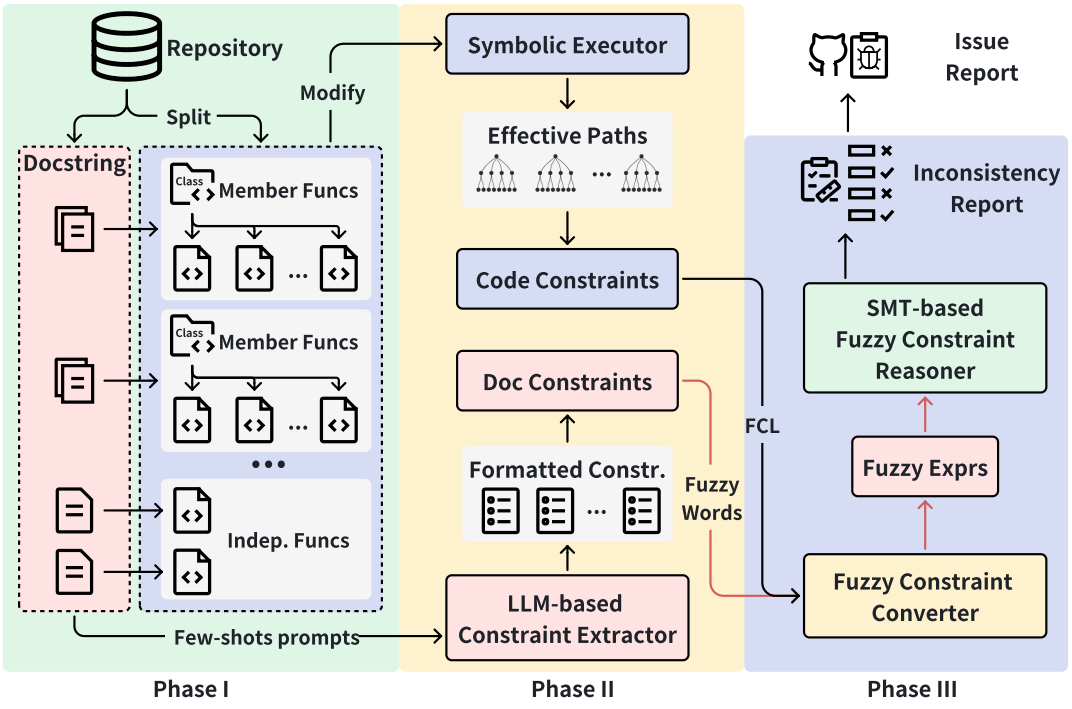


FIGURE 4.3: The architecture of MPCHECKER.

This section defines the issue of code-documentation inconsistency caused by multi-parameter constraints and provides a detailed description of the approach. An API documentation error is an inconsistency between the library source code and its API documentation. Multi-parameter constraints refer to conditional dependency

relationships that exist among multiple parameters within functions or classes. If a constraint is never violated across all execution paths in the code, it is considered a benign constraint; otherwise, it indicates a potential documentation error. According to the literature [88–90], API documentation inconsistency can be categorized into two types: incorrectness and incompleteness. Incorrectness refers to cases where the documentation describes behavior that is not implemented in the code, while incompleteness arises when certain code behaviors are not reflected in the documentation. Typically, incorrectness issues are considered more critical than incompleteness.

In addition, when it comes to constraint extraction, compared to single-parameter constraints, the multi-parameter constraint extraction problem can be classified into two types, as discussed in Section 4.2.1, 1) explicit constraint and 2) implicit constraint.

MPChecker processes a repository through the three phases in Figure 4.3:

1. **Phase I: Preprocessing** separates documentation from code and associates parameter descriptions with the relevant functions (Section 4.3.1).
2. **Phase II: Constraint Extraction** has two branches: the LLM-based Constraint Extractor produces documentation constraints, and the Symbolic Executor produces code path constraints (Section 4.3.2). Code transformation prepares functions for symbolic execution, as indicated by the "Modify" arrow in the figure.
3. **Phase III: Inconsistency Detection** converts the extracted constraints into expressions for comparison and applies SMT-based fuzzy constraint reasoning (Section 4.3.3). The specialized predicates and constraint conversion supporting these comparisons are described in Section 4.3.3.1 and Section 4.3.3.2.

4.3.1 Preprocessing

Phase I of Figure 4.3 separates documentation from code in the input repository while preserving their correspondence. It also extracts parameter-description pairs to guide documentation constraint extraction in Phase II.

In modern data science libraries, documentation is typically auto-generated using Sphinx, a tool that can automatically create HTML documentation from Python code. Sphinx supports various docstring styles, with Google style and NumPy style being commonly used. Figure 4.4 displays docstring examples of two different styles from the official Sphinx website [91, 92]. Google-style docstrings use a clear and concise format with a minimalistic structure. This style divides the docstring into sections like **Args**, **Attributes**, etc., with each section using plain indentation. Similarly, NumPy-style docstrings organize sections more rigidly. Sections are divided by using **Parameters**, **Attributes**, etc. with horizontal dash lines "**- -**" under the section header. The number of dashes is the same as the number of letters in the section header. Regardless of the style used, the docstring is normally placed at the beginning inside its corresponding class or function.

After downloading the project, the tool first converts every Python file from the project into an Abstract Syntax Tree (AST) and isolates the classes and independent functions. This chapter focuses on the CDI issue, so in this step, MPChecker filters out code without documentation and separately extracts the code and documentation from the remaining code. Since Python supports object-oriented programming but current symbolic execution tools have limited support for classes, the experimental units are limited to functions. For independent functions, the scope of constraints in the documentation usually applies within the function itself. For classes, however, the constraints cover the entire class, including each member function. Therefore, MPChecker creates a new directory for every class and independent function, with member function directories placed within their corresponding class directories to maintain structural consistency. If the member function has its own documentation, it will also be retained.

To help the LLM better focus on the constraints between parameters and reduce the occurrence of erroneous constraints, the preprocessing step retains parameters or attributes and their corresponding descriptions in the form of key-value pairs, based on the two aforementioned docstring styles. A rule-based heuristic approach is subsequently applied to retain documentation that potentially contains constraints and to discard the rest. For instance, if none of the parameters or attributes appear in other descriptions, this indicates the absence of multi-parameter constraints in that documentation.

NumPy Style Docstrings

```
class ExampleNumPyStyle():
"""Exceptions are documented

  Parameters
  ----------
  msg : str
    Human readable string
    describing the exception.
  code : obj:`int`, optional
    Numeric error code.


  Attributes
  ----------
  msg : str
    Human readable string
    describing the exception.
  code : int
    Numeric error code.
  """

  def __init__(self, msg, code):
    self.msg = msg
    self.code = code
```

Google Style Docstrings

```
class ExampleGoogleStyle():
  """Exceptions are documented
  Note:
    Do not include the `self`
    parameter in the ``Args``
    section.

  Args:
    msg (str): Human readable
    string describing the
    exception.
    code (obj:`int`, optional):
    Error code.

  Attributes:
    msg (str): Human readable
    string describing the
    exception.
    code (int): Exception error
    code
  """
  def __init__(self, msg, code):
    self.msg = msg
    self.code = code
```

FIGURE 4.4: Examples of two docstring styles.

4.3.2 Constraint Extraction

This subsection describes Phase II of Figure 4.3. It explains how to extract path constraints from code and convert them into expressions that can be solved by an SMT solver, as well as how to use LLMs to extract constraints from documentation and transform them into expressions containing fuzzy words.

4.3.2.1 Code Constraint Expression Extraction

The goal of MPCHECKER is to verify whether the constraints between multiple parameters in documentation align with the logic during actual code execution. This requires the tool to understand and analyze deeper constraint relationships. Therefore, MPCHECKER employs symbolic execution to capture as many path conditions as possible and precisely handle complex paths and constraints, as implemented by the *Symbolic Executor* component in Figure 4.3.

The study modified current advanced dynamic symbolic execution tools [93–97] for path exploration. Unfortunately, supporting symbolic execution for dynamic languages like Python is more challenging than supporting it for statically typed languages such as Java and C. Despite Python's rapid evolution, symbolic execution tools specifically designed for Python have developed slowly, struggling to keep pace with the growing new syntax and features. This necessitates appropriate modifications to the source code extracted directly from repositories. However, these modifications must not alter the path constraints of the original code; they should be equivalent code transformations that do not affect path exploration. The main modifications were as follows:

1. Current Python symbolic execution tools cannot handle classes directly. Therefore, it is necessary to split the class into functions (i.e., member functions). The corresponding member variables also need to be changed and used as symbolic inputs.

2. Replace complex structures and operations, such as lists and dictionaries, that are difficult to handle and do not affect the path, as well as external function calls that may cause path explosion, with symbolic inputs.

3. Replace the handling of exceptions and warnings that do not affect the path with *return*.

4. Add a *return* statement in a fixed format to capture concrete values of potential symbols.

5. Replace unsupported advanced syntax with equivalent code to enable the extraction of useful path constraints. For example, replace a ternary operator with a conventional if-else statement.

In the limit, MPChecker strives to explore all feasible paths in a Python function by following these processes: 1) Running the function with specific input to trace a path through the control flow of the function; 2) Symbolically executing the path to determine how its conditions depend on the function's input parameters; 3) Utilizing Z3 to generate new parameter values that guide the function toward paths that have not been covered yet.

Original source code

```
def fit(self, sample_weight):
  if sample_weight is not None and self.strategy == "uniform":
    raise ValueError("Warning Info")
  if sample_weight is not None:
    sample_weight = _check_sample_weight(sample_weight, X)
```

(A) Original source code

Modified source code

```
def fit(sample_weight, strategy, call__check_sample_weight):
  if sample_weight != 'None' and strategy == 'uniform':
    return '(sample_weight)_(strategy)_ERROR_END'
  if sample_weight != 'None':
    sample_weight = call__check_sample_weight
  return (f'(sample_weight = {sample_weight}) ^ (call__check_sample_weight
    = {call__check_sample_weight}) ^ (strategy = {strategy})')
```

(B) Modified source code

FIGURE 4.5: Source code transformation for constraint extraction.

Although MPCHECKER supports a certain level of external function call analysis, in complex real-world code, an external function call often corresponds to additional function calls, leading to path explosion. Furthermore, documentation constraints are usually handled within the target function, so the preferred strategy remains to avoid introducing external function calls and to focus the analysis within the target function. Additionally, similar to the current concolic symbolic execution tools for Python, MPCHECKER does not yet provide strong support for the theory of strings. Thus, during the actual execution process, MPCHECKER replaces the string with a unique large number, which does not affect the exploration of condition constraints.

The example in Figure 4.5 shows simplified original source code (a) and modified code (b) from the popular data science project `scikit-learn`. The corresponding path constraints are shown in Figure 4.6. Together, these figures illustrate the extraction phase. The *fit* function is a member function within a class, and thus member variables such as "self.strategy" also exist within the code. "None" is also converted to a string to make it easier to be captured, since it is represented as a number 0 during symbolic execution. The tool first modifies the code and replaces exception handling and external function calls with symbolic inputs, marked as "ERROR_END" and "call_", respectively. For those paths whose final states are

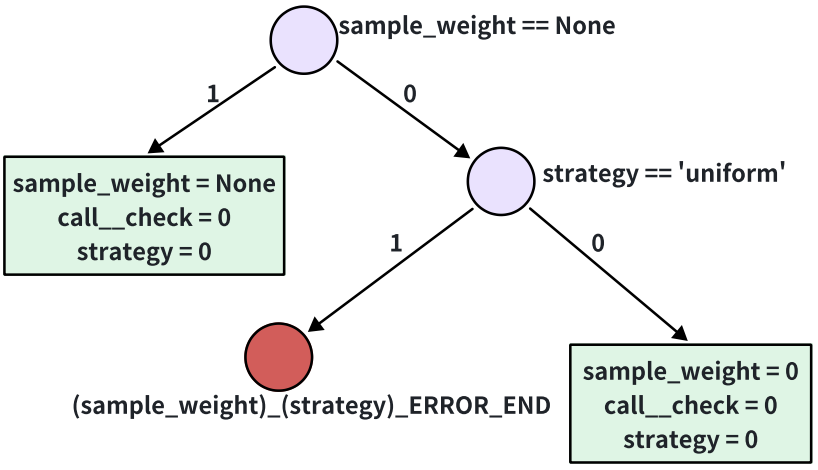


FIGURE 4.6: Path constraints corresponding to the code in Figure 4.5.

"ERROR_END", the final results of the conjunction of the documentation constraint and these paths will be negated during the reasoning phase.

Each code constraint $p \in P$ is a conjunction of symbolic branch conditions along an explored path, with conditions negated when the corresponding branches evaluate to false. The concrete endpoint values in Figure 4.6 describe individual executions rather than equalities that hold for all inputs satisfying the path condition.

4.3.2.2 Documentation Constraint Expression Extraction

In this step, MPCHECKER applies LLMs to extract constraints from Python documentation, as shown by the *LLM-based Constraint Extractor* in Figure 4.3. Since Python documentation can vary in quality and may contain informal writing [98], the important task is to understand the parameter information within the documentation. To achieve this, MPCHECKER uses SOTA LLMs. Given Python documentation as input, the LLM is asked to first extract constraint-related sentences and then output them in a standard logical expression format. This includes two steps, model selection and prompt design.

Model Selection. MPCHECKER uses GPT-4, which is pretrained on a diverse corpus and shows excellent performance in natural language understanding. Based on the preliminary study, GPT-4's performance stands out compared to that of

Gemini-1.5 [99] and LLaMA-3 [100] due to its ability to capture details, and it is also well-acquainted with the context of code documentation [101].

Prompt Design. Because the constraint extraction task is relatively complex and can be broken down into clear steps, the prompting strategy uses the chain-of-thought approach [80], which has been widely proven effective in improving GPT-based model performance. The prompting task is divided into two steps, document input and constraint extraction. Figure 4.7 shows the structure and some details of the used prompt. The prompt mechanism for each step is detailed below.

Document Input Prompt. Some documentation may be too lengthy to provide to GPT-4 in a single input, considering that GPT-4 has a maximum token length limit of 8,192 tokens [102]. This study also finds that LLMs exhibit lower performance when dealing with long and complex text inputs as noted in previous research [103, 104]. Thus, the lengthy documents are segmented into smaller sections. To determine a heuristic chunk size, ten lengthy Python documentation files were randomly selected, split into chunks of varying word lengths, and used as inputs for GPT-4. The constraint extraction task performance of GPT-4 was then evaluated based on these inputs to determine which chunk size yields better results. Based on the findings, the chunk size is standardized to 1,500 words (around 2,048 tokens [105]). The parameter list obtained in Section 4.3.1 is also provided to GPT-4 to help the model better recognize the information related to parameters. The details of the document input prompt are shown in Prompt 1 in Figure 4.7.

Constraint Extraction Prompt. For the constraint extraction task, the prompt is divided into three parts to guide GPT-4 in recognizing text related to constraints in the original documentation, and then, based on that text, to generate a formatted logical expression of the constraint.

The first part involves defining the logical symbols that can be used in the logical format, including implication, negation NOT, logical AND, logical OR, and also defining parentheses to indicate the precedence of logical expressions.

The second part arises from the preliminary study, which found that some Python documentation uses vague terms such as “override”, “specify”, “have an effect”, “no effect”, “significant”, and “ignore” when mentioning constraints related to parameters. To preserve as much detail as possible from the documentation, prompts are designed to guide GPT-4 so that if text related to parameter constraints contains vague keywords, these keywords should be retained in the final logical expression.

In the third part, to ensure that the format of the logical expression in GPT-4’s output is consistent each time and convenient to process, the prompting strategy applies in-context learning techniques that are widely used in previous works [106, 107] to enable GPT-based models to handle tasks specific to a domain. The prompt includes four examples that contain pairs of original constraint-related sentences selected from Python documentation and their corresponding logical expression constraints.

Documentation constraints are expressed in a quantifier-free constraint language comprising atomic comparisons and path-dependent predicates combined by Boolean connectives. Figure 4.8 specifies the core syntax for comparisons between parameters and literal values, such as `strategy = "uniform"`. Parameter names refer to the parameters or attributes of the documented function or class. Specialized predicates, including `ignore(x)` and `exist(x)`, extend this core and are evaluated according to whether the parameter is used along an execution path, as described in Section 4.3.3.1.

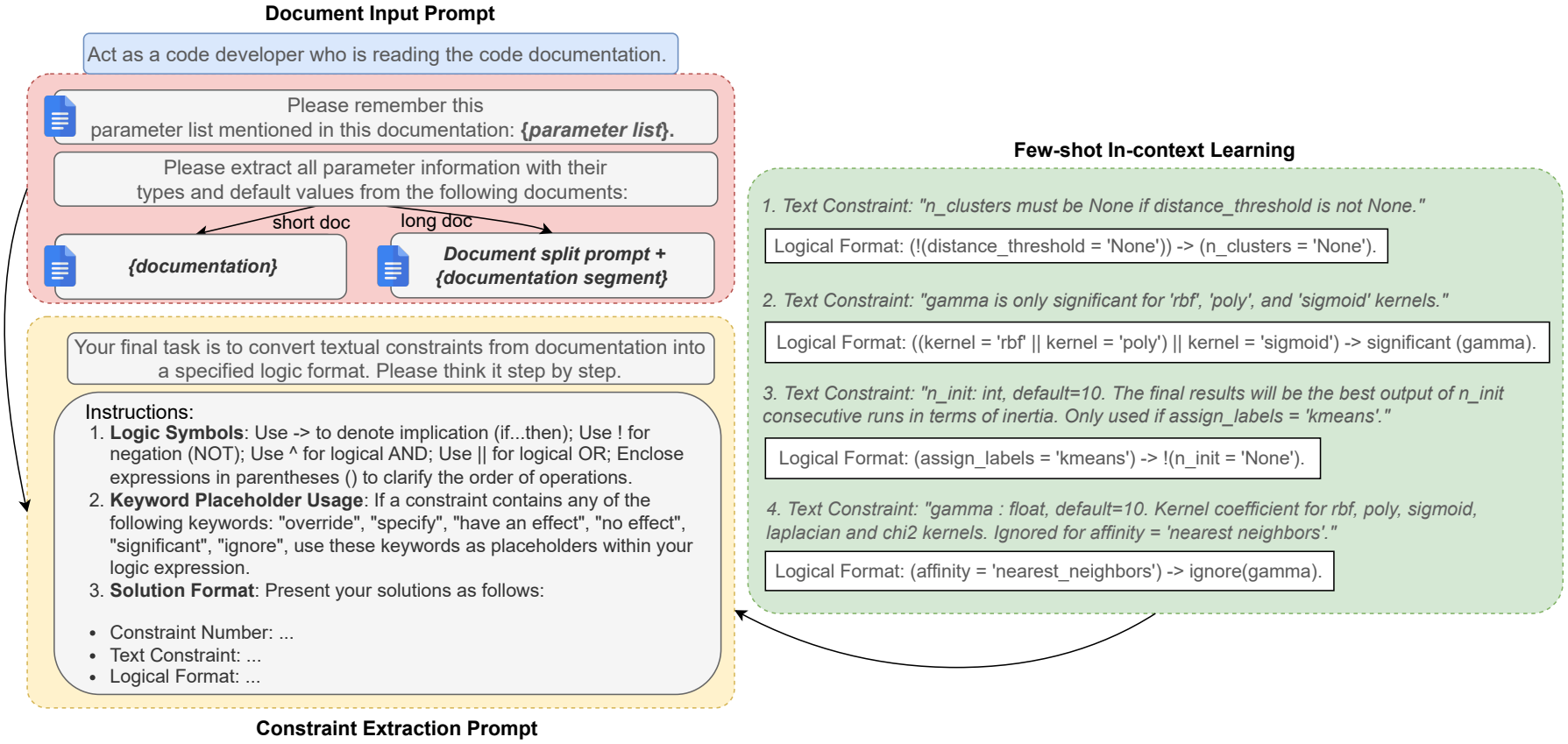


FIGURE 4.7: Prompt structure for constraint extraction.

4.3.3 Inconsistency Detection

In the second phase, constraints are extracted from both documentation and code. While *code-constraints* are deterministic in nature, *doc-constraints* inherently contain uncertainties stemming from two main sources. First, there are *implicit constraints* arising from vague or incomplete descriptions, which are addressed by defining fuzzy words to extend them into soft constraints. Second, there are uncertainties introduced by generative models' limited reasoning capabilities and unavoidable hallucination issues, for which no validator exists to definitively determine the correctness of generated constraints. To address this challenging scenario, this study introduces a customized fuzzy constraint logic to mitigate such vagueness. With the help of fuzzy words and fuzzy constraint logic, the converter can effectively handle both explicit and implicit constraints. The converter ultimately produces fuzzy expressions, which are then processed by a reasoner based on the Z3 SMT (Satisfiability Modulo Theories) solver to detect inconsistencies. These operations correspond to the *Fuzzy Constraint Converter* and *SMT-based Fuzzy Constraint Reasoner* in Phase III of Figure 4.3.

The reasoner offers two strategies, from relaxed to strict, to identify inconsistencies from the perspectives of satisfiability and equivalence. Given a *doc-constraint* c and a *code-constraint* set P containing a group of path constraints p, the detection strategies are defined as follows:

- **Unsatisfiability** checking determines whether the *doc-constraint* c is unsatisfiable under all path constraints. If the conjunction of c and every path constraint p is unsatisfiable, it indicates a contradictory inconsistency, meaning that under all possible execution paths, the code violates the constraint stipulated in the documentation.

$$\forall p \in P, \neg(c \wedge p) \tag{4.1}$$

- **Nonequivalence** checking determines whether the *doc-constraint* c and *code-constraints* are logically equivalent. If equivalence holds only under specific

conditions, a behavioral inconsistency may arise, implying that the constraints implemented in the code are not fully equivalent to the documentation.

$$\exists p \in P, \neg(c \Leftrightarrow p) \tag{4.2}$$

For constraints containing sub-constraints that do not exist in the code logic, MPChecker employs a heuristic approach to provide suggestions. Specifically, when a constraint mentioned in the documentation is not present in the corresponding code logic, MPChecker issues a warning and labels this constraint as a potential weak constraint to prompt further investigation by the user.

The documentation for `sample_weight` in Figure 4.5 states: "Cannot be used when `strategy` is set to `"uniform"`." Replacing "used" with "None" changes only one word but reverses the requirement. The extracted *doc-constraint* is:

$$(\texttt{strategy} = \texttt{"uniform"}) \rightarrow (\texttt{sample_weight} \neq \texttt{None}).$$

Symbolic execution extracts the following path constraint for the path that raises `ValueError`, marked as `ERROR_END` in Figure 4.6:

$$p_{\text{err}} = (\texttt{strategy} = \texttt{"uniform"}) \wedge (\texttt{sample_weight} \neq \texttt{None}).$$

In Phase III of Figure 4.3, the reasoner checks whether $c \wedge p_{\text{err}}$ is satisfiable, where c conjoins the condition and requirement of the *doc-constraint*, and p_{err} is the path constraint. This uses the conjunction underlying the unsatisfiability checking in Equation (4.1), together with the handling of error paths described above. This conjunction is satisfiable, so the parameter combination can reach the exception. The `ERROR_END` handling described above therefore negates the Boolean checking result to `False`. The `False` result indicates that the *doc-constraint* fails the consistency check for this error path under the unsatisfiability-checking strategy. Accordingly, MPChecker reports `BAD CONSTRAINT WITH ERROR`, identifying the conflicting constraint and path.

4.3.3.1 Fuzzy Words

The example from Section 4.2.1.2 illustrates a very typical implicit constraint with fuzzy words, where part of the constraint is clearly defined while other parts remain

uncertain. This study introduces a series of fuzzy words to help LLMs extract constraints better. These fuzzy words frequently appear in documentation but do not represent specific values, making it challenging for the LLM to extract them directly. These fuzzy words fall into two general categories: *existence* and *non-existence*. In fuzzy words, *non-existence* includes "ignore", "no effect", "unused", "override", indicating that a parameter either is unused or does not exist within code segments where other conditions are met. Similarly, *existence* includes "specify", "have an effect", "exist", "significant", indicating that the parameter is used or exists when other conditions are met.

Several specialized predicates are implemented to evaluate such implicit constraints. MPChecker first traces the target variable's definition-use chain (DU-chain) [108, 109] and then checks if the definition and usage of it exist or not under a specific program path. For instance, "`exist(x)`" checks if the definition and usage of a variable `x` exist in the program path with other explicit conditions. If found, it returns `True`; otherwise, it returns `False`. Similarly, "`ignore(x)`" checks whether the definition and usage of `x` are absent under the program path with given explicit conditions. It can be further extended to check weak *doc-constraints*. As in the example in Section 4.2.1.2, *gamma* is ignored when `affinity` is set to `nearest_neighbors`, `precomputed_nearest_neighbors`, or `precomputed`.

4.3.3.2 Fuzzy Constraint Satisfaction

Although Phase II employs several strategies to maximize the LLM's understanding of constraints and restrict randomness in outputs, inaccurate extraction is still unavoidable. The main reasons are threefold: (1) typos inevitably occur when developers write documentation; (2) the hallucination issues inherent to black-box generative models; and (3) the intrinsic ambiguity in natural languages. This implies that correct documentation descriptions can generate incorrect *doc-constraints*, and incorrect documentation descriptions can also have the chance to generate correct *doc-constraints*.

In the absence of LLM unpredictability, detecting CDI issues is a crisp constraint satisfaction problem (CSP), deciding whether a *doc-constraint* is consistent with the actual code implementation. Nevertheless, due to minor errors introduced by LLMs, such as a single letter being wrongly spelled in a parameter name, or

$$
\begin{aligned}
c \in \mathit{Constraint} &::= e \mid \neg c \mid c \vee c \mid c \wedge c \\
e \in \mathit{Expression} &::= p \bowtie v \\
p \in \mathit{Parameter} &::= \mathit{char}, \{\mathit{char} \mid \mathit{digit}\} \\
\bowtie \in \mathit{Operator} &::= < \mid > \mid <= \mid >= \mid = \mid \mathbf{!}= \\
v \in \mathit{Value} &::= \mathit{string} \mid \mathit{number} \mid \mathit{bool}
\end{aligned}
$$

FIGURE 4.8: Extended Backus-Naur form for multi-parameter constraints.

a comparison operator being reversed, e.g., writing "<" instead of ">", a *doc-constraint* can be mistakenly identified as inconsistent when it is actually correct.

To address this, a customized fuzzy constraint logic was proposed that reconciles such unpredictability. In a traditional fuzzy constraint [83, 87], a membership function assigns a degree of satisfaction (ranging from 0 to 1) to each possible variable value. It enables partial fulfillment of a condition, with satisfaction measured on a continuous scale. In this setting, a constraint needs to be measured on a new scale, assessing "how likely" the extracted *doc-constraint* conforms to the *code-constraints*. Therefore, a unique similarity computation was introduced which serves as the membership function.

Figure 4.8 shows an EBNF grammar for the multi-parameter constraints. A multi-parameter constraint is a combination and nesting of binary expressions and Boolean operators, which can be viewed as a complete binary tree where leaf nodes are binary expressions over single parameters and non-leaf nodes are logical operators connecting them. Without loss of generality, the constraints retain only negation, conjunction, and disjunction; logical relations such as implications can be simplified accordingly. The fuzziness of a constraint is defined with respect to a set of *environment expressions*, facts that are known to hold (with a truth value of 1). In other words, the instantiation of a specific tree structure and nodes is a constraint c evaluated against a set of expressions $\{e_1, e_2, \ldots, e_n\}$. Next, the membership function of the proposed fuzzy constraint logic is defined through a few similarity functions.

Definition 4.1 (Expression Similarity)**.** The similarity between two expressions e_1 and e_2 is defined as follows:

$$
\sigma(e_1, e_2) = \alpha * (1 - \frac{LD(p_1, p_2)}{\max(|p_1|, |p_2|)}) + \beta * (\frac{\delta_{\bowtie_1} \cdot \delta_{\bowtie_2}}{\|\delta_{\bowtie_1}\| \|\delta_{\bowtie_2}\|}) + \alpha * (1 - \frac{LD(v_1, v_2)}{\max(|v_1|, |v_2|)}), \quad (4.3)
$$

where α and β denote the relative weights, p, $\bowtie$, and v are the parameter, operator, and value, respectively; $|p|$ and $|v|$ denote the lengths of p and v, respectively, and $\|\delta_{\bowtie}\|$ denotes the magnitude (or Euclidean norm) of the vector $\delta_{\bowtie}$.

The similarity between two expressions is considered separately for the parameters, operators, and values appearing in the expressions. Both p and v can be treated as text; therefore, Levenshtein distance (a.k.a. edit distance) is used to represent their similarity. The normalized Levenshtein distance (NLD) is given in Equation (4.4), where s denotes strings (p or v) and $|s|$ denotes the length of it.

$$\eta(s_1, s_2) = NLD = 1 - \frac{LD(s_1, s_2)}{\max(|s_1|, |s_2|)} \tag{4.4}$$

$$\delta_{\bowtie} = (C, E, G, L, N), \text{where } C, E, G, L, N \in \{0, 1\} \tag{4.5}$$

$$cos\theta(\bowtie_1, \bowtie_2) = \frac{\delta_{\bowtie_1} \cdot \delta_{\bowtie_2}}{\|\delta_{\bowtie_1}\| \|\delta_{\bowtie_2}\|} \tag{4.6}$$

As illustrated in Equation (4.5), the proposed approach uses an operator embedding across five key dimensions: **C**omparison, **E**quality, **G**reater than, **L**ess than, and **N**egativity. This way, the similarity between two operators can be calculated as the cosine similarity between two vectors. The result is highly intuitive. For example, with $\delta_{<} = (1, 0, 0, 1, 0)$, $\delta_{>} = (1, 0, 1, 0, 0)$, and $\delta_{<=} = (1, 1, 0, 1, 0)$, the similarity between "<" and ">" is 0.5, while the similarity between "<" and "<=" is 0.82.

The weight of operator similarity is set as β such that the weights of operators, values, and parameters within a given expression should sum to one. Thus, the derivation gives $\alpha = \frac{1-\beta}{2}$ and the similarity σ of two single parameter expressions (i.e., atomic constraints) can be calculated according to Equation (4.3).

Definition 4.2 (Constraint Similarity)**.** Let c be a constraint and $\Phi = \{e_i | i = 1, \ldots, n\}$ be a set of environment expressions assumed to hold true. The similarity

of c against Φ is given by the following set of calculations.

$$\rho(c, \Phi) = \begin{cases} \arg\max_{e_i \in \Phi} \sigma(e, e_i), & \text{if } c \text{ is an expression } e \\ 1 - \sigma(c', \Phi), & \text{if } c = \neg c' \\ \min\{\sigma(c_1, \Phi), \sigma(c_2, \Phi)\}, & \text{if } c = c_1 \wedge c_2 \\ \max\{\sigma(c_1, \Phi), \sigma(c_2, \Phi)\}, & \text{if } c = c_1 \vee c_2 \end{cases} \tag{4.7}$$

Consider an atomic constraint with a single expression; its similarity to Φ associated with a set of *environment expressions* can be represented by the maximum expression similarity among all expressions within Φ. Based on the *conjunctive combination principle* [110], when combining two constraints using a conjunction, their degree of joint similarity ρ should be represented by the minimum similarity between them. Similarly, based on the *disjunctive combination principle* [110], when they are combined with a disjunction, the maximum similarity should be used. For negation, the complementary similarity is used.

Definition 4.3 (Membership Function for Fuzzy Constraint Satisfaction)**.** Constraint similarity serves as the membership function μ_Ω, quantifying the degree to which a given constraint ϵ is consistent with the code, which is represented as a set Ω of path constraints ω:

$$\mu_\Omega(\epsilon) = \rho(\epsilon, \Phi_\Omega) \cdot \epsilon[e \mapsto e_{\Phi_\Omega}] \tag{4.8}$$

where Φ_Ω denotes the set of expressions aggregated from all the path constraints in Ω, and $\epsilon[e \mapsto e_{\Phi_\Omega}]$ is a rewrite of ϵ, where each expression e has been replaced by its closest counterpart from Φ_Ω, according to the expression similarity σ.

The inconsistency between the modified constraint $\epsilon[e \mapsto e_{\Phi_\Omega}]$ and Ω is then evaluated (according to Equation (4.1) or Equation (4.2)), yielding a binary result (`True` or `False`). To enable the probabilistic interpretation, a linear transformation ensures complementary probabilities. For instance, "0.7·False = 0.3·True", indicating a 70% probability of inconsistency or a 30% probability of consistency.

4.3.3.3 Constraint Similarity Threshold.

LLMs demonstrate great potential in constraint extraction, yet they still encounter errors such as using incorrect parameter names or values and introducing non-existent constraints. To reduce false positives from these inevitable issues, MPChecker sets a constraint similarity threshold of 0.85. This is based on the observation that a high constraint similarity (>0.85) indicates a high likelihood of misinterpretation or conflation by the LLM.

For instance, in `scikit-learn`, the documentation of `LinearSVC` states: "*If n_samples < n_features and optimizer supports chosen loss, multi_class and penalty, then dual will be set to True*". However, the extracted constraint is "(`samples<features`)∧(`dual=True`)", where two parameters are mistakenly mapped to similar names. Their constraint similarity is 0.86, exceeding the threshold, leading MPChecker to discard the result. Moreover, in most cases where the expression contains only a single parameter, the threshold exhibits stronger filtering capability.

However, setting a threshold cannot entirely exclude all false positives, as there is no definitive rule to ascertain whether an error originates from the documentation or the LLM. Another example from `scikit-learn` illustrates this limitation: the documentation of `estimator_` states: *"The child estimator template used to create the collection of fitted sub-estimators"*. Yet, the extracted constraint "(`estimator_=child_estimator_template`)∧(`collection=fitted_sub_estimators`)" has a constraint similarity of 0.67, which is below the threshold, leading MPChecker to accept the result.

4.4 Evaluation

This section describes the evaluation of MPChecker. It first presents the research questions, then details the experiment setup and evaluation subjects. Finally, it analyzes the experimental results and answers each research question. The evaluation was guided by the following research questions:

1. **RQ1:** How accurate is MPChecker in extracting constraints from API documentation?

2. **RQ2:** How effective is MPChecker in detecting errors related to multi-parameter constraints in API documentation?

3. **RQ3:** How effectively can MPChecker detect unknown inconsistency issues?

4.4.1 Experiment Setup

4.4.1.1 Dataset.

There is currently no well-established dataset specifically focusing on multi-parameter API documentation errors. To better evaluate the effectiveness of the tool, two datasets were constructed: a constraint dataset and an inconsistency dataset.

Constraint Dataset. The constraint dataset contains 72 constraints from 4 popular open-source data science libraries with well-maintained documentation, including `scikit-learn`, `scipy`, `numpy`, and `pandas`. The constraints were gathered by analyzing commits from each GitHub repository, focusing on developers' modifications to documentation related to multi-parameter constraints. To streamline this process, an automated script was developed to collect all documentation-related commits and identify changes within parameter (including attribute) descriptions, adhering to two distinct docstring styles (see details in Figure 4.4). By mapping parameter names to their descriptions, it then cross-checks if any parameter names appear within others' descriptions to keep potential constraint-related documentation. Approximately 90% to 95% of irrelevant commits were excluded, leaving a smaller subset of commits that may contain constraints. Additionally, random sampling of the excluded commits was performed to assess and minimize the impact of this heuristic. Despite the approach's proven effectiveness, the remaining subset still contains a substantial number of submissions. Each of the four repositories has over 30,000 commits, requiring manual verification of approximately 1,500 commits per repository to further identify multi-parameter constraints. For each constraint, its source information (repository name, SHA, file path, etc.) was logged and the code file was retained, enabling swift extraction of documentation and code for reproducibility. In some cases, the condition enforcing the constraint is located not in the current function but in a function it

calls, leading to a mismatch between documentation and code. To address this, such mismatches were recorded and the constraint description was relocated to the function where the check actually occurs.

Inconsistency Dataset. Based on the constraint dataset, an initial inconsistency dataset was constructed comprising 126 multi-parameter constraints that lead to code-documentation inconsistencies. An analysis of around 20 resolved GitHub issues related to multi-parameter constraints identified eight common patterns that may cause CDI: (1) Parameter name change; (2) Value Change; (3) Logic Change; (4) Remove Parameter; (5) Add Constraints; (6) Remove Constraints; (7) Missing Documentation; (8) Modify Description. To better evaluate the capabilities of the tool, these eight patterns were subsequently applied to expand the dataset through mutation based on the *Constraint Dataset*. For each constraint, two types of modifications were applied, resulting in an expanded dataset containing 216 constraints. The mutated constraints and the original constraints were fed into an SMT solver to verify whether the mutations violated the original constraints. Each constraint was also manually inspected to verify its classification as consistent or inconsistent. It is important to note that modifying a correct constraint does not necessarily turn it into an incorrect one. The final expanded dataset contained 126 inconsistent constraints and 90 consistent constraints. The dataset represents potential outcomes of documentation changes that may occur during development.

Table 4.1: Data science libraries used in the experiments.

Project	class	class w/ doc	func	func w/ doc	KLOC	avg. params	#Stars
scikit-learn	878	306	9,332	1,535	400.1	1.42	61.8K
pandas	2,211	102	28,880	1,432	620.6	1.14	45.2K
scipy	2,570	142	22,059	1,705	517.9	1.30	13.6K
numpy	2,000	51	12,618	902	276.3	0.83	29.4K
keras	1,370	254	9,877	724	218.5	1.42	62.9K
dask	284	26	6,914	368	157.5	1.33	13.1K
statsmodels	2,184	273	11,590	1,894	424.6	1.32	10.6K

4.4.1.2 Subjects

Table 4.1 lists 7 popular libraries on which MPChecker was evaluated, the first four for dataset construction and the last three for assessing MPChecker's ability to detect unknown issues. The selected libraries are of high quality and widely used, with tens of thousands of stars on GitHub. These libraries are substantial third-party libraries, averaging 1,642 classes, 14,467 functions, with an average of 373.6 thousand lines of code, and each function containing an average of 1.3 parameters. The tool extracts documentation constraints and path constraints from these libraries and uses a fuzzy constraint reasoner to detect inconsistencies.

MPChecker was implemented in Python. All the experiments were performed on an Intel(R) Xeon(R) Silver 4214 CPU @ 2.20GHz machine with 252GB of RAM, running Ubuntu 18.04, with Python 3.8.19 and Z3 4.13.0. For the constraint extraction evaluation (RQ1), GPT-4 was accessed through OpenAI's API. For result validation, given the absence of established benchmarks in this domain, two volunteer researchers independently reviewed the constraint extraction results from GPT-4, manually assessing each constraint's consistency with the original Python documentation. Any discrepancies were resolved through consensus discussion. In RQ1, *Equivalent* denotes a correctly extracted constraint that preserves the meaning of the original documentation, whereas *Non-Equivalent* groups incorrect extractions and missing constraints. Thus, these categories assess documentation constraint extraction, rather than equivalence between documentation and code constraints.

4.4.2 Results

4.4.2.1 Accuracy of LLM in extracting constraints from API documentation.

Table 4.2 displays the result of RQ1. The experiment shows that the tool correctly identified and extracted 66 out of 72 constraints contained in the Python documentation collected in the benchmark, achieving an accuracy of 91.7%, which demonstrates that the tool can successfully extract most of the constraints accurately, with few errors or omissions. Next, the remaining failed cases are examined to investigate the reasons for the inaccuracy. The analysis showed that out of the

TABLE 4.2: Results of MPCHECKER on constraint extraction.

Configuration	Equivalent	Non-Equivalent		Accuracy
	Correct extraction	Incorrect extraction	Missing constraints	
MPCHECKER	66	2	4	91.7%
MPCHECKER w/o few-shot learning	45	20	7	62.5%
MPCHECKER w/o chain-of-thought	57	7	8	79.2%

All configurations are evaluated on 72 constraints. Accuracy is the proportion of correctly extracted constraints.

6 extraction failures, 4 involved missing constraints during the documentation processing. For example, one of the missing constraints is: "(batch_size = auto) → (batch_size = min(200, n_samples))". Although this case involves a constraint between multiple parameters, the format is tricky because one of the parameters is within a function, which may have misled GPT-4 and caused it to miss this constraint during extraction. In the last 2 cases, the constraint information was identified but converted into incorrect logic expressions due to the complex logic or sentence structure.

An ablation study further explores MPCHECKER's abilities. The results show that MPCHECKER without few-shot learning achieves an accuracy of 62.5%. Most failures occur in the incorrect extraction of constraints. This indicates that including few-shot learning is important for MPCHECKER to generate accurate constraints. Next, MPCHECKER without applying chain-of-thought techniques results in an accuracy of 79.2%, with the number of missed constraints accounting for more than half of total non-equivalent cases. This suggests that GPT tends to miss more details in documentation when chain-of-thought is removed. After chain-of-thought and few-shot learning are incorporated, MPCHECKER's performance shows a clear improvement.

Answer to RQ1: MPCHECKER correctly extracted 66 constraints out of 72 in total, achieving an accuracy of 91.7%. This demonstrates that MPCHECKER is effective in extracting constraints from Python documentation.

4.4.2.2 MPChecker's effectiveness in detecting multi-parameter API documentation errors

To measure MPChecker's effectiveness in detecting multi-parameter API documentation errors, MPChecker was evaluated on the inconsistency dataset. Table 4.3 shows the results of MPChecker in detecting multi-parameter CDI on the inconsistency dataset.

The Vanilla MPChecker does not include fuzzy words or apply fuzzy constraint satisfaction theory, limiting its ability to handle implicit constraints. Despite these limitations, it achieved an impressive 69% recall by detecting 87 inconsistencies. With fuzzy words and fuzzy constraint logic, MPChecker's performance significantly improved, successfully identifying 117 inconsistencies with a recall of 92.8% and an accuracy $= \frac{TP+TN}{TP+TN+FP+FN} = \frac{117+88}{216} = 94.9\%$. This demonstrates that incorporating fuzzy words and fuzzy constraints can expand the range of detectable constraints. However, two false positives were identified because the constraints lie between the parameters and method calls, rather than among the parameters. One example is shown below:

$$(\texttt{shape} = \texttt{None}) \wedge (\texttt{axes} \neq \texttt{None}) \rightarrow (\texttt{shape} = \texttt{numpy.take}(\texttt{x.shape}, \texttt{axes}, \texttt{axis} = 0))$$

where the value of "shape" is the return value of the function "take()" from the NumPy library. Modern software increasingly emphasizes code maintainability and reusability, leading to highly complex function calls, often involving nested or chained calls. To avoid the high risk of path explosion in symbolic execution, MPChecker replaces function calls with symbolic inputs during the preprocessing phase, which leads to misclassification of these two inconsistencies.

The remaining 9 unresolved inconsistent constraints stem from external function dependencies. While incorporating external function code could resolve these constraints, this approach risks infinite recursive dependencies and path explosion. This study takes the view that constraints should be handled within the documented function itself.

A notable situation arose during one issue report: even though the issue had been confirmed, one developer expressed dissatisfaction. He believed the reporters were an automated tool or AI-based bot because of their anonymous status, which diminished his enthusiasm for addressing the issue. With a mass of LLM-based

TABLE 4.3: Results of LLM and MPCHECKER on detecting multi-parameter Code-Documentation Inconsistencies (CDIs).

Checker	FN	TP	Recall
LLM	119	7	5.6%
LLM+C	74	52	41.3%
VANILLA MPCHECKER	39	87	69.0%
FUZZY MPCHECKER	9	117	92.8%

LLM: raw documentation and corresponding code; LLM+C: extracted *doc-constraints* from documentation and corresponding code; VANILLA MPCHECKER: MPCHECKER without fuzzy words and fuzzy constraint logic; FUZZY MPCHECKER: MPCHECKER with fuzzy words and fuzzy constraint logic.

program analysis or inconsistency checkers now available, while they offer insights sometimes, their results often require more manual verification effort than those of traditional tools due to higher uncertainty.

Comparative Study. Therefore, a comparative experiment was conducted between the tool and the approach of using only LLMs as a constraint checker on the same dataset. To align with the experiment settings, GPT-4, one of the leading models, was selected for comparison and evaluated under two settings: 1) LLM: providing the raw documentation and code as input, directly prompting GPT to check for consistency, and 2) LLM+C: first extracting constraints using the LLM and then providing both these constraints and their corresponding code to the LLM for a consistency check. In addition, the LLM was also required to provide justifications for its answers.

As shown in Table 4.3, when raw documentation and the corresponding code were provided as inputs to the LLM, the LLM demonstrated significant limitations in detecting multi-parameter CDIs, finding only 7 inconsistencies with a recall of 5.6%. When extracted constraints and code were given as inputs, LLM+C demonstrated heightened awareness of the task and had a higher probability of locating the constraint-related code segments. However, it still struggled to determine inconsistency. Out of 126 inconsistent constraints, 52 were identified correctly, yielding a recall of 41.3%. After a thorough review of the LLM's responses, the analysis showed that LLM+C gave correct results in many cases but provided unreasonable or even wrong explanations. These results demonstrate that the LLM still has limitations in detecting complicated multi-parameter CDIs, highlighting that the

method's design is the key factor in enhancing detection performance rather than any reliance on potential pretraining data leakage.

Answer to RQ2: The large language model (LLM) exhibits limited capability in handling multi-parameter CDIs. Compared to LLM+C with a recall of 41.3%, FUZZY MPCHECKER successfully detected 117 out of 126 inconsistent constraints, achieving a recall of 92.8%. Notably, FUZZY MPCHECKER achieved a recall 23.8 percentage points higher than VANILLA MPCHECKER, substantiating the effectiveness of incorporating fuzzy words and fuzzy constraint logic.

4.4.2.3 Practical effect of MPCHECKER

The tool's effectiveness in detecting unknown multi-parameter inconsistencies was validated by manual review and developer feedback. Fourteen inconsistencies identified by MPCHECKER were reported to the library maintenance team and received positive engagement and warm responses. Two of them even sparked further discussions about potential issues. This not only affirmed the reports' quality but also reflected the enthusiasm of the open-source community.

For example, an issue confirmed by the `scikit-learn` team originates from the independent function "lars_path", as shown in Figure 4.9. Apparently, an inconsistency exists between the documentation and code regarding whether "Gram" is None when "X" is None. Therefore, the issue report [111] detailed the inconsistent documentation sections alongside the corresponding code snippet. The developer made a bit of archeology, admitted the documentation needed to be updated, and asked whether the reporters would submit a PR to correct this error. Finally, the documentation description was fixed to "If X is None, Gram must also be None".

For most issue reports, feedback was received promptly, and 11 inconsistencies were confirmed and improvements were made to documentation or code. Of the remaining three cases, two were reported at the initial phase of the experiments, when the understanding of the project architecture was still insufficient. The checks for these two constraints were implemented in other deeper files and could not be accurately verified at that time. The third inconsistency stemmed from ambiguity in the natural language, which resulted in a different interpretation that diverged

Constraint description of `X` and `Gram` in function `lars_path`

> **X** : None or ndarray of shape (n_samples, n_features)
Input data. Note that **if X is None then the Gram matrix must be specified**, i.e., cannot be None or False.

Corresponding code snippet in function `lars_path`

```
def lars_path(...):
    if X is None and Gram is not None:
        raise ValueError("X cannot be None if Gram is not None. Use
    lars_path_gram to avoid passing X and y.")
```

Fixed Constraint description of `X` and `Gram` in function `lars_path`

> **X** : None or ndarray of shape (n_samples, n_features)
Input data. Note that **if X is None, Gram must also be None.** If only the Gram matrix is available, use lars_path_gram instead.

FIGURE 4.9: Example of the fixed documentation from `Scikit-learn`.

from the developers' original intent. Furthermore, these reported issues have contributed to four DS/ML repositories (`scikit-learn`, `keras`, `statsmodels`, `dask`), which emphasizes the generalization of the tool.

At the time of writing, 10 out of 11 confirmed inconsistencies have been resolved: 7 through documentation fixes and 3 through updates to both documentation and code. This aligns with intuition: code errors are more likely to cause runtime failures and are thus easier to detect, whereas documentation errors and their potential efficiency impacts are often subtler and harder to identify.

Answer to RQ3: Fourteen multi-parameter inconsistencies detected by MPCHECKER were reported to library developers, who had confirmed 11 inconsistencies by the time of submission (confirmation rate = 78.6%) [111–121]. These results demonstrate that MPCHECKER can effectively detect unknown API documentation errors. Some of them are even in unseen libraries, which highlights its strong generalization capability.

4.5 Discussions

4.5.1 Threats to Validity

Internal Validity. There is no established ground truth for multi-parameter code-documentation inconsistencies. To mitigate this, inconsistencies detected by MPChecker were manually reviewed and verified. Given the complexity of multi-parameter constraints, two authors spent an additional 10 minutes per inconsistency to verify whether it was a true positive. Moreover, many of the confirmed true positives were further validated by the original library developers, enhancing the credibility of the manual labeling. Additionally, since the GPT-4 model used in the experiments had its last knowledge update in April 2023, and the first issue submissions occurred in February 2024, the risk of data leakage is not a significant concern for the approach.

External Validity. The tool may not fully generalize to all Python libraries, particularly those outside the data science domain. Although the approach is designed to be broadly applicable, the study focused on data science libraries due to several practical considerations: (1) they are among the most widely used in the Python ecosystem, (2) they commonly employ the two major docstring formats that MPChecker supports, and (3) they provide well-structured documentation with rich multi-parameter constraints. To enhance the representativeness of the evaluation, high-quality, widely adopted libraries with comprehensive API documentation and accessible source code were selected. A further limitation arises from the immature Python symbolic execution tools, which may not yet robustly handle all the latest language features. Addressing these challenges will require continued engineering efforts to broaden the applicability of the tool.

4.5.2 Application Prospects

The framework's language-agnostic design extends beyond dynamic languages like Python to languages such as Java, where type information facilitates more comprehensive CDI detection.

This study represents an effective integration of LLMs and traditional software analysis, with fuzzy constraint logic (FCL) acting as the glue that enables smooth synergy between the two. For example, LLMs have been recently used to infer program specifications from code [122]. In contrast to conventional verification techniques that yield binary outcomes, either `True` or `False`, FCL enables probabilistic evaluation of invariant validity within code contexts. This probabilistic framework will integrate better with LLMs, which may generate close yet incorrect program specifications. In particular, FCL can reduce non-logical errors caused by confusion between similar terms.

4.6 Related Work

API Documentation Analysis. Numerous empirical studies have revealed the challenges of maintaining high-quality API documentation [88, 123–133]. These studies indicate that documentation errors are prevalent, even in well-established and widely-used libraries. Additionally, an empirical study from Aghajani et al. [123] shows that linguistic antipatterns in APIs increase the likelihood of developers introducing errors and raising more questions compared to using clean APIs. Saied et al. [130] conducted an observational study focusing on API usage constraints and their documentation. Zhong and Su [134] proposed a method that combines natural language processing (NLP) with code analysis to identify errors in API documentation, specifically targeting grammatical mistakes (such as spelling errors) and incorrect code references (i.e., names that do not exist in the source code). Lee et al. [135] developed a technique to extract change rules from code revisions and apply them to detect outdated API names in Java documentation, with a particular focus on names of Java classes, methods, and fields.

Another related field is code-comment inconsistency [136–143]. Existing research on code comment analysis predominantly follows two approaches. The traditional method employs program analysis and heuristic rules to detect inconsistencies between comments and the code. Technologies like CUP [79], CUP2 [144], and HebCup [145] exemplify this approach, focusing on automatic just-in-time comment updates when corresponding code changes. The alternative approach leverages NLP techniques to retrieve and extract information from software artifacts.

MPChecker focuses on a distinct problem, specifically on API documentation errors arising from multi-parameter constraints. These issues are more subtle and challenging to detect, particularly within data science libraries built on the dynamic language Python.

LLM-based Program Analysis. A line of research [146–151] focuses on using LLMs for program analysis. Wadhwa et al. [146] focus on using LLMs to resolve code quality issues in multiple programming languages. Several recent studies [147–149] address applying LLMs to program repair. Nam et al. [150] apply the GPT model to explain code and provide usage details. The existing approaches focus on different purposes compared to MPChecker. Zhang et al. [151, 152] use LLMs to extract constraints from code comments, and apply AST-based program analysis to identify inconsistencies. Rong et al. [153] propose C4RLLaMA, a fine-tuned large language model based on the open-source Code Llama, to detect and correct code comment inconsistencies.

Overall, MPChecker's approach is distinct in two aspects. First, MPChecker specifically focuses on detecting inconsistencies in multi-parameter constraints, which is a missing piece in state-of-the-art works. Next, MPChecker deals with code documentation, which involves longer and more complex text, and is more diverse than most code comments.

Chapter 5

Repo-level Code Localization

5.1 Introduction

The rapid evolution of artificial intelligence is reshaping every stage of the Software Development Life Cycle (SDLC), fundamentally changing how developers interact with codebases. Through natural language interfaces, developers can now gain comprehensive insight into code repositories and delegate sophisticated tasks such as code refactoring, feature implementation, and defect repair. Central to these capabilities is *code localization*, which refers to the process of identifying source code snippets, ranging from individual methods to complex logic blocks, that are relevant to a natural language query within a large-scale repository. As the community pivots towards autonomous software engineering agents, effective code localization has become a critical primary bottleneck. Accurately mapping high-level functional intent to intricate implementation logic is no longer merely a retrieval problem, but a prerequisite for enabling AI systems to reliably navigate, reason about, and modify real-world software.

State-of-the-art code localization approaches can be broadly categorized into three paradigms: embedding-based retrieval, pipeline-guided workflows, and agent-driven exploration. These approaches have demonstrated impressive performance on issue-solving benchmarks such as SWE-bench [154]. However, the investigation in this study identifies a fundamental yet overlooked bias, termed the **Keyword Shortcut**. As the primary channel through which users report software failures to maintainers, issue reports often contain explicit debugging cues, such as stack

TABLE 5.1: Effect of keyword shortcuts on localization performance.

Level (Acc@1)	**w/o Keyword**	**w/ Keyword**	**Δ (pp)**
File	64.84%	89.04%	↑24.20
Module	47.66%	82.88%	↑35.22
Function	34.40%	63.57%	↑29.17

traces, distinctive class or function names, and even verbatim code snippets. These **keywords** effectively serve as "cheat sheets", enabling localization models to identify relevant code through lexical matching (e.g., using grep) rather than semantic reasoning. For example, in the filtered SWE-bench Lite benchmark, 146 of the 274 instances explicitly mention identifiers associated with the ground-truth location. Table 5.1 reports the performance of LocAgent [155], a representative code localization approach. On instances containing such keywords, LocAgent achieves improvements of 24.20, 35.22, and 29.17 Acc@1 percentage points at the file, module, and function levels, respectively, compared with instances without keyword cues.

This performance gap exposes a key limitation of the current localization paradigms: they often rely on lexical anchors and struggle when such anchors are unavailable or unreliable. To isolate this missing capability, this study introduces a new diagnostic task, **Keyword-Agnostic Logical Code Localization (KA-LCL)**, where models or agents must identify code locations satisfying explicit logical constraints rather than naming hints. Unlike general issue-solving code localization tasks, which can often degrade into lexical or semantic matching, KA-LCL represents a higher-order *structural reasoning* challenge. For example, a KA-LCL query may ask, *"Find all functions where: (1) the function has more than 15 parameters, and (2) the function is not an __init__ method."* Such queries are strongly constrained and leave little room for plausible guessing. Every predicted location must meet the required constraints, and the answer set is directly verifiable. Moreover, KA-LCL is not intended to mimic human querying behavior. Instead, it focuses on structural reasoning which is a prerequisite for handling latent semantic mappings. In this sense, it can be viewed as a building block for intent-driven tasks, such as *identifying complex APIs that should be refactored*.

To systematically evaluate the limitations of existing tools, this study constructs

KA-LogicQuery, a diagnostic benchmark for keyword-agnostic logical code localization. KA-LogicQuery complements issue-solving benchmarks for localization tasks by providing a controlled setting where queries are purely logical and devoid of naming hints. It decomposes code structures to synthesize logical queries targeting structural and behavioral code patterns. The evaluation shows a sharp performance drop in state-of-the-art methods, exposing a critical *reasoning gap* in current AI localization systems.

The difficulty of KA-LCL stems from two intertwined bottlenecks. ❶ **Without lexical anchors, localization requires searching a much larger repository context.** Existing localization methods often use identifiers as pruning signals to narrow the set of relevant files, classes, or functions. When such signals are absent, models are forced to process substantially broader structural contexts, which aggravates the *lost-in-the-middle* problem. Critical logical patterns and structural constraints become submerged within long input sequences, severely impairing the system's ability to robustly retrieve and utilize them. ❷ **Current methods lack deterministic repository-level structural reasoning.** Although pipeline-guided LLM workflows and agentic exploration attempt to model code relationships, they often operate as probabilistic recommendation systems that generate a ranked list of likely candidates, rather than performing rigorous structural deduction. As a result, these systems fall short of providing deterministic localization or verifiable rationales.

Conceptually, code localization can be viewed as a specialized form of code search [156]. Unlike general textual information, however, source code has formally defined syntax and semantics, enabling precise parsing and program analysis. This formal nature endows code with intrinsic reasoning affordances. From a high-level perspective, an effective repository-level localization engine requires a robust intermediate representation (IR) to bridge natural language intent and implementation logic. Such an IR should effectively capture code entities, dependencies, and structural hierarchies in an interpretable and actionable form for LLM-based agents.

To overcome the aforementioned limitations, this study proposes DataLoc, a neuro-symbolic framework for repository-level code localization that combines LLM-based intent interpretation with rule-based deterministic Datalog reasoning. DataLoc first employs static analysis to extract **program facts** from the target

repository, forming a structured IR that captures both elemental properties and relational dependencies. Given a natural language query, the LLM agent translates the intended constraints into a Datalog program over these facts. The program is then executed by **Soufflé**, a high-performance Datalog engine, to derive the code locations that satisfy the query. To improve the reliability of LLM-generated logic programs, DataLoc further incorporates a synthesize-check-refine loop with parser-gated validation, semantic repair, and mutation-based intermediate-rule diagnosis. Notably, the framework is broadly applicable to declarative logic languages, including CodeQL [157]. The framework uses Soufflébecause it provides LLM-friendly relational syntax, avoids the integration constraints of CodeQL's binary-only CLI and core engine, and supports rapid iterative execution of partial rules, which enables the mutation-based diagnostics to expose zero-row intermediate relations and likely over-constraining clauses.

Crucially, by offloading structural reasoning to a deterministic engine, DataLoc provides verifiable answer sets rather than fixed top-n recommendations, while reducing execution time and token consumption. This design also enables the agent to return definitive negative responses when no matches exist, avoiding a common pitfall of probabilistic systems that hallucinate plausible but incorrect candidates.

In summary, the contributions of this chapter are as follows.

1. This study identifies the *Keyword Shortcut* bias in issue-solving benchmarks and introduces a new diagnostic benchmark, KA-LogicQuery, for Keyword-Agnostic Logical Code Localization (KA-LCL). It contains 225 logic-intensive queries across 9 projects, along with KA-LogicQuery-Neg, an empty-ground-truth variant for evaluating abstention behavior.

2. This study proposes DataLoc, a neuro-symbolic framework that represents repositories as program facts and synthesizes Datalog programs from natural language constraints. By delegating structural traversal and logical filtering to a deterministic reasoning engine, DataLoc enables verifiable repository-level localization cost-effectively.

3. An execution-guided synthesize-check-refine mechanism is designed for LLM-generated Datalog programs, combining parser-gated validation and repair

with conservative mutation-based intermediate-rule feedback. The framework is implemented as an automated end-to-end command-line tool; the tool and benchmark are available at: `https://zenodo.org/records/22844061`.

4. DataLoc is evaluated against state-of-the-art localization methods. The results show that DataLoc performs strongly on KA-LogicQuery, achieving 66.98% precision and 58.67% success rate at the fine-grained function level, whereas the best-performing baselines reach only 5.87% precision and 11.73% success rate. DataLoc also maintains robust abstention behavior by explicitly returning *no match found* in over 70% of KA-LogicQuery-Neg cases. On SWE-bench Lite, DataLoc remains competitive while returning concise verifiable locations instead of top-n recommendations.

5.2 Motivating Example

Background on program facts and Datalog is provided in Section 2.2.2 of Chapter 2. This section illustrates their application to code localization through a motivating example. Consider a KA-LCL task that requires identifying functions in a codebase satisfying specific structural constraints: (1) the function has more than 15 parameters, and (2) the function is not an __init__ method.

Figure 5.1 demonstrates a basic localization process. Given a natural language query, the Large Language Model translates the question's intention into a formal Datalog program. The generated program operates over two types of facts: *Extensional Database (EDB)* facts extracted directly from source code, such as `function_definition` containing metadata about each function's location, parameters, and context; and *Intensional Database (IDB)* facts derived through logical inference, such as `LargeFunctions`. The Datalog query defines an inference rule (lines 8–11) that identifies target functions by matching against `function_definition` facts. Attributes whose values are not needed in the rule are represented by the anonymous variable "_", while the rule filters for functions with param_count $>$ 15 and excludes constructors with either unqualified or class-qualified names.

When executed by the Soufflé Datalog engine, the query precisely localizes two functions meeting the specified criteria: the `convolve_fft` function

Question

Find all functions where: (1) the function has more than 15 parameters, and (2) the function is not an __init__ method.

❶ Generated by LLM

Datalog Query

```
% EDB: Facts extracted from source code
.decl function_definition(file_path: symbol, function_name: symbol,
    start_line: number, end_line: number, param_count: number,
    is_async: symbol, containing_class: symbol)

% IDB: Derived analytical facts
.decl LargeFunctions(file_path: symbol, function_name: symbol,
    start_line: number, param_count: number, containing_class: symbol)

% Inference rule for localization
LargeFunctions(file_path, function_name, start_line, param_count,
    containing_class) :-
    function_definition(file_path, function_name, start_line, _,
        param_count, _, containing_class),
    param_count > 15,
    function_name != "__init__".

% Query
.output LargeFunctions
```

❷ Executed by Soufflé

Query Result

file_path	function_name	start_line	param_count	containing_class
astropy/convolution/convolve.py	convolve_fft	442	19	module_level
astropy/io/fits/column.py	_verify_keywords	952	17	Column

FIGURE 5.1: A motivating example of a logic query.

with 19 parameters at line 442 in `astropy/convolution/convolve.py`, and the `Column._verify_keywords` method with 17 parameters at line 952 in `astropy/io/fits/column.py`. This example highlights the key advantages of the approach: the LLM bridges the semantic gap between natural language and formal logic, while the Datalog engine evaluates the generated rules over the extracted program facts to identify matching locations, enabling precise localization without exhaustive manual repository traversal. A complete example involving extracted facts, query refinement, and source verification is presented in Section 5.3.5.

5.3 Methodology

This section first formalizes the repository-level code localization problem. It then analyzes the intrinsic challenges that motivate the hybrid approach, DataLoc, which combines the strengths of large language models with the rigorous logical reasoning of Datalog.

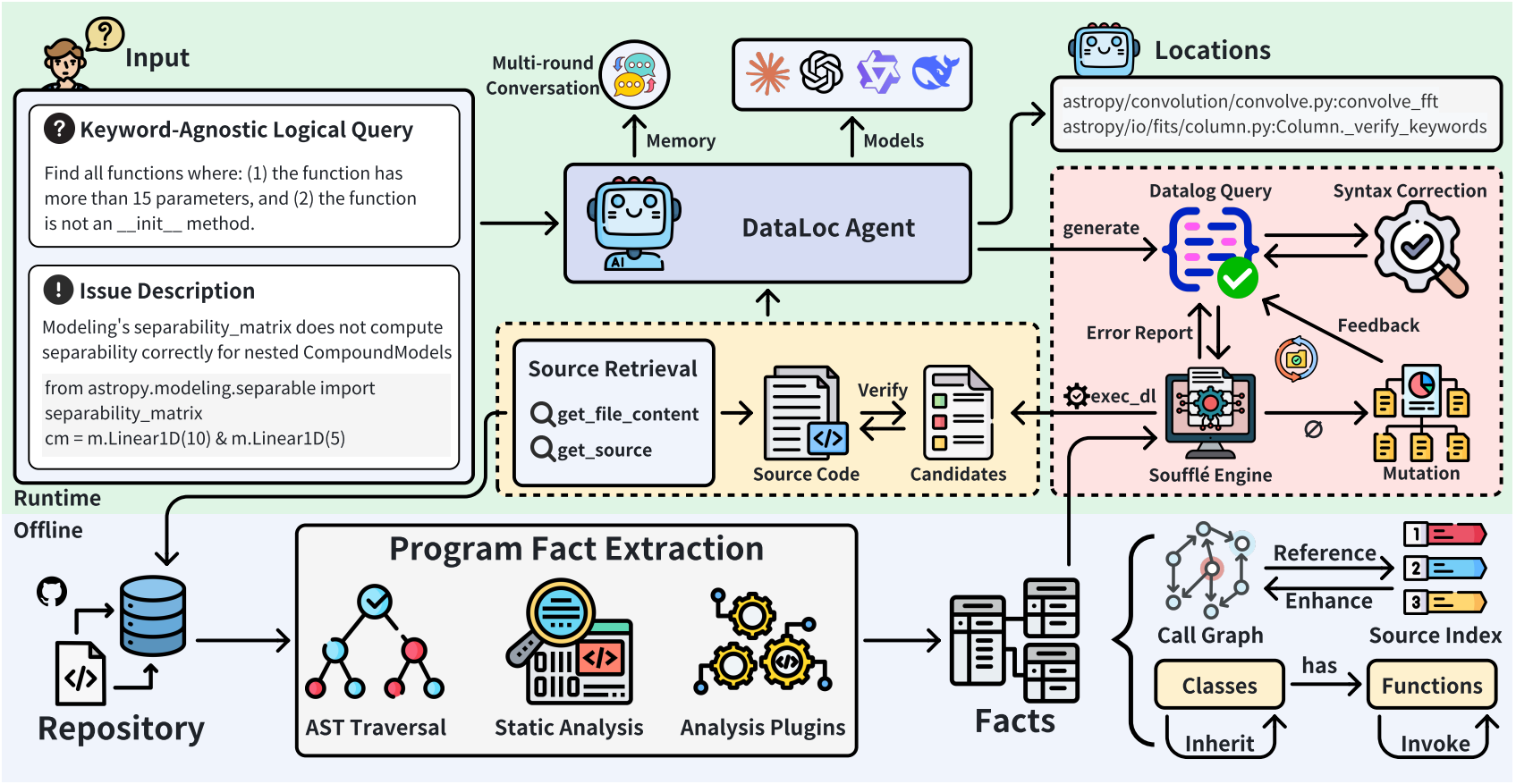


FIGURE 5.2: Overview of the DATALOC framework.

Given a natural language query q (e.g., a bug report or feature request) and a target codebase $\mathcal{C}$, the goal of **repository-level code localization** is to identify a list of relevant code locations $\mathcal{L} = \{l_1, l_2, \ldots, l_k\}$. Effective localization bridges the gap between informal natural language and the rigid execution logic of software. Three primary challenges are identified:

- **Challenge 1: Semantic and Lexical Disconnect.** There exists a non-trivial gap between the informal vocabulary of q and the formal identifiers in $\mathcal{C}$. This disconnect is categorized into three progressive layers: (1) *Keyword Absence*, where a query lacks any direct textual anchors present in the code; (2) *Latent Semantic Mapping*, where high-level task descriptions (e.g., `login failure`) lack direct textual overlap with low-level implementation entities (e.g., `AuthManager` or `ValidateToken`); (3) *Lexical Divergence*, where developers use synonyms or abbreviations (e.g., `fqn` for `fullyQualifiedName`) that elude exact match search.
- **Challenge 2: Non-local Structural Dependencies.** Relevant code is often not directly mentioned in queries but connected through dependencies or calls. A single feature may be implemented across multiple files. For example, identifying *password validation* logic may require traversing complex call chains and data flows scattered across authentication modules and database layers. Existing pipeline-based tools rely on local directory traversal and fail to capture these deep, cross-file relational dependencies.
- **Challenge 3: Complex Logical Pattern Constraints.** Questions may contain logical pattern requirements that candidate code must satisfy. Certain localization tasks are defined by structural patterns rather than keywords. For example, *identifying the conditional statement that raises three distinct error types in different branches* requires satisfying specific logical constraints defined by the language's syntax and semantics. Such patterns are difficult to locate by retrieving class or function information alone; they require repository-level structural understanding and reasoning.

To address these challenges, this study proposes DataLoc, a synergy between LLMs and Datalog. The underlying intuition is that LLMs excel at resolving *Challenge 1* by translating vague natural language intents into structured requirements, while Datalog provides the relational and reasoning power to solve *Challenge 2*

and *Challenge 3* by evaluating relational constraints over the extracted facts. In the proposed framework, the codebase $\mathcal{C}$ is represented as a set of pre-extracted program facts $\mathcal{F}$, where $\mathcal{F}$ captures both the source entities and the structural relations, such as function-call information, class membership, and declared inheritance relationships. Each location $l_i \in \mathcal{L}$ corresponds to a specific level of granularity, such as a file, module, or function, that is essential for resolving the query q. Figure 5.2 illustrates the overview of the framework of DataLoc. The framework operates in two stages. First, an offline program fact extraction stage analyzes the codebase to build a structured knowledge base. Second, an automated agent execution stage leverages these extracted facts to perform code localization by synthesizing high-quality Datalog queries.

5.3.1 Program Facts Extraction

This subsection describes the details of extracting program facts from a given repository. Program facts are extracted through a modular pipeline that separates source discovery, structural parsing, and fact emission, shown as the offline *Program Fact Extraction* stage in Figure 5.2. First, the extraction pipeline enumerates source files under the repository root using the configured file-selection rules. A language-specific frontend then analyzes the files using the most suitable intermediate representation. For Python, the frontend parses source code into an abstract syntax tree and emits facts during traversal. The frontend produces Datalog facts annotated with precise source locations and stable identifiers, enabling traceability and incremental updates.

These frontends emit facts describing program entities (files, modules, functions, classes, variables) and relations (containment, inheritance, import/use, call, and reference edges), together with optional control flow and data flow information when available. This representation directly addresses *Challenge 2* by making cross-file interactions explicit and queryable [158]. Instead of relying on local directory traversal, localization can now operate over global dependency graphs, enabling the identification of code relevant to a feature even when it is scattered across multiple modules and layers.

The extracted facts also enable expressive logical pattern matching, which helps solve *Challenge 3*. Structural requirements, such as specific exception patterns or

multi-step call sequences, can be formulated as Datalog queries over extracted facts. This allows localization tasks defined by program structure rather than surface keywords to be handled systematically, without requiring the LLM to reason over raw source code.

Relational schema and extraction semantics. Following the relational representation introduced in Section 2.2.2, the Python fact interface used in this study comprises 17 relations. Table 5.2 lists these relations with their original argument names and order. Each relation contains a finite set of tuples extracted from the analyzed repository snapshot, with argument types specified below. Line numbers, column positions, span bounds, counts, and parameter indices have type `number`; all other arguments have type `symbol`, including Boolean flags represented as `True` and `False`.

TABLE 5.2: Python program-fact relations and their arguments.

Relation	Arguments in declaration order
`source_file`	`file_path`, `content_hash`, `total_lines`
`source_line`	`file_path`, `line_number`, `content`
`ast_node`	`file_path`, `line`, `column`, `node_type`, `parent_line`, `parent_column`
`class_definition`	`file_path`, `class_name`, `start_line`, `end_line`, `base_class`
`class_nesting`	`file_path`, `nested_class`, `parent_class`, `line`
`class_method`	`file_path`, `class_name`, `method_name`, `line`, `is_static`, `is_class_method`
`class_attribute`	`file_path`, `class_name`, `attribute_name`, `line`, `attribute_type`
`function_definition`	`file_path`, `function_name`, `start_line`, `end_line`, `param_count`, `is_async`, `containing_class`
`function_parameter`	`file_path`, `function_name`, `function_line`, `param_name`, `param_type`, `default_value`, `param_index`
`function_decorator`	`file_path`, `function_name`, `function_line`, `decorator_name`
`function_call`	`file_path`, `calling_function`, `line`, `called_function`, `arg_count`, `call_type`
`type_annotation`	`file_path`, `entity_name`, `line`, `type_expression`, `annotation_type`
`variable_definition`	`file_path`, `variable_name`, `line`, `variable_type`, `initial_value`, `scope_name`
`import_statement`	`file_path`, `line`, `imported_name`, `local_alias`, `import_type`
`control_structure`	`file_path`, `structure_type`, `start_line`, `end_line`, `condition`
`exception_handling`	`file_path`, `line`, `exception_types`, `handler_description`
`docstring`	`file_path`, `entity_name`, `line`, `content`, `docstring_type`

Paths are relative to the repository root, and source spans include both boundary lines in the analyzed snapshot. Method names are class-qualified, as in `TimeSeries.__init__`. A `control_structure` fact records the construct kind, full syntax-tree span, and normalized condition; an `if` span therefore includes its alternative branches. The `exception_handling` relation describes exception-handling constructs, not explicit `raise` statements.

The facts provide a syntactic representation rather than complete control-flow or data-flow graphs. Calls are recorded without resolving dynamic targets, and condition summaries may omit parts of compound expressions. Constraints not preserved by this representation, such as whether two raises occur in different branches, require source verification, as illustrated in Section 5.3.5.

5.3.2 Agentic Workflow

This section elaborates on the agent-based workflow for automated repository-level code localization, which builds upon the program facts constructed offline.

The end-to-end agent is designed to accept natural language queries and return precise code locations. Unlike approaches that provide a fixed top-n list of candidates, the system outputs a dynamic set of potential locations to maximize precision. To mitigate hallucinations and enhance abstention ability, the framework decouples reasoning from generation. A deterministic engine handles inference while the LLM acts as a coordinator for query analysis and result calibration. The agent is equipped with three basic tools: `exec_dl` for executing Datalog programs, and `get_file_contents` along with `get_sources` for retrieving source code and specific line ranges.

5.3.2.1 Query Analysis and Information Resolution

The workflow begins with a preliminary analysis of the query to extract the core technical concepts and structural elements, performed by the *DataLoc Agent* in Figure 5.2. By identifying program entities such as file paths, module names, and structure descriptions, the agent establishes an internal context. This preparatory step provides the necessary predicates and constraints for the subsequent synthesis of Datalog programs.

5.3.2.2 Synthesize-Check-Refine Loop

As shown in the red panel in Figure 5.2, before execution, DataLoc follows a *synthesize-check-refine* loop to mitigate the impact of hallucinations and improve the executability of LLM-generated Datalog programs. It mainly contains two

critical, feedback-driven phases (details in Section 5.3.3 and Section 5.3.4): (1) *Syntax correction.* Each synthesized program undergoes parser-gated validation to ensure syntactic well-formedness. The workflow adopts a *best-effort repair, then fallback* strategy: unambiguous cases are handled via conservative rule-based fixes, revalidated with Soufflé's parser, and all other cases return error feedback to the LLM. (2) *Intermediate-rule diagnosis.* The diagnostic mechanism instruments the program to track row counts of intermediate relations, thereby identifying rules that produce empty results. Controlled mutation probes then classify such emptiness as fragile, stable, or inconclusive, giving the model targeted evidence for refinement. Validated programs are then executed through the `exec_dl` tool to generate a list of candidates. Excessive locations are treated as failures, triggering refinement with stricter constraints to reduce noise and improve precision.

5.3.2.3 Context Retrieval and Final Verification

As depicted in the yellow panel in Figure 5.2, once a set of potential code locations has been determined, the agent retrieves the relevant code snippets using `get_sources` or `get_file_contents` and conducts a verification against the original query. These verified locations are then returned in a standardized format (e.g., `FILE_PATH:CLASS.METHOD`). Although the agent may undergo multiple internal reasoning iterations, the user experience is streamlined into a single step: submitting a query and receiving a list of locations. This fully automated closed-loop design ensures both usability and seamless integration into production development environments.

5.3.3 Repairing LLM-Generated Datalog

LLM-generated Datalog programs (the dialect used by Soufflé) often contain syntactic and semantic errors, especially when the model is not fine-tuned for Datalog programming.

As shown in the red panel in Figure 5.2, before executing an LLM-generated Datalog program, the framework enforces a *parser-gated validation* workflow to ensure that only syntactically well-formed programs reach later stages of the pipeline. This design is motivated by the observation that some failures are caused by superficial

syntactic issues that can be repaired mechanically, whereas more complex parse failures often require global restructuring that is better handled by the LLM. The workflow therefore follows a *repair-or-feedback* strategy: the workflow applies conservative rule-based fixes when the repair is unambiguous, re-checks the program using Soufflé's parser, and otherwise returns error feedback to the LLM.

The workflow invokes a lightweight parser helper (based on Soufflé's parsing frontend) to validate the generated program. If parsing fails, the workflow first attempts a small set of mechanical rewrite rules targeting high-frequency issues. For example, LLMs frequently introduce naming collisions by using reserved or special identifiers as variable names. One common case is using `count` as a variable name while it is a reserved keyword in Soufflé. Such cases can be fixed locally via deterministic renaming.

However, not all parser errors admit a reliable deterministic repair. In these cases, the workflow does not attempt speculative transformations. Instead, the workflow returns the raw parser diagnostics (optionally augmented with concise hints) to the LLM, allowing the model to revise the program directly.

Any program that fails parser validation is rejected and never proceeds to later stages. Once a program passes syntactic validation (either initially or after rule-based repair), the workflow applies semantic correctness checks that encode Soufflé-specific usage rules and common domain conventions. These checks detect syntactically valid but semantically incorrect programs arising from common misconceptions in LLM-generated Datalog code. Only programs that satisfy both syntactic and semantic validation proceed to execution.

A representative example is the use of string containment constraints. Soufflé provides a constraint function `contains(sub:symbol,full:symbol)`, which is defined such that the second argument must contain the first (i.e., full includes sub). However, LLMs are observed to frequently invert this positional relationship by producing atoms like `contains(content,“keyword”)` instead of the correct `contains(“keyword”,content)`. Since such inversions conform to both the syntax and type specifications, they often lead to silent failures or empty results that are difficult for LLMs to self-correct, even after multiple rounds of feedback.

Prompt techniques such as few-shot learning cannot effectively eliminate this kind of error. Therefore, a library of semantic rules is constructed to check the correct

usage of built-in predicates with non-commutative argument semantics and other similar constraints. These repairs are applied only when the transformation is high-confidence and semantics-preserving. When uncertain, the checker records the issue for subsequent feedback to the LLM rather than applying a blind fix, thereby guiding the refinement in subsequent iterations. The contribution of this mechanism is evaluated through an ablation study in Section 5.4.3.4, demonstrating its effectiveness in improving synthesis quality.

5.3.4 Diagnosing Intermediate Rules via Conservative Mutation Analysis

An intermediate-rule diagnostic mechanism is introduced for LLM-generated Datalog programs, corresponding to *Mutation* in the red panel of Figure 5.2. As illustrated in Figure 5.3, rather than evaluating a program solely by its final output, DataLoc instruments intermediate derived relations, records their cardinalities, and applies diagnostic mutations to relations that derive zero tuples. Since an empty relation may represent either a legitimate result or an overly restrictive rule, these mutations serve only as diagnostic probes. Their outputs are used to localize the source of failure and are never returned as part of the final answer.

Mutation catalog. For a zero-cardinality relation R, the diagnostic mechanism mutates only rules that define R. The catalog is prioritized and capped for cost control; each mutation preserves the rule head and changes only the body.

- M1. Contains-Literal: replace each body string literal `"s"` with a fresh variable V_i and append `contains("s", `V_i`)`, relaxing exact equality to substring matching.
- M2. Drop-Single-Atom: remove one selected body atom at a time while retaining the rest. This general ablation is applied only when a rule has at least two body atoms and is bounded by a small variant cap.
- M3. Match-Literal: replace each body string literal with V_i and append `match("s.*", `V_i`)`, testing whether the literal is a prefix of the actual fact value.

- M4. DROP-CONTAINS: remove one `contains(...)` atom at a time, isolating overly strict string filters.
- M5. DROP-SINGLE-NEGATION: remove one negated atom (`!pred(...)`) at a time, probing whether an exclusion constraint is too strong.

M1 and M3 apply only when quoted string literals occur in the rule body; M2, M4, and M5 apply only when their target atom exists and the remaining body still contains at least one atom.

Feedback construction. After applicable probes run, DATALOC classifies each zero-cardinality relation as: *fragile-empty* if any successful mutation yields rows; *stable-empty* if at least one mutation succeeds and all successful mutations remain empty; and *inconclusive* if no mutation applies or all probes fail. Feedback reports the relation name, original zero cardinality, mutation identity, relaxed or removed constraint, rewritten rule, resulting cardinality, and optionally a small sample of surfaced tuples. This gives the LLM evidence for revision without silently weakening the user's intended query semantics.

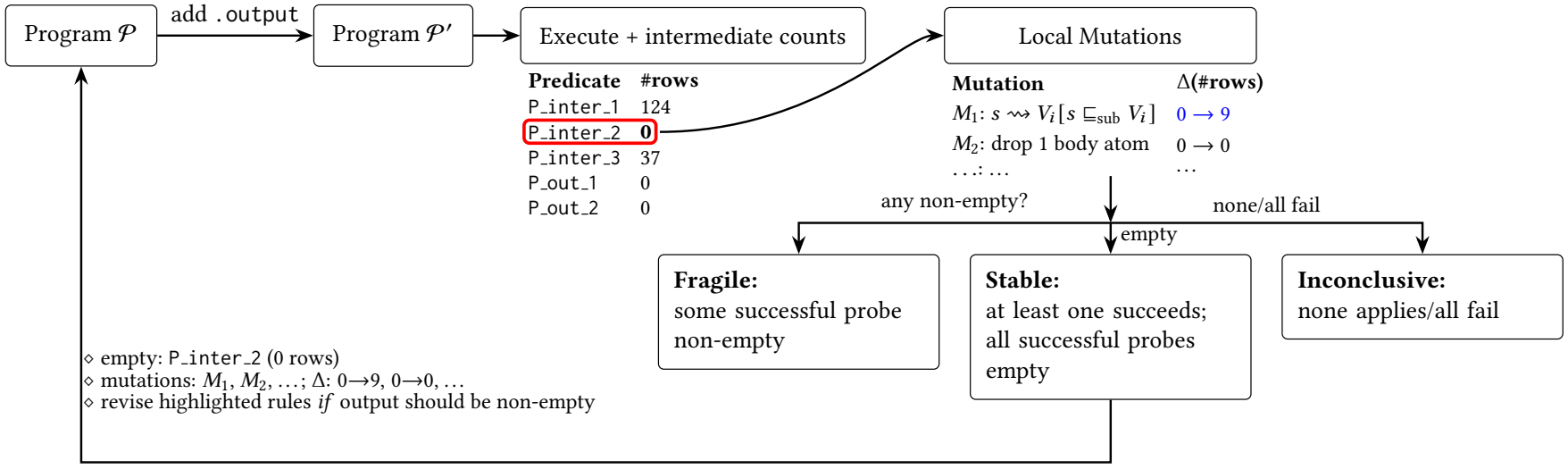


FIGURE 5.3: Intermediate-rule mutation and feedback.

5.3.5 End-to-End Localization Example

Using Astropy commit `d16bfe05a744909de4b27f5875fe0d4ed41ce607`, this example traces a recorded localization interaction through the corresponding source code and extracted program facts.

Original user query. The user submits the following localization request:

> Find all classes where: (1) the `__init__` method contains an if statement that checks whether a parameter is None, (2) one branch of this if statement raises TypeError and the other branch raises ValueError

Relevant facts extracted from the codebase. Figure 5.4a and b show relevant program facts for `TimeSeries` and `WCS`, respectively, following the schema in Table 5.2 and its accompanying description. These facts record the line ranges of `__init__` methods and conditionals, together with source lines containing explicit `raise` statements.

Extracted facts: TimeSeries

```
function_definition("astropy/timeseries/sampled.py", "TimeSeries.__init__", 61, 135, 7, False, "TimeSeries").
control_structure("astropy/timeseries/sampled.py", "if_statement", 107, 130, "time_start IsNot None").
source_line("astropy/timeseries/sampled.py", 113, "raise TypeError(\\"'time' is scalar, so 'time_delta' is required\\")").
source_line("astropy/timeseries/sampled.py", 125, "raise ValueError(\\"Length of 'time' ({}) should match \\"").
```

(A) Extracted facts for `TimeSeries`.

Extracted facts: WCS

```
function_definition("astropy/wcs/wcs.py", "WCS.__init__", 380, 546, 12, False, "WCS").
control_structure("astropy/wcs/wcs.py", "if_statement", 391, 527, "header Is None").
source_line("astropy/wcs/wcs.py", 429, "raise TypeError(").
source_line("astropy/wcs/wcs.py", 413, "raise ValueError(").
```

(B) Extracted facts for `WCS`.

FIGURE 5.4: Relevant program facts from the Astropy repository.

Initially generated Datalog query. Figure 5.5a shows the initial query generated by the agent. `InitFunc` targets `__init__` methods, and `IfNone` targets enclosed conditionals that test for `None`. The variable pairs `f_start`/`f_end` and `if_start`/`if_end` denote their inclusive line bounds. Argument types are specified in Section 5.3.1; `_` denotes an anonymous variable.

Checking, repair, and refinement. Exact matching against `__init__` yields no `InitFunc` tuples because method names are class-qualified. The empty result triggers a Contains-Literal diagnostic mutation (Section 5.3.4), which replaces exact matching of `__init__` with substring matching. The mutated query produces a nonempty result, yielding *fragile-empty* feedback that identifies the name constraint as a possible cause. Guided by this feedback, the agent inspects the recorded method names and revises the query to match the substring `.__init__`.

The next candidate query introduces `RaiseT` and `RaiseV` for lines containing `TypeError` and `ValueError` raises, and `HasBoth` for conditional spans containing both. It retrieves `TimeSeries` and `WCS`. However, finding both exception types within the same conditional span does not establish that they occur in different branches.

Figure 5.5b presents the final query after further agent revisions. Here, `ST` denotes the recorded control-structure type, `c` the source-line context, and `l1` and `l2` the line numbers of the matched `TypeError` and `ValueError` raises, respectively. Compared with the preceding candidate query, it accepts either `if_statement` or `unknown`, requires only the conditional's starting line to fall within the `__init__` method, and changes the lower-bound checks on raise locations from `>` to `>=`. These changes relax the candidate-selection constraints but return the same two candidates. Source verification is therefore still required to determine whether the raises occur in different branches.

Both panels retain the submitted `contains` argument order, corrected to Soufflé's substring-first convention by semantic validation before execution (Section 5.3.3). The figure omits only the shared fact-input include directive.

Initially generated query

```
.decl InitFunc(file_path: symbol, class_name: symbol, f_start: number, f_end: number)
InitFunc(file_path, containing_class, f_start, f_end) :-
    function_definition(file_path, "__init__", f_start, f_end, _, _, containing_class),
    containing_class != "".
.decl IfNone(file_path: symbol, class_name: symbol, if_start: number, if_end: number, condition: symbol)
IfNone(file_path, class_name, if_start, if_end, condition) :-
    InitFunc(file_path, class_name, f_start, f_end),
    control_structure(file_path, "if_statement", if_start, if_end, condition),
    contains(condition, "None"),
    if_start >= f_start, if_end <= f_end.
.output IfNone
.output InitFunc
```

(A) Initial query for `__init__` methods and enclosed conditionals.

Final query submitted by the agent

```
.decl InitFunc(file_path: symbol, class_name: symbol, f_start: number, f_end: number)
InitFunc(file_path, containing_class, f_start, f_end) :-
    function_definition(file_path, function_name, f_start, f_end, _, _, containing_class),
    containing_class != "", contains(function_name, ".__init__").
.decl IfNone(file_path: symbol, class_name: symbol, if_start: number, if_end: number, condition: symbol)
IfNone(file_path, class_name, if_start, if_end, condition) :-
    InitFunc(file_path, class_name, f_start, f_end),
    control_structure(file_path, ST, if_start, if_end, condition),
    (ST = "if_statement" ; ST = "unknown"),
    contains(condition, "None"), if_start >= f_start, if_start <= f_end.
.decl RaiseT(file_path: symbol, line: number)
RaiseT(file_path, line) :- source_line(file_path, line, c), contains(c, "raise TypeError").
.decl RaiseV(file_path: symbol, line: number)
RaiseV(file_path, line) :- source_line(file_path, line, c), contains(c, "raise ValueError").
.decl HasBoth(file_path: symbol, class_name: symbol, if_start: number, if_end: number, condition: symbol)
HasBoth(file_path, class_name, if_start, if_end, condition) :-
    IfNone(file_path, class_name, if_start, if_end, condition),
    RaiseT(file_path, l1), l1 >= if_start, l1 <= if_end,
    RaiseV(file_path, l2), l2 >= if_start, l2 <= if_end.
.output HasBoth
```

(B) Complete final query, before semantic validation.

FIGURE 5.5: Initial and complete final queries from the Astropy interaction, with original identifiers and constraints.

Localization result. After semantic validation, the final query returns the following two candidates:

Repository-relative path	Class	Start	End	Condition
`astropy/timeseries/sampled.py`	`TimeSeries`	107	130	`time_start IsNot None`
`astropy/wcs/wcs.py`	`WCS`	391	527	`header Is None`

The LLM retrieves the corresponding source code and checks each candidate against the original request. It excludes `WCS` because both exception types occur within the same `else` subtree, rather than in different branches of the conditional. `TimeSeries` satisfies the branch constraint, yielding the final location:

Final localization: `astropy/timeseries/sampled.py:TimeSeries`

5.4 Evaluation

To evaluate the effectiveness and practicality of the approach, the following research questions are formulated:

1. **RQ1:** How effective is DataLoc in keyword-agnostic logical code localization?
2. **RQ2:** How effective is DataLoc for issue-based code localization?
3. **RQ3:** How efficient is DataLoc compared to baselines?
4. **RQ4:** How does each component of DataLoc contribute to its performance?

5.4.1 Benchmarks and Baselines

SWE-bench Lite [154] is a curated subset of SWE-bench for cost-effective evaluation of autonomous issue resolution. Each instance contains a real-world GitHub issue, repository metadata, and patch-derived ground-truth locations. Following Suresh et al. [159], 274 of the 300 instances whose patches modify existing functions or classes are retained, and those that introduce new functions or imports are excluded, thereby focusing evaluation on localization within existing code structures.

KA-LogicQuery is a diagnostic benchmark for KA-LCL, containing 225 logic-intensive queries across 9 projects. Each query is formulated as a composite logical proposition over code features across multiple dimensions, such as structural granularity (e.g., classes, methods), behavioral attributes (e.g., control flow, exception handling), code metrics (e.g., inheritance depth and branch count), as summarized in Table 5.3. From the large feature-composition space induced by these dimensions, 25 representative combinations are selected and each is instantiated on 9 repositories, with only minimal query adjustments to ensure at least one valid ground-truth location.

Table 5.3: Taxonomy of Python Code Dimensions and Representative Elements.

Query Dimensions	**Examples / Typical Elements**
Code Structure	Functions, Methods, Classes, Modules, Decorators
Control Flow	Conditional (`if-elif-else`), Iteration (`for`, `while`), Context Management (`with`)
Condition Logic	Comparison (`==`, `>`), Identity (`is`), Membership (`in`), Type Checks (`isinstance`), Logical Operators (`and`, `or`, `not`), Early Exit (`return`, `break`, `continue`)
Data Structure	Built-in Collections (`list`, `dict`, `set`), Primitive Types (`int`, `str`)
Function Signatures	Default Values, Variadic Parameters (`*args`, `**kwargs`), Type Annotations
Exception Handling	Exception Propagation (`try-except-finally`), Exceptions (`TypeError`)
Code Metrics	Nesting Depth, Inheritance Depth, Assertion Count, Branch count

KA-LogicQuery-Neg, a variant of KA-LogicQuery, is designed to evaluate abstention capability when no valid location satisfies a query. Since this ability can be tested without using the full benchmark scale, KA-LogicQuery-Neg is constructed by sampling a single repository version and deliberately modifying its queries to ensure the corresponding ground-truth sets are empty. Existing methods often rely on top-n ranking to maximize recall, but an ideal localization tool should not only provide well-supported answers but also explicitly abstain when no valid answer can be ascertained, thereby avoiding false positives. Such a *refusal mechanism* serves as a critical metric for the robustness of autonomous agents in production environments.

DataLoc is compared with four state-of-the-art baselines:

- **SweRank** [160] (*Embedding*): It utilizes a retrieve-and-rerank architecture to identify issue locations. It employs SWERankEmbed (137M/7B parameters) to perform initial retrieval and SWERankLLM (7B/32B parameters) to rerank the results.
- **Agentless** [161] (*Pipeline*): This approach employs a hierarchical filtering strategy within a procedural workflow. It progressively prunes the search space from the file level down to specific classes or functions, utilizing an LLM to rank and select candidates at each stage.
- **LocAgent** [155] (*Agentic*): It constructs a graph-based representation and sparse indexes of the project and enables an autonomous agent to perform iterative, tool-assisted retrieval.
- **CoSIL** [162] (*Agentic*): This framework focuses on structural dependency traversal through call graphs to identify implicit locations via iterative exploration. It incorporates pruning to maintain context efficiency and restrict the search to high-relevance execution paths.

All experiments were conducted on an Ubuntu 22.04.5 LTS server with an Intel Xeon Silver 4216 CPU (2.10 GHz) and 62 GB RAM. The framework was implemented in Python 3.12.11 with the Soufflé2.4 Datalog engine. DataLoc was evaluated using `gpt-4o-20240513` via the OpenAI API, `claude-3-5-sonnet-20241022` via AWS Bedrock, and `Qwen3-Max` via Alibaba Cloud, using a temperature of 0.3 and a maximum of 20 iterations. The SWE-bench Lite comparison in Table 5.5 additionally reports results for GPT-5.1. Baselines ran in their official environments with required dependencies for fair comparison.

5.4.2 Metrics

Localization performance is evaluated at three levels of granularity: *file, module, and function*. Let Q denote the set of query instances, G_q the set of ground-truth locations for query $q \in Q$, and $\mathcal{P}_q$ the set of predicted locations inferred by the agent workflow. $\mathbf{1}(\cdot)$ denotes the **indicator function**, which equals 1 if the logical condition holds and 0 otherwise. The evaluation uses the following six metrics:

(1) Accuracy@k (ACC@k): It measures the ability to achieve full coverage, where a success requires all ground-truth locations to be present within the top-k predicted locations. When k equals the length of the prediction set, this metric becomes the **Success Rate (SR)**:

$$Acc@k = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1}(G_q \subseteq \mathcal{P}_{q,k}) \tag{5.1}$$

(2) Recall (REC): It represents the proportion of ground-truth locations successfully captured for each query, averaged across queries:

$$REC = \frac{1}{|Q|} \sum_{q \in Q} \frac{|G_q \cap \mathcal{P}_q|}{|G_q|} \tag{5.2}$$

(3) Precision (PRE): This metric penalizes overprediction by calculating the fraction of predicted locations that are correct:

$$Pre = \frac{1}{|Q|} \sum_{q \in Q} \frac{|G_q \cap \mathcal{P}_q|}{|\mathcal{P}_q|} \tag{5.3}$$

(4) Average Jaccard Similarity (AJS): It quantifies the overlap between the predicted and ground-truth sets, which penalizes both missing targets and redundant predictions:

$$AJS = \frac{1}{|Q|} \sum_{q \in Q} \frac{|G_q \cap \mathcal{P}_q|}{|G_q \cup \mathcal{P}_q|} \tag{5.4}$$

(5) Perfect Localization Rate (PLR): The most stringent metric, measuring the ratio of instances where the predicted set $\mathcal{P}_q$ exactly matches the ground-truth set G_q. A PLR of 1.0 indicates perfect localization without any extraneous noise (i.e., $AJS = 1.0$):

$$PLR = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1}(\mathcal{P}_q = G_q) \tag{5.5}$$

(6) Hit Rate (HR): The most lenient metric, measuring the ratio of instances where the predicted set $\mathcal{P}_q$ provides at least one correct location:

$$HR = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1}(\mathcal{P}_q \cap G_q \neq \emptyset) \tag{5.6}$$

5.4.3 Results

5.4.3.1 Effectiveness on Logic Queries

As shown in Table 5.4, DataLoc consistently outperforms all baselines across metrics and localization granularities on KA-LogicQuery. At the file level, DataLoc with Qwen3-Max achieves the best Precision (PRE) and Success Rate (SR), reaching 73.35% and 68.44%, respectively. It also attains a Perfect Localization Rate (PLR) of 48.44%, whereas all baselines remain near zero. Even at the finest function-level granularity, DataLoc still maintains a PLR of 38.27%, while all baselines drop to 0%. These results highlight the advantage of DataLoc in capturing code structure and reasoning capabilities in assisting precise localization. DataLoc maintains strong performance across both evaluated foundation models. Switching the underlying model from Claude-3.5 to Qwen3-Max yields gains across most localization metrics at all three granularities, although hit rate decreases slightly.

Table 5.4: Evaluation results on KA-LogicQuery.

Method	Model	File Level (%)						Module Level (%)						Function Level (%)					
		SR	REC	PRE	AJS	PLR	HR	SR	REC	PRE	AJS	PLR	HR	SR	REC	PRE	AJS	PLR	HR
SweRank	Small	8.00	14.84	11.13	6.64	0	31.11	4.17	9.44	5.24	3.45	0	24.07	1.02	2.85	1.28	0.74	0	11.73
	Large	4.44	9.36	10.26	4.66	0	23.56	1.85	4.49	3.61	1.91	0	14.81	1.53	3.19	1.48	0.82	0	11.22
Agentless	GPT-4o	13.33	18.83	15.66	10.07	2.22	32.44	8.33	12.86	6.93	4.23	0	24.54	5.10	8.02	2.04	1.49	0	14.29
	Claude-3.5	12.00	18.62	18.99	11.46	2.67	34.22	6.02	11.05	9.05	5.71	0.46	24.54	4.08	6.90	4.87	3.29	0	15.31
LocAgent	GPT-4o	17.33	25.90	8.44	6.21	0.89	42.22	7.87	13.95	4.03	2.81	0	29.63	5.10	7.93	1.60	1.08	0	17.35
	Claude-3.5	22.22	36.78	11.02	8.73	0.44	54.67	15.74	26.85	6.63	4.97	0	47.69	11.73	20.79	4.32	3.22	0	40.82
CoSIL	GPT-4o	10.22	17.12	15.62	9.39	2.22	32.89	6.94	11.69	9.47	5.06	0	24.54	3.06	4.98	3.42	1.87	0	10.71
	Claude-3.5	14.22	20.42	16.16	10.39	1.33	34.67	10.19	14.37	10.17	6.50	0.93	27.78	6.12	9.50	5.87	3.57	0	19.39
DataLoc	Claude-3.5	64.44	73.97	68.96	58.87	42.22	**89.78**	59.72	70.71	65.37	54.49	35.65	**88.89**	57.14	68.74	62.11	51.19	32.65	**88.78**
	Qwen3-Max	**68.44**	**77.74**	**73.35**	**65.28**	**48.44**	87.11	**62.96**	**75.19**	**70.40**	**61.44**	**42.59**	86.11	**58.67**	**72.55**	**66.98**	**58.03**	**38.27**	84.69

The granularity trend further exposes the limitations of existing localization methods under keyword-agnostic settings. Even under Hit Rate (HR), the most lenient metric that only requires at least one correct location, baselines degrade sharply from file-level to module-level and function-level localization. At the function level, even the best baseline results reach only 5.87% precision and 3.57% AJS. None of the baselines achieves a nonzero PLR. However, the strongest baseline recall is 20.79%, indicating that partial retrieval remains possible even when exact localization is unsuccessful. In contrast, DataLoc achieves HR values ranging from 84.69% to 89.78% across both models and all three granularities. These results indicate that DataLoc maintains strong localization performance at finer granularities.

A key difference lies in how methods handle the size of the returned set. Baselines typically rely on top-n recommendations to increase the probability of covering relevant locations. However, since the correct number of targets varies across queries and is not known in advance, this approach is inherently a compromise rather than an optimal solution. AJS and PLR assess the agreement between the returned sets and the ground truth. For instance, LocAgent with Claude-3.5 reaches a file-level HR of 54.67%, but its AJS is only 8.73%, suggesting that true positives are diluted within an inflated candidate set containing substantial noise. By contrast, DataLoc achieves substantially higher AJS across granularities, reflecting greater precision in returning constraint-satisfying locations. This precision is important for industrial deployment, as it reduces the validation burden for developers or downstream agents.

Abstention when no valid locations exist is further evaluated using KA-LogicQuery-Neg. The investigation of SWE-bench Lite shows that each issue instance involves only 1.15 ground-truth code changes on average. This sparsity raises a key question of whether existing tools truly pinpoint root causes or simply guess likely locations in a narrow search space. In KA-LogicQuery-Neg, query constraints are deliberately modified to create a mismatch between the query and the codebase, making the original ground-truth locations invalid. In this setting, the only correct behavior is to return *no match found*. Unfortunately, all baselines suffer from a tendency to guess and persistently return top-n recommendations even when constraints are not satisfied, yielding false positives. This may suggest

that the strong performance of baselines is partly due to their recommendation-centric design rather than genuine localization reasoning. A lack of refusal ability can mislead downstream agents, waste computation, and introduce regression risk. In contrast, DataLoc returns a clear *no match found* response for over 70% of KA-LogicQuery-Neg queries, demonstrating the necessary discernment to abstain when ground-truth sets are empty.

Answer to RQ1: DataLoc outperforms the evaluated baselines on KA-LogicQuery, attaining function-level precision of 66.98% and a success rate of 58.67%. It also returns *no match found* for over 70% of KA-LogicQuery-Neg queries.

5.4.3.2 Effectiveness on Issue-Based Localization

DataLoc is evaluated on issue-based localization using SWE-bench Lite, where queries are real GitHub issues rather than synthetic logical constraints. As shown in Table 5.5, DataLoc remains competitive on this standard benchmark. With Claude-3.5, DataLoc achieves 72.26%, 80.66%, and 81.02% Acc@1/3/5 at the file level, 75.55% Acc@5/10 at the module level, and 68.98% Acc@5/10 at the function level.

Acc@k should be interpreted together with the size of the returned candidate set. Most baselines are explicitly optimized for fixed top-n recommendation: CoSIL and LocAgent typically return top-5 function-level candidates, Agentless returns 5 candidates regardless of issue complexity, and SweRank adopts an aggressive strategy that utilizes top-100 rankings. By comparison, DataLoc does not prescribe n in advance and only returns an average of 2 candidate locations per issue. Although Acc@k metrics are comparable, DataLoc's compact candidate set provides a more precise and verifiable recommendation, minimizing the overhead of downstream validation and inspection. This precision-oriented design also yields gains in resource efficiency (Details in Section 5.4.3.3).

Answer to RQ2: DataLoc remains competitive on SWE-bench Lite while returning an average of two candidate locations per issue, rather than requiring a fixed top-n prediction set.

TABLE 5.5: Comparison of different methods and models across various localization granularities.

Method	Model	File (%)			Module (%)		Function (%)	
		Acc@1	Acc@3	Acc@5	Acc@5	Acc@10	Acc@5	Acc@10
SweRank	Small	78.10	92.34	94.53	89.05	92.70	79.56	86.13
	Large	83.21	94.89	95.99	90.88	93.43	81.39	88.69
Agentless	gpt-4o	67.50	74.45	74.45	67.15	67.15	55.47	55.47
	claude-3.5	72.63	79.20	79.56	68.98	68.98	58.76	58.76
LocAgent	Qwen2.5-32B(ft)	75.91	90.51	92.70	85.77	87.23	71.90	77.01
	claude-3.5	77.74	91.97	94.16	86.50	87.59	73.36	77.37
DataLoc	gpt-5.1	71.53	77.38	78.47	70.80	72.26	63.14	64.96
	claude-3.5	72.26	80.66	81.02	75.55	75.55	68.98	68.98

5.4.3.3 Efficiency

Figure 5.6 compares average execution time and token consumption on KA-LOGICQUERY, where the y-axis denotes execution time and bubble size represents token usage in thousands. DATALOC with Claude-3.5 achieves a favorable trade-off between efficiency and effectiveness, completing each task in 39.3s on average with 16.2k tokens. Although slightly slower than SweRank (Small) at 33.5s, DATALOC achieves significantly better performance on KA-LCL with low token consumption.

Temporal efficiency. Among the evaluated pipeline-based and agentic localization methods, DATALOC with Claude-3.5 is the fastest, requiring about half the runtime of the fastest baseline, LocAgent with GPT-4o (71.9s). Longer runtimes do not yield better KA-LCL performance, suggesting that, without keyword shortcuts, baselines fail to conduct meaningful repository-level inference despite producing intermediate reasoning traces. DATALOC with Qwen3-Max is slower mainly because its frequent Datalog syntax violations trigger more correction, diagnostics, and mutation-based feedback. Although these steps increase runtime overhead, the improved validity and localization precision demonstrate the effectiveness of the mechanism.

Token economy. Token consumption varies substantially across methods and foundation models. As shown in Figure 5.6, LocAgent consumes 343.9k tokens with GPT-4o and 393.0k with Claude-3.5, over an order of magnitude more than the

16.2k tokens used by DataLoc with Claude-3.5. Agentless, however, uses 13.3k and 15.2k tokens with these models, respectively, slightly fewer than DataLoc with Claude-3.5. With Qwen3-Max, DataLoc consumes 36.4k tokens per task, also exceeding the token consumption of the two Agentless configurations. Thus, the observed token savings are specific to the model and baseline being compared. Multi-round exploration can be costly; for example, in the evaluation, more than ten queries caused LocAgent to enter repeated loops without producing results.

The efficiency of DataLoc stems from its hybrid architecture. In the design, the LLM-based agent is strategically confined to high-level tasks: query analysis, Datalog program synthesis, and final candidate verification. By delegating deep inference to a specialized engine, DataLoc greatly reduces the overhead of redundant multi-round exploration. In addition, errors carry a small cost, requiring only the regeneration of a Datalog program. Beyond this low overhead, the parser-gated validation and intermediate-rule feedback actively assist the LLM in synthesizing high-quality programs.

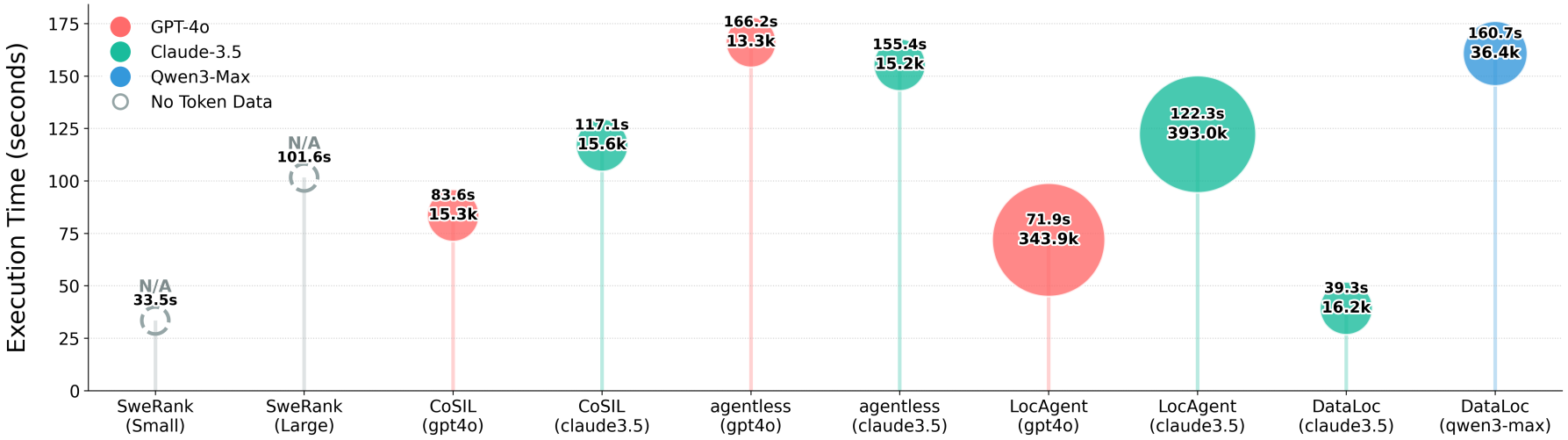


FIGURE 5.6: Average execution time and token consumption of KA-LOGICQUERY.

Answer to RQ3: On KA-LogicQuery, DataLoc with Claude-3.5 requires 39.3 seconds and 16.2k tokens per query on average. Delegating structural reasoning to Datalog reduces repeated LLM exploration, although diagnostic execution can add overhead for models that generate more invalid programs.

5.4.3.4 Ablation Study

An ablation study is designed to quantify how two mechanisms detailed in Section 5.3.3 and Section 5.3.4 improve the executability and usefulness of LLM-generated Datalog programs.

Configurations The ablation study evaluates three configurations that progressively enable these mechanisms: **Base** directly executes the LLM-generated Datalog program without any validation or additional feedback, except the output or error messages from Soufflé itself. **VAL** enables parser-gated validation with deterministic syntactic repairs and high-confidence semantic checks. **Full** further enables intermediate-rule diagnostic probes and feedback. All configurations share the same LLM, initial prompt, iteration budget, and underlying fact bases.

Metrics The ablation study compares (i) execution success rate and non-empty result rate, (ii) mean LLM iterations to first successful execution and (iii) final answer correctness. This evaluation was conducted on a representative subset of KA-LogicQuery, consisting of the first 25 queries.

Table 5.6 summarizes the ablation results under two different LLMs, comparing the baseline system with progressively enabled mechanisms. The results show that validation and repair (VAL) improves the quality of LLM-generated Datalog for both models, with a stronger effect on Qwen3-Max than on Claude-3.5. This difference is expected given the models' baseline capabilities: without VAL, Qwen3-Max exhibits a lower execution success rate due to its weaker ability to consistently produce syntactically correct Soufflé Datalog, whereas Claude already achieves a relatively high baseline level of syntactic validity.

For Qwen3-Max, enabling VAL leads to a substantial improvement across nearly all metrics. The non-empty result rate rises from 49% to 75%, indicating that a

TABLE 5.6: Ablation Results under Three Configurations.

Metric		Qwen3-Max			Claude-3.5		
		Base	**VAL**	**Full**	**Base**	**VAL**	**Full**
Rates ↑	ExecSucc	73.59	84.12	84.12	89.42	94.67	94.67
	$\neq \varnothing$	49.41	75.68	78.52	67.95	75.86	80.69
Cost (Iter, Time) ↓	First	1.48	1.28	1.24	1.16	1.04	1.08
	Iter	12.92	10.16	10.92	4.20	4.28	4.08
	Time	427	104	136	38	40	37
Correctness (function) ↑	SR	20.00	40.00	56.00	44.00	44.00	61.90
	PRE	21.67	46.35	65.71	45.35	46.31	62.70
	REC	18.89	42.70	60.56	49.05	47.46	63.63
	AJS	17.54	37.70	55.63	37.73	38.53	57.09
	PLR	16.00	32.00	40.00	24.00	32.00	42.86
	HR	20.00	44.00	60.00	52.00	52.00	80.95

Configurations. **Base**: no mechanism; **VAL**: validation & repair; **Full**: VAL plus intermediate-rule feedback (INT).

Metrics. **ExecSucc**: successful execution rate. $\neq \varnothing$: non-empty answer rate. **First**: mean iterations to first successful execution. **Iter**: mean total iterations per query. **Time**: mean wall-clock seconds per query.

large fraction of previously failing or unproductive queries were recoverable once parser-gated validation and conservative repairs were applied. At the same time, the average end-to-end execution time drops sharply from 427s to 104s, reflecting a reduction in wasted iterations caused by unrecoverable parser errors and repeated failed executions. Correctness also improves: function-level metrics show substantial gains, with precision increasing from 21.67% to 46.35%, demonstrating that VAL does not merely enable execution but also materially improves the semantic adequacy of the resulting queries. In contrast, the effect of VAL on Claude is more moderate: execution success rate increases by +5 percentage points, and the non-empty result rate increases by +8 percentage points.

Enabling intermediate-rule feedback (INT) in the Full configuration produces a different pattern. The primary role of INT is to guide the LLM toward identifying zero-row intermediate rules and the local constraints that may be over-restrictive. As a result, INT may slightly increase iteration count or execution time in some cases due to additional diagnostic executions, but this overhead consistently leads to higher non-empty rates and improved final answer correctness.

Answer to RQ4: Validation and repair (VAL) improves execution success and reduces wasted execution for Qwen3-Max. Intermediate-rule diagnostics in the Full configuration further increase non-empty result rates and improve final localization, while adding diagnostic cost in some cases.

5.5 Threats to Validity

Internal Validity. The primary internal threats concern baseline implementation and data leakage. The evaluation uses the official implementations of all baselines with default configurations. For baselines that rely on large language models, the evaluation employs the same base models (GPT-4o and Claude-3.5-Sonnet) to avoid model-related bias. The KA-LCL queries in KA-LogicQuery and KA-LogicQuery-Neg are constructed based on the decomposed code structure, resulting in novel query instances. Candidate locations were generated by multiple advanced AIDE agents and independently validated by two authors. Disagreements were resolved through discussion grounded in the formal Python syntax specification. These measures mitigate the risk of data leakage.

External Validity. A main threat to external validity is that the current evaluation focused only on Python. Although the proposed framework is language-agnostic in principle, extending it to additional languages or incorporating more analysis results into program facts remains future work, and addressing this challenge needs further engineering efforts to broaden the applicability.

5.6 Related Work

LLM for Issue Resolution and Question Answering. Large language models are increasingly integrated into software engineering, motivating a wide range of benchmarks for codebase question answering and issue resolution. Existing benchmarks fall into two main categories. Code QA benchmarks [163–165] evaluate models on either code snippets (e.g. CodeQueries [166], CodeQA [167], CoSQA [168]) or repository-level contexts derived from GitHub issues (e.g. CodeRepoQA [169],

CoReQA [170], SWE-QA [171]). End-to-end issue resolution benchmarks, such as SWE-Bench [13] and its extensions, assess full issue-solving capabilities [13, 155, 172–182], primarily focusing on bug-fix tasks and a limited set of programming languages. However, excessive keywords in these benchmarks provide models with too many shortcuts for code localization by superficial lexical matching. To mitigate this bias, this study proposes KA-LogicQuery, which removes semantic keywords and only keeps logic structures, and KA-LogicQuery-Neg, an empty-ground-truth variant for evaluating abstention ability.

Code Localization. Existing LLM-based retrieval techniques have been proposed to improve code localization performance by leveraging semantic understanding [155, 160–162, 183–189]. Recent research has proposed numerous code localization approaches that can be broadly categorized into three classes: (1) *Embedding-based approaches* (e.g., SWERankEmbed [160], CodeSage [190]) encode code entities and natural language descriptions as embeddings, ranking them based on semantic similarity. While they achieve high recall by retrieving a broad range of relevant candidate locations, they can only identify code snippets that "look similar" without understanding the logical relationships between them. Furthermore, they suffer from hallucination and noise interference, such as methods with identical names but entirely different functionalities. (2) *Pipeline-based LLM approaches* (e.g., Agentless [161]) follow a structured, multi-stage workflow from files to functions. However, such hierarchical localization design overly depends on initial file-level localization and fails to capture cross-level dependencies, implicitly assuming that developers adhere to good naming conventions. (3) *Agentic approaches* (e.g., LocAgent [155], CoSIL [162], Orca Loca [191], GraphLocator [192]) offer greater flexibility by allowing LLMs to autonomously traverse the repository graph [193]. However, they only consider surface-level relevance and exhibit rapid performance degradation without explicit contextual guidance.

5.7 Summary

This chapter identified the *Keyword Shortcut* in issue-based code localization benchmarks, where lexical anchors can enable matching without repository-level structural reasoning. It formalized Keyword-Agnostic Logical Code Localization

(KA-LCL) and introduced KA-LogicQuery, comprising 225 logic-intensive queries across 9 projects, together with KA-LogicQuery-Neg for evaluating abstention under empty ground truth.

The proposed DataLoc framework represents repositories as program facts, translates natural-language constraints into Datalog programs, and improves query reliability through parser-gated validation and conservative intermediate-rule diagnostics. Diagnostic mutations provide evidence for refinement; their outputs are never returned as final answers.

On KA-LogicQuery, DataLoc achieves function-level precision of 66.98% and a success rate of 58.67%, compared with the best baseline values of 5.87% and 11.73%, respectively. It returns *no match found* for over 70% of KA-LogicQuery-Neg queries and remains competitive on SWE-bench Lite while returning an average of two candidate locations per issue. These results suggest that combining LLM intent interpretation with formal program reasoning offers a practical route toward more precise, verifiable, and cost-aware code localization.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This thesis addresses the fundamental tension in modern software engineering: the need for rapid, AI-driven software evolution versus the requirement for deterministic reliability. While Large Language Models (LLMs) have ushered in a paradigm shift toward autonomous intelligence, their probabilistic nature often fails to meet the rigorous demands of interconnected dependency networks. To bridge this gap, this research introduces a neurosymbolic approach that anchors generative intelligence within formal logical frameworks, systematically addressing the three core pillars of evolutionary reliability.

- In **Chapter 3**, this thesis introduces CompSuite, a dataset that includes 123 real-world Java client-library pairs where upgrading the library causes an incompatibility issue in the corresponding client. Each incompatibility issue in CompSuite is associated with a test case authored by the developers, which can be used to reproduce the issue. The dataset also provides a command-line interface that simplifies the execution and validation of each issue. By providing a structured dataset for library upgrade incompatibilities, CompSuite allows AI agents to move beyond anecdotal fixes toward a systematic understanding of how third-party dependency shifts propagate through client systems.

- In **Chapter 4**, this thesis presents MPChecker for detecting inconsistencies between code and documentation, specifically focusing on multi-parameter constraints. MPChecker identifies these constraints at the code level by exploring execution paths through symbolic execution and further extracts corresponding constraints from documentation using large language models (LLMs). The approach uses a customized fuzzy constraint logic to reconcile the unpredictability of LLM outputs and detect logical inconsistencies between the code and documentation constraints. This not only ensures internal semantic alignment but also provides a reliable interface contract for external dependencies, mitigating the risks of external usage.

- In **Chapter 5**, this thesis identifies the *Keyword Shortcut* in issue-based code localization and formalizes Keyword-Agnostic Logical Code Localization (KA-LCL). It introduces KA-LogicQuery, with 225 logic-intensive queries across 9 projects, and KA-LogicQuery-Neg for evaluating abstention under empty ground truth. The proposed DataLoc framework combines LLM-based intent interpretation with deterministic Datalog reasoning over extracted program facts. Parser-gated validation and conservative mutation-based diagnostics help refine generated programs, with zero-cardinality intermediate relations classified as fragile-empty, stable-empty, or inconclusive. Diagnostic probe outputs are never returned as final answers. The evaluation reports function-level precision of 66.98% and a success rate of 58.67%, compared with the best baseline values of 5.87% and 11.73%, respectively. DataLoc returns *no match found* for over 70% of KA-LogicQuery-Neg queries and remains competitive on SWE-bench Lite while returning an average of two candidate locations per issue.

In conclusion, this dissertation demonstrates that reliable software evolution in the age of AI depends not on choosing between human-centered logic and AI-driven automation, but on effectively integrating the two. Through the systematic investigation of dependency management, code-documentation consistency, and precise code localization, this research shows how neurosymbolic approaches can combine the adaptability of neural methods with the explicit reasoning and reliability of symbolic techniques. Taken together, these contributions provide both a theoretical and practical foundation for developing autonomous software maintenance systems that are intelligent, flexible, and dependable.

6.2 Implications of Advances in Large Language Models

Advances in large language models warrant reassessing design choices motivated by the limitations of earlier models. For example, Claude Fable 5.1 and GPT-6 Astra support context windows of one million and 1.05 million tokens, respectively [194, 195]. Larger context windows may reduce the need to split documentation or source code into smaller inputs. Improvements in task-specific reliability may also reduce the need for corrective prompting and repair. Some supporting mechanisms adopted for earlier models may therefore become unnecessary.

The appropriate simplifications depend on both the task and the codebase. Large codebases can substantially exceed even million-token context windows. Google's monolithic repository, for example, contains billions of lines of code [196, 197]. Selecting task-relevant information remains important at this scale. Even when an input fits within the window, its effective use requires separate evaluation. NoLiMa minimizes lexical overlap between questions and relevant passages to test retrieval through semantic associations. Retrieval performance declined with increasing context length in its evaluated models [198], motivating task-specific evaluation of long-context models.

The following discussion considers which parts of each system could be simplified and which retain value through executable checks or reusable program representations. It also considers whether these components can reduce total computation at comparable task accuracy. The proposed changes and their benefits are prospective, distinct from the implementations and measurements reported in the preceding chapters.

6.2.1 CompSuite: Reproducible Evaluation

CompSuite provides reproducible library-upgrade incompatibility cases and an execution tool. As a dataset and evaluation infrastructure, it does not rely on an LLM inference pipeline whose components would need to be simplified as models improve.

Stronger models may reduce the guidance and repair iterations needed to resolve the incompatibilities recorded in CompSuite. Its fixed client-library versions, reproducible environments, and developer-written tests would still provide a common basis for evaluating these workflows. If future systems solve every recorded case, the benchmark would become less informative for distinguishing repair effectiveness, but could still support comparisons of model calls, token consumption, and elapsed time required to achieve the same test outcomes. These outcomes establish success on the available tests rather than correctness across all possible executions. More broadly, even when both library and client code are generated entirely by AI, their independent evolution can still introduce incompatible interfaces or behaviors. The recorded cases therefore remain useful as concrete failure patterns for compatibility analysis and regression testing.

6.2.2 MPChecker: Simplifying Constraint Extraction

The documentation processing in MPChecker offers concrete opportunities for simplification. The 1,500-word chunks in Section 4.3.2.2 were selected for the evaluated GPT-4 configuration. Larger context windows could allow complete API descriptions to be processed together, reducing fragmentation and repeated prompt instructions. More reliable constraint extraction could also support fewer examples and shorter prompts. These changes may reduce model calls and repeated input, although processing unrelated documentation can offset those savings. Input selection should therefore depend on the information needed for extraction rather than the maximum available context window.

Better extraction also warrants reassessing fuzzy processing. Similarity-based tolerance of incorrect parameter names or operators could be reduced or removed if a newer model produces sufficiently accurate constraints. Predicates for documentation terms such as “ignore” and “exist” serve a different purpose by representing relationships between parameter use and execution paths. Those semantics still need to be represented, but their current fuzzy implementation is not necessarily the only suitable approach. The ablation in Section 4.4 removes both fuzzy words and fuzzy constraint logic, so it does not establish which parts could be removed as extraction improves.

Explicit constraints, symbolic execution, and SMT checking retain value even as extraction becomes more accurate. Within the scope of the represented constraints and analyzed paths, these mechanisms provide inspectable evidence of whether program behavior satisfies or violates documented requirements. Such evidence supports review by both developers and AI agents in human–AI collaboration and fully automated software engineering workflows. During maintenance of large systems, these checks can help identify overlooked violations of textual constraints and direct further investigation to the relevant code. By executing explicit checks rather than repeatedly asking an LLM to infer whether code satisfies each requirement, automated software engineering systems may reduce their reliance on model reasoning and the associated inference costs.

6.2.3 DataLoc: Fewer Repairs with Reusable Program Facts

For DataLoc, more reliable Datalog generation could reduce synthesis and repair iterations. Larger context windows and fewer hallucinations could help the model generate rules that better capture the intended query. As generation becomes more reliable, a smaller probe budget and fewer specialized repair heuristics may suffice, reducing diagnostic executions and subsequent model calls.

Program facts and the Datalog engine remain valuable as a reusable basis for structural code queries. For queries supported by the extracted facts, the engine computes the results without requiring the LLM to repeatedly infer code relationships from source code. The facts can be reused across queries over the same repository version, while parser validation checks the syntax of generated rules. Together, the rules, facts, and execution results allow developers and AI agents to inspect how candidate locations were selected. During maintenance of large systems, this division of work can reduce repeated model inference for structural search, allowing model reasoning to focus on interpreting user intent and assessing relevant candidates.

Source-level candidate verification checks whether the retrieved code matches the intended query, complementing the results produced by Datalog execution. Stronger models may improve verification quality, reducing both the rejection of relevant

candidates and the acceptance of irrelevant ones. Retaining this step allows improvements in model reasoning to strengthen the assessment of candidates produced by structural queries.

6.3 Future Work

Building on the contributions and the discussion of evolving model capabilities in Section 6.2, this section outlines further directions for reliable AI-assisted software evolution.

6.3.1 Benchmarking AI-Assisted Self-Healing Under Dependency Evolution

Current software engineering benchmarks for AI, such as SWE-bench, primarily focus on issue resolution and bug fixing within self-contained repositories. However, software projects usually exist as nodes in a vast dependency graph, simultaneously depending on upstream libraries and being depended on by downstream consumers. Compatibility issues caused by library upgrades represent one of the defining challenges of modern software evolution. Such problems are particularly acute in ecosystems characterized by dense dependency graphs and rapid release cycles, as exemplified by data science and deep learning libraries. Therefore, there is a pressing need for a comprehensive study and benchmark to evaluate the end-to-end capability of AI systems to autonomously detect, diagnose, and resolve such cross-project incompatibilities.

It is worth noting that testing AI-related libraries introduces unique constraints compared to traditional software. These projects rely heavily on hardware like GPUs, and many failures can only be triggered in specific compute environments. To navigate the high costs of verification and the inherent stochasticity of model training, many projects have adopted runnable example scripts as a form of end-to-end testing as a compromise. This points to the need for test oracles specifically adapted to AI training tasks. Furthermore, a nuance arises in the context of autonomous remediation: a patch may not manifest as a direct source code modification. In many cases, it could involve strategic dependency management, such

as upgrading or pinning specific auxiliary libraries. This places high demands on the ability of AI systems to understand code comprehensively and reason about complex dependencies.

The envisioned benchmark aims to serve not only as a rigorous evaluation platform but also as a catalyst for developing AI systems capable of self-healing within an ever-evolving software supply chain.

6.3.2 Automated Code-Documentation Consistency Repair

MPChecker demonstrates the feasibility of automatically detecting multi-parameter inconsistencies between code and documentation. However, detection alone is insufficient. A critical follow-up question is how to automatically repair such inconsistencies once they are identified. This involves two complementary directions.

First, on the code-to-documentation side, when the code implementation is considered the source of truth, the AI-driven system should be able to generate documentation patches that accurately reflect the actual code behavior and complicated constraints enforced by the code. This requires going beyond simple textual replacement to produce natural language descriptions that are both semantically precise and stylistically consistent with the existing documentation. Second, on the documentation-to-code side, when the documentation captures the intended specification, the system should generate code patches that align the implementation with the documented constraints. In either direction, formal verification of the generated patches is essential. By extending the fuzzy constraint satisfaction framework introduced in MPChecker, one could design a closed-loop pipeline where a neurosymbolic system detects the inconsistency, generates a candidate repair, and then re-validates the repaired artifact against the constraint specification, ensuring the fix does not introduce new inconsistencies or break existing ones.

6.3.3 Technical Debt and Dependency Fundamentalism

AI-powered coding tools have substantially accelerated software development, but they also introduce systematic risks of technical debt accumulation. There are two

main problems. First, the generative nature of AI can lead it to disregard existing code implementation in its pursuit of immediate task completion, frequently modifying codebases arbitrarily, and even replacing dependencies during certain changes. This results in tightly coupled codebases with low modularity, making them difficult to maintain and extend. Second, due to temporal lag, the underlying models often generate code that relies on outdated libraries and deprecated APIs. Even when aware of newer API versions, these probabilistic models may still suggest deprecated APIs simply because they are prevalent in the training data. Additionally, hallucinations cause the model to conflate interface names across libraries, invoking syntactically plausible but semantically incorrect calls. Together, these problems elevate vulnerability exposure, erode evolutionary stability, and impose compounding maintenance overhead.

To address these challenges, this thesis proposes a new concept, *dependency fundamentalism*, a principle for software evolution that emphasizes the critical importance of anchoring projects to specific dependency versions and their prescribed library interfaces during evolution. Concretely, this thesis advocates enhancing AI's version- and interface-awareness through integrating formal methods for dependency management and interface verification. This approach is expected to meaningfully limit the introduction of deprecated or misused API calls, effectively managing technical debt and fostering long-term sustainability.

6.3.4 Human-AI Interaction: from Intention to Code

Converting real human intention into expected executable code has long been a deeply challenging problem. The inherent ambiguity of natural language and the complexity of programming languages make full alignment between the two difficult to achieve. While approaches such as specification-driven development (SDD) have been proposed to assist users in clarifying requirements and breaking down tasks, they remain largely prompt-level optimizations and fall short when applied to repository-scale software development.

To truly bridge the gap between human intention and code, the entire human-AI interaction (HAI) paradigm needs to be reconsidered. A *Verifiable Intermediate Language (VIL)* should be introduced as a critical software requirement specification layer between user intentions and the final code output. Ideally, VIL is a

structured, formalized, and human-readable representation of requirements that enables precise property verification and logical auditing before code generation. With the help of VIL, it is possible to ensure that the generated code adheres to specified safety, correctness, and performance constraints while maintaining full traceability from requirements to implementation.

In summary, looking beyond the immediate extensions of this thesis, the broader trajectory of neurosymbolic software engineering points toward a future where autonomous systems can reliably manage the full lifecycle of software evolution, from requirement changes through code adaptation to deployment verification, with formal guarantees on correctness.

Achieving this vision requires progress on several fronts. The formal methods community must develop specification languages that are expressive enough to capture real-world software properties yet accessible enough for LLMs to synthesize and manipulate. The machine learning community must build models that can reason about program structure, not merely predict token sequences, perhaps through architectures that organically incorporate or natively contain symbolic reasoning. And the software engineering community must establish benchmarks and evaluation methodologies that rigorously test for genuine reasoning capabilities rather than superficial pattern matching.

The neurosymbolic paradigm articulated in this dissertation is expected to play an important role in realizing the next generation of software engineering tools that combine the creative flexibility of AI with the uncompromising rigor of formal logic to build software that evolves reliably in an increasingly complex world.

Bibliography

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. xxi, 11, 12

[2] Building effective agents. `https://www.anthropic.com/engineering/building-effective-agents`, 2024. xxi, 14

[3] Michael K Chen. A comparative study of neurosymbolic ai approaches to interpretable logical reasoning. *arXiv preprint arXiv:2508.03366*, 2025. xxi, 19, 20

[4] Chenguang Zhu, Mengshi Zhang, Xiuheng Wu, Xiufeng Xu, and Yi Li. Client-specific upgrade compatibility checking via knowledge-guided discovery. *ACM Trans. Softw. Eng. Methodol.*, 32(4):1–31, may 2023. ISSN 1049-331X. doi: 10.1145/3582569. URL `https://doi.org/10.1145/3582569`. xxi, 25, 35

[5] Yi Li. Managing software evolution through semantic history slicing. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1014–1017. IEEE, 2017. 1

[6] Manny M Lehman. Laws of software evolution revisited. In *European workshop on software process technology*, pages 108–124. Springer, 1996. 1

[7] Jacob Mahon, Chenxi Hou, and Zhihao Yao. PyPitfall: Dependency chaos and software supply chain vulnerabilities in Python. *arXiv preprint arXiv:2507.18075*, 2025. 2

[8] National Institute of Standards and Technology. CVE-2021-44228 detail. `https://nvd.nist.gov/vuln/detail/CVE-2021-44228`, 2021. 2

[9] Cursor: The AI Code Editor. `https://cursor.com/`, 2025. Accessed: 2025-09-18. 4

[10] Anthropics. Claude code. `https://github.com/anthropics/claude-code`, 2025. Accessed: 2025-11-25. 4

[11] Google. Gemini CLI. `https://github.com/google-gemini/gemini-cli`, 2025. Accessed: 2025-11-25. 4

[12] Stack Overflow. AI → developer tools, 2025 developer survey. https://survey.stackoverflow.co/2025/ai#developer-tools, 2025. Accessed: 2025-09-18. 4

[13] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66. 4, 22, 110

[14] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. Hallucination is inevitable: An innate limitation of large language models. *arXiv preprint arXiv:2401.11817*, 2024. 5

[15] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. 13

[16] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020. 13

[17] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, et al. The prompt report: A systematic survey of prompt engineering techniques. *arXiv preprint arXiv:2406.06608*, 2024. 13

[18] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*, 1, 2024. 13

[19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020. 15

[20] OpenAI function calling guide. https://developers.openai.com/api/docs/guides/function-calling/, 2023. 15

[21] Model Context Protocol (MCP). https://www.anthropic.com/news/model-context-protocol, 2024. 15

[22] LLM powered autonomous agents. https://lilianweng.github.io/posts/2023-06-23-agent/, 2023. 15

[23] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022. 15

[24] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023. 15

[25] Roberto Baldoni, Emilio Coppa, Daniele Cono D'elia, Camil Demetrescu, and Irene Finocchi. A survey of symbolic execution techniques. *ACM Computing Surveys (CSUR)*, 51(3):1–39, 2018. 15, 36

[26] Patrice Godefroid, Nils Klarlund, and Koushik Sen. DART: Directed automated random testing. In *Proceedings of the 2005 ACM SIGPLAN conference on Programming language design and implementation*, pages 213–223, 2005. 16

[27] Koushik Sen, Darko Marinov, and Gul Agha. CUTE: A concolic unit testing engine for c. *ACM SIGSOFT software engineering notes*, 30(5):263–272, 2005. 16

[28] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In *OSDI*, volume 8, pages 209–224, 2008. 16

[29] Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer, 2008. 16

[30] Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, et al. cvc5: A versatile and industrial-strength SMT solver. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 415–442. Springer, 2022. 16

[31] Daniel Kahneman. *Thinking, fast and slow*. macmillan, 2011. 19

[32] Gianluca Mezzetti, Anders Møller, and Martin Toldam Torp. Type regression testing to detect breaking changes in node. js libraries. In *32nd european conference on object-oriented programming (ECOOP 2018)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2018. 21

[33] Anders Møller and Martin Toldam Torp. Model-based testing of breaking changes in node. js libraries. In *Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, pages 409–419, 2019. 21

[34] Darius Foo, Hendy Chua, Jason Yeo, Ming Yi Ang, and Asankhaya Sharma. Efficient static checking of library updates. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 791–796, 2018. 21

[35] Ying Wang, Rongxin Wu, Chao Wang, Ming Wen, Yepang Liu, Shing-Chi Cheung, Hai Yu, Chang Xu, and Zhiliang Zhu. Will dependency conflicts affect my program's semantics? *IEEE Transactions on Software Engineering*, 48(7):2295–2316, 2021. 21, 35

[36] Federico Mora, Yi Li, Julia Rubin, and Marsha Chechik. Client-specific equivalence checking. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, pages 441–451, 2018. 21, 35

[37] Chenguang Zhu, Owolabi Legunsen, August Shi, and Milos Gligoric. A framework for checking regression test selection tools. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pages 430–441. IEEE, 2019. 21, 35

[38] René Just, Darioush Jalali, and Michael D Ernst. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*, pages 437–440, 2014. 22, 36

[39] Danning Xie, Mingwei Zheng, Xuwei Liu, Jiannan Wang, Chengpeng Wang, Lin Tan, and Xiangyu Zhang. Core: Benchmarking LLMs code reasoning capabilities through static analysis tasks. *arXiv preprint arXiv:2507.05269*, 2025. 22

[40] Julia Rubin and Martin Rinard. The challenges of staying together while moving fast: An exploratory study. In *Proceedings of the 38th International Conference on Software Engineering*, pages 982–993, 2016. 24

[41] Alex Gyori, Owolabi Legunsen, Farah Hariri, and Darko Marinov. Evaluating regression test selection opportunities in a very large open-source ecosystem. In *2018 IEEE 29th International Symposium on Software Reliability Engineering (ISSRE)*, pages 112–122. IEEE, 2018. 24, 35

[42] Raula Gaikovina Kula, Daniel M German, Ali Ouni, Takashi Ishio, and Katsuro Inoue. Do developers update their library dependencies? an empirical study on the impact of security advisories on library migration. *Empirical Software Engineering*, 23(1):384–417, 2018. 24

[43] Ying Wang, Bihuan Chen, Kaifeng Huang, Bowen Shi, Congying Xu, Xin Peng, Yijian Wu, and Yang Liu. An empirical study of usages, updates and risks of third-party libraries in java projects. In *2020 IEEE International conference on software maintenance and evolution (ICSME)*, pages 35–45. IEEE, 2020. 25

[44] Wei Wu, Yann-Gaël Guéhéneuc, Giuliano Antoniol, and Miryung Kim. Aura: a hybrid approach to identify framework evolution. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering-Volume 1*, pages 325–334, 2010. 25

[45] Lingchao Chen, Foyzul Hassan, Xiaoyin Wang, and Lingming Zhang. Taming behavioral backward incompatibilities via cross-project testing and analysis. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, pages 112–124, 2020. 25

[46] Dependabot. Dependabot. `https://dependabot.com`, 2020. 25

[47] Jamie Cleare and Claudia Iacob. GemChecker: Reporting on the Status of Gems in Ruby on Rails Projects. In *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 700–704, 2018. 25

[48] Shin Yoo and Mark Harman. Regression Testing Minimization, Selection and Prioritization: A Survey. *Software Testing, Verification and Reliability*, 22 (2):67–120, 2012. 26

[49] Github. Github, 2023. `https://github.com/`. 28

[50] Maven. Maven Central Repositories, 2023. `https://mvnrepository.com/`. 28

[51] Shaikh Mostafa, Rodney Rodriguez, and Xiaoyin Wang. A study on behavioral backward incompatibility bugs in Java software libraries. In *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*, pages 127–129. IEEE, 2017. 28

[52] Wei Wu, Foutse Khomh, Bram Adams, Yann-Gaël Guéhéneuc, and Giuliano Antoniol. An exploratory study of API changes and usages based on apache and eclipse ecosystems. *Empirical Software Engineering*, 21:2366–2412, 2016.

[53] Dong Qiu, Bixin Li, and Hareton Leung. Understanding the API usage in Java. *Information and software technology*, 73:81–100, 2016.

[54] Raula Gaikovina Kula, Coen De Roover, Daniel German, Takashi Ishio, and Katsuro Inoue. Visualizing the evolution of systems and their library dependencies. In *2014 Second IEEE Working Conference on Software Visualization*, pages 127–136. IEEE, 2014. 28

[55] Maven. Apache Maven, 2023. `https://maven.apache.org/`. 28

[56] Maven. Maven Versions Plugin, 2023. `https://maven.apache.org/plugins/index.html`. 31

[57] Github. Github Fork, 2023. `https://docs.github.com/en/get-started/quickstart/fork-a-repo`. 32

[58] Github. Github Tag, 2023. `https://docs.github.com/en/desktop/contributing-and-collaborating-using-github-desktop/managing-commits/managing-tags`. 32

[59] Li Li, Tegawendé F Bissyandé, Haoyu Wang, and Jacques Klein. Cid: Automating the detection of API-related compatibility issues in android apps. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 153–163, 2018. 35

[60] Dongjie He, Lian Li, Lei Wang, Hengjie Zheng, Guangwei Li, and Jingling Xue. Understanding and detecting evolution-induced compatibility issues in android apps. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, pages 167–177, 2018.

[61] Simone Scalabrino, Gabriele Bavota, Mario Linares-Vásquez, Michele Lanza, and Rocco Oliveto. Data-driven solutions to detect API compatibility issues in android: an empirical study. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*, pages 288–298. IEEE, 2019.

[62] Lili Wei, Yepang Liu, and Shing-Chi Cheung. Pivot: learning API-device correlations to facilitate android compatibility issue detection. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pages 878–888. IEEE, 2019.

[63] Huaxun Huang, Lili Wei, Yepang Liu, and Shing-Chi Cheung. Understanding and detecting callback compatibility issues for android applications. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, pages 532–542, 2018. 35

[64] Ying Wang, Ming Wen, Zhenwei Liu, Rongxin Wu, Rui Wang, Bo Yang, Hai Yu, Zhiliang Zhu, and Shing-Chi Cheung. Do the dependency conflicts in my project matter? In *Proceedings of the 2018 26th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, pages 319–330, 2018. 35

[65] Ying Wang, Ming Wen, Rongxin Wu, Zhenwei Liu, Shin Hwei Tan, Zhiliang Zhu, Hai Yu, and Shing-Chi Cheung. Could i have a stack trace to examine the dependency conflict issue? In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pages 572–583. IEEE, 2019. 35

[66] Milos Gligoric, Lamyaa Eloussi, and Darko Marinov. Practical regression test selection with dynamic file dependencies. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, pages 211–222, 2015. 35

[67] Owolabi Legunsen, Farah Hariri, August Shi, Yafeng Lu, Lingming Zhang, and Darko Marinov. An extensive study of static regression test selection

in modern software evolution. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 583–594, 2016.

[68] Lingming Zhang. Hybrid regression test selection. In *Proceedings of the 40th International Conference on Software Engineering*, pages 199–209, 2018. 35

[69] Andreas Zeller. Yesterday, my program worked. today, it does not. why? *ACM SIGSOFT Software engineering notes*, 24(6):253–267, 1999. 36

[70] W Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. A survey on software fault localization. *IEEE Transactions on Software Engineering*, 42(8):707–740, 2016. 36

[71] Quang-Cuong Bui, Riccardo Scandariato, and Nicolás E Díaz Ferreyra. Vul4J: a dataset of reproducible Java vulnerabilities geared towards the study of program repair techniques. In *Proceedings of the 19th International Conference on Mining Software Repositories*, pages 464–468, 2022. 36

[72] Kamil Jezek and Jens Dietrich. API evolution and compatibility: A data corpus and tool evaluation. *J. Object Technol.*, 16(4):2:1–2:23, 2017. 36

[73] Chenguang Zhu, Yi Li, Julia Rubin, and Marsha Chechik. A dataset for dynamic discovery of semantic changes in version controlled software histories. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*, pages 523–526. IEEE, 2017. 36

[74] Lili Wei, Yepang Liu, and Shing-Chi Cheung. Taming android fragmentation: Characterizing and detecting compatibility issues for android apps. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, pages 226–237, 2016.

[75] Sebastian Nielebock, Paul Blockhaus, Jacob Krüger, and Frank Ortmeier. AndroidCompass: A dataset of Android compatibility checks in code repositories. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, pages 535–539. IEEE, 2021. 36

[76] Emad Aghajani, Csaba Nagy, Olga Lucero Vega-Márquez, Mario Linares-Vásquez, Laura Moreno, Gabriele Bavota, and Michele Lanza. Software documentation issues unveiled. In *2019 IEEE/ACM 41st International Conference on Software Engineering*, ICSE, pages 1199–1210. IEEE, 2019. 39

[77] Sawan Rai, Ramesh Chandra Belwal, and Atul Gupta. A review on source code documentation. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 13(5):1–44, 2022. 39

[78] Inderjot Kaur Ratol and Martin P Robillard. Detecting fragile comments. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering*, ASE, pages 112–122. IEEE, 2017. 40

[79] Zhongxin Liu, Xin Xia, Meng Yan, and Shanping Li. Automating just-in-time comment updating. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, ASE, pages 585–597, 2020. 40, 72

[80] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022. 40, 52

[81] statsmodels. Statistical modeling and econometrics in Python, 2024. URL `https://github.com/statsmodels/statsmodels`. 42

[82] scikit learn. Machine learning in Python, 2024. URL `https://github.com/scikit-learn/scikit-learn`. 42

[83] Bart Kosko and Satoru Isaka. Fuzzy logic. *Scientific American*, 269(1):76–81, 1993. 44, 58

[84] Sally C Brailsford, Chris N Potts, and Barbara M Smith. Constraint satisfaction problems: Algorithms and applications. *European journal of operational research*, 119(3):557–581, 1999. 44

[85] Pedro Meseguer, Francesca Rossi, and Thomas Schiex. Soft constraints. In *Foundations of Artificial Intelligence*, volume 2, pages 281–328. Elsevier, 2006. 45

[86] Thomas Schiex. Possibilistic constraint satisfaction problems or "how to handle soft constraints?". In *Uncertainty in Artificial Intelligence*, pages 268–275. Elsevier, 1992. 45

[87] Zsofi Ruttkay. Fuzzy constraint satisfaction. In *Proceedings of 1994 IEEE 3rd International Fuzzy Systems Conference*, pages 1263–1268. IEEE, 1994. 45, 58

[88] Gias Uddin and Martin P Robillard. How API documentation fails. *Ieee software*, 32(4):68–75, 2015. 46, 72

[89] Yu Zhou, Ruihang Gu, Taolue Chen, Zhiqiu Huang, Sebastiano Panichella, and Harald Gall. Analyzing APIs documentation and code to detect directive defects. In *2017 IEEE/ACM 39th International Conference on Software Engineering*, ICSE, pages 27–37. IEEE, 2017.

[90] Chenguang Zhu, Ye Liu, Xiuheng Wu, and Yi Li. Identifying solidity smart contract API documentation errors. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, ASE, pages 1–13, 2022. 46

[91] Sphinx. Example google style Python docstrings, 2024. URL `https://www.sphinx-doc.org/en/master/usage/extensions/example_google.html`. 47

[92] Sphinx. Example numpy style Python docstrings, 2024. URL `https://www.sphinx-doc.org/en/master/usage/extensions/example_numpy.html`. 47

[93] PyExSMT. Python symbolic execution, 2024. URL `https://github.com/FedericoAureliano/PyExSMT`. 49

[94] PyExZ3. Python exploration with z3, 2024. URL `https://github.com/thomasjball/PyExZ3`.

[95] pySMT. A library for SMT formulae manipulation and solving, 2024. URL `https://github.com/pysmt/pysmt`.

[96] Thomas Ball and Jakub Daniel. Deconstructing dynamic symbolic execution. In *Dependable Software Systems Engineering*, pages 26–41. IOS Press, 2015.

[97] Alessandro Disney Bruni, Tim Disney, and Cormac Flanagan. A peer architecture for lightweight symbolic execution. *Universidad de California, Santa Cruz*, 2011. 49

[98] Pooja Rani, Suada Abukar, Nataliia Stulova, Alexandre Bergel, and Oscar Nierstrasz. Do comments follow commenting conventions? a case study in java and python. In *2021 IEEE 21st International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pages 165–169. IEEE, 2021. 51

[99] Google. AI for every developer, 2024. URL `https://ai.google.dev`. 52

[100] Meta. Introducing llama 3, 2024. URL `https://www.llama.com`. 52

[101] Shubhang Shekhar Dvivedi, Vyshnav Vijay, Sai Leela Rahul Pujari, Shoumik Lodh, and Dhruv Kumar. A comparative analysis of large language models for code documentation generation. In *Proceedings of the 1st ACM International Conference on AI-Powered Software*, 2024. 52

[102] OpenAI. GPT-4 model, 2026. URL `https://developers.openai.com/api/docs/models/gpt-4`. 52

[103] Chi Han, Qifan Wang, Hao Peng, Wenhan Xiong, Yu Chen, Heng Ji, and Sinong Wang. LM-infinite: Zero-shot extreme length generalization for large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3991–4008, 2024. 52

[104] Hongye Jin, Xiaotian Han, Jingfeng Yang, Zhimeng Jiang, Zirui Liu, Chia-Yuan Chang, Huiyuan Chen, and Xia Hu. Llm maybe longlm: Self-extend LLM context window without tuning. *arXiv preprint arXiv:2401.01325*, 2024. 52

[105] OpenAI. What are tokens and how to count them?, 2024. URL `https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them`. 52

[106] Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? *arXiv preprint arXiv:2202.12837*, 2022. 53

[107] Ohad Rubin, Jonathan Herzig, and Jonathan Berant. Learning to retrieve prompts for in-context learning. *arXiv preprint arXiv:2112.08633*, 2021. 53

[108] Ken Kennedy. Use-definition chains with applications. *Computer Languages*, 3(3):163–179, 1978. 57

[109] Mary Jean Harrold and Mary Lou Soffa. Efficient computation of interprocedural definition-use chains. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 16(2):175–204, 1994. 57

[110] Lotfi A Zadeh. Fuzzy sets. *Information and Control*, 1965. 60

[111] Scikit-learn issue#30099. `https://github.com/scikit-learn/scikit-learn/issues/30099`, 2024. 69, 70

[112] Scikit-learn issue#28469. `https://github.com/scikit-learn/scikit-learn/issues/28469`, 2024.

[113] Scikit-learn issue#28470. `https://github.com/scikit-learn/scikit-learn/issues/28470`, 2024.

[114] Scikit-learn issue#28473. `https://github.com/scikit-learn/scikit-learn/issues/28473`, 2024.

[115] Scikit-learn issue#29440. `https://github.com/scikit-learn/scikit-learn/issues/29440`, 2024.

[116] Scikit-learn issue#29463. `https://github.com/scikit-learn/scikit-learn/issues/29463`, 2024.

[117] Scikit-learn issue#29464. `https://github.com/scikit-learn/scikit-learn/issues/29464`, 2024.

[118] Statsmodels issue#9304. `https://github.com/statsmodels/statsmodels/issues/9304`, 2024.

[119] Scikit-learn issue#29509. `https://github.com/scikit-learn/scikit-learn/issues/29509`, 2024.

[120] Dask issue#11336. `https://github.com/dask/dask/issues/11336`, 2024.

[121] Keras issue#20141. `https://github.com/keras-team/keras/issues/20141`, 2024. 70

[122] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proceedings of the 47th International Conference on Software Engineering*, ICSE, April 2025. 72

[123] Emad Aghajani, Csaba Nagy, Gabriele Bavota, and Michele Lanza. A large-scale empirical study on linguistic antipatterns affecting apis. In *2018 IEEE International conference on software maintenance and evolution (ICSME)*, pages 25–35. IEEE, 2018. 72

[124] Venera Arnaoudova, Massimiliano Di Penta, and Giuliano Antoniol. Linguistic antipatterns: What they are and how developers perceive them. *Empirical Software Engineering*, 21:104–158, 2016.

[125] Barthélémy Dagenais and Martin P Robillard. Creating and evolving developer documentation: understanding the decisions of open source contributors. In *Proceedings of the eighteenth ACM SIGSOFT International Symposium on Foundations of Software Engineering*, FSE, pages 127–136, 2010.

[126] Emad Aghajani, Csaba Nagy, Mario Linares-Vásquez, Laura Moreno, Gabriele Bavota, Michele Lanza, and David C Shepherd. Software documentation: the practitioners' perspective. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ICSE, pages 590–601, 2020.

[127] Andrew Head, Caitlin Sadowski, Emerson Murphy-Hill, and Andrea Knight. When not to comment: Questions and tradeoffs with API documentation for c++ projects. In *Proceedings of the 40th International Conference on Software Engineering*, ICSE, pages 643–653, 2018.

[128] Yang Liu, Mingwei Liu, Xin Peng, Christoph Treude, Zhenchang Xing, and Xiaoxin Zhang. Generating concept based API element comparison using a knowledge graph. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, ASE, pages 834–845, 2020.

[129] Martin Monperrus, Michael Eichberg, Elif Tekes, and Mira Mezini. What should developers be aware of? an empirical study on the directives of API documentation. *Empirical Software Engineering*, 17:703–737, 2012.

[130] Mohamed Aymen Saied, Houari Sahraoui, and Bruno Dufour. An observational study on API usage constraints and their documentation. In *2015 IEEE 22nd International conference on software analysis, evolution, and reengineering (SANER)*, pages 33–42. IEEE, 2015. 72

[131] Lin Shi, Hao Zhong, Tao Xie, and Mingshu Li. An empirical study on evolution of API documentation. In *14th International Conference on Fundamental Approaches to Software Engineering (FASE), Held as Part of the Joint European Conferences on Theory and Practice of Software (ETAPS), Saarbrücken, Germany, March 26–April 3, 2011. Proceedings 14*, pages 416–431. Springer, 2011.

[132] Hao Zhong, Na Meng, Zexuan Li, and Li Jia. An empirical study on API parameter rules. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ICSE, pages 899–911, 2020.

[133] Hong Jin Kang and David Lo. Active learning of discriminative subgraph patterns for API misuse detection. *IEEE Transactions on Software Engineering*, 48(8):2761–2783, 2021. 72

[134] Hao Zhong and Zhendong Su. Detecting API documentation errors. In *Proceedings of the 2013 ACM SIGPLAN international conference on Object oriented programming systems languages & applications*, pages 803–816, 2013. 72

[135] Seonah Lee, Rongxin Wu, Shing-Chi Cheung, and Sungwon Kang. Automatic detection and update suggestion for outdated API names in documentation. *IEEE Transactions on Software Engineering*, 47(4):653–675, 2019. 72

[136] Arianna Blasi and Alessandra Gorla. Replicomment: identifying clones in code comments. In *Proceedings of the 26th Conference on Program Comprehension*, pages 320–323, 2018. 72

[137] Andrew Habib and Michael Pradel. Is this class thread-safe? inferring documentation using graph-based learning. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, ASE, pages 41–52, 2018.

[138] Shuang Liu, Jun Sun, Yang Liu, Yue Zhang, Bimlesh Wadhwa, Jin Song Dong, and Xinyu Wang. Automatic early defects detection in use case documents. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering*, ASE, pages 785–790, 2014.

[139] Pengyu Nie, Rishabh Rai, Junyi Jessy Li, Sarfraz Khurshid, Raymond J Mooney, and Milos Gligoric. A framework for writing trigger-action todo comments in executable format. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE, pages 385–396, 2019.

[140] Sheena Panthaplackel, Junyi Jessy Li, Milos Gligoric, and Raymond J Mooney. Deep just-in-time inconsistency detection between comments and source code. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 427–435, 2021.

[141] Daniela Steidl, Benjamin Hummel, and Elmar Juergens. Quality analysis of source code comments. In *2013 21st International Conference on Program Comprehension*, ICPC, pages 83–92. IEEE, 2013.

[142] Juan Zhai, Xiangzhe Xu, Yu Shi, Guanhong Tao, Minxue Pan, Shiqing Ma, Lei Xu, Weifeng Zhang, Lin Tan, and Xiangyu Zhang. CPC: Automatically classifying and propagating natural language comments via program analysis.

In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ICSE, pages 1359–1371, 2020.

[143] Fengcai Wen, Csaba Nagy, Gabriele Bavota, and Michele Lanza. A large-scale empirical study on code-comment inconsistencies. In *2019 IEEE/ACM 27th International Conference on Program Comprehension*, ICPC, pages 53–64. IEEE, 2019. 72

[144] Zhongxin Liu, Xin Xia, David Lo, Meng Yan, and Shanping Li. Just-in-time obsolete comment detection and update. *IEEE Transactions on Software Engineering*, 49(1):1–23, 2021. 72

[145] Bo Lin, Shangwen Wang, Kui Liu, Xiaoguang Mao, and Tegawendé F Bissyandé. Automated comment update: How far are we? In *2021 IEEE/ACM 29th International Conference on Program Comprehension*, ICPC, pages 36–46. IEEE, 2021. 72

[146] Nalin Wadhwa, Jui Pradhan, Atharv Sonwane, Surya Prakash Sahu, Nagarajan Natarajan, Aditya Kanade, Suresh Parthasarathy, and Sriram Rajamani. CORE: Resolving code quality issues using LLMs. *Proc. ACM Softw. Eng.*, (FSE), 2024. 73

[147] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. InferFix: End-to-end program repair with LLMs. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE, pages 1646–1656. Association for Computing Machinery, 2023. 73

[148] Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F. Bissyandé, and Shunfu Jin. CREF: An LLM-based conversational software repair framework for programming tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA, pages 882–894. ACM, 2024.

[149] Chunqiu Steven Xia and Lingming Zhang. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA, pages 819–831, New York, NY, USA, 2024. Association for Computing Machinery. 73

[150] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. Using an LLM to help with code understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE. Association for Computing Machinery, 2024. 73

[151] Yichi Zhang. Detecting code comment inconsistencies using LLM and program analysis. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, FSE, pages 683–685. Association for Computing Machinery, 2024. 73

[152] Yichi Zhang, Zixi Liu, Yang Feng, and Baowen Xu. Leveraging large language model to assist detecting rust code comment inconsistency. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ASE, pages 356–366. Association for Computing Machinery, 2024. 73

[153] Guoping Rong, Yongda Yu, Song Liu, Xin Tan, Tianyi Zhang, Haifeng Shen, and Jidong Hu. Code comment inconsistency detection and rectification using a large language model. In *2025 IEEE/ACM 47th International Conference on Software Engineering*, ICSE, pages 1832–1843. IEEE Computer Society, 2025. 73

[154] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench Lite dataset. `https://huggingface.co/datasets/SWE-bench/SWE-bench_Lite`, 2024. Accessed: 2025-10-04. 75, 96

[155] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8697–8727, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.426. URL `https://aclanthology.org/2025.acl-long.426/`. 76, 98, 110

[156] Luca Di Grazia and Michael Pradel. Code search: A survey of techniques for finding code. *ACM Computing Surveys*, 55(11):1–31, 2023. 77

[157] Github. CodeQL. `https://codeql.github.com/`, 2026. 78

[158] Xiuheng Wu, Chenguang Zhu, and Yi Li. Diffbase: A differential factbase for effective software evolution management. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE, pages 503–515, 2021. 84

[159] Tarun Suresh, Revanth Gangi Reddy, Yifei Xu, Zach Nussbaum, Andriy Mulyar, Brandon Duderstadt, and Heng Ji. Cornstack: High-quality contrastive data for better code ranking. *arXiv preprint arXiv:2412.01007*, 2024. URL `https://arxiv.org/abs/2412.01007`. 96

[160] Revanth Gangi Reddy, Tarun Suresh, JaeHyeok Doo, Ye Liu, Xuan Phi Nguyen, Yingbo Zhou, Semih Yavuz, Caiming Xiong, Heng Ji, and Shafiq Joty. SweRank: Software issue localization with code ranking. *arXiv preprint arXiv:2505.07849*, 2025. 98, 110

[161] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024. 98, 110

[162] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. CoSIL: Software issue localization via LLM-driven code repository graph searching. *arXiv preprint arXiv:2503.22424*, 2025. 98, 110

[163] Jan Strich, Florian Schneider, Irina Nikishina, and Chris Biemann. On improving repository-level code QA for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 4: Student Research Workshop)*, pages 209–244, 2024. 109

[164] Linyi Li, Shijie Geng, Zhenwen Li, Yibo He, Hao Yu, Ziyue Hua, Guanghan Ning, Siwei Wang, Tao Xie, and Hongxia Yang. Infibench: Evaluating the question-answering capabilities of code large language models. *Advances in Neural Information Processing Systems*, 37:128668–128698, 2024.

[165] Zehan Li, Jianfei Zhang, Chuantao Yin, Yuanxin Ouyang, and Wenge Rong. ProCQA: a large-scale community-based programming question answering dataset for code search. *arXiv preprint arXiv:2403.16702*, 2024. 109

[166] Surya Prakash Sahu, Madhurima Mandal, Shikhar Bharadwaj, Aditya Kanade, Petros Maniatis, and Shirish Shevade. Codequeries: A dataset of semantic queries over code. In *Proceedings of the 17th Innovations in Software Engineering Conference*, pages 1–11, 2024. 109

[167] Chenxiao Liu and Xiaojun Wan. CodeQA: A question answering dataset for source code comprehension. *arXiv preprint arXiv:2109.08365*, 2021. 109

[168] Junjie Huang, Duyu Tang, Linjun Shou, Ming Gong, Ke Xu, Daxin Jiang, Ming Zhou, and Nan Duan. Cosqa: 20,000+ web queries for code search and question answering. *arXiv preprint arXiv:2105.13239*, 2021. 109

[169] Ruida Hu, Chao Peng, Jingyi Ren, Bo Jiang, Xiangxin Meng, Qinyun Wu, Pengfei Gao, Xinchen Wang, and Cuiyun Gao. CodeRepoQA: A large-scale benchmark for software engineering question answering. *arXiv preprint arXiv:2412.14764*, 2024. 109

[170] Jialiang Chen, Kaifa Zhao, Jie Liu, Chao Peng, Jierui Liu, Hang Zhu, Pengfei Gao, Ping Yang, and Shuiguang Deng. CoreQA: uncovering potentials of language models in code repository question answering. *arXiv preprint arXiv:2501.03447*, 2025. 110

[171] Weihan Peng, Yuling Shi, Yuhang Wang, Xinyun Zhang, Beijun Shen, and Xiaodong Gu. SWE-QA: Can language models answer repository-level code questions? *arXiv preprint arXiv:2509.14635*, 2025. 110

[172] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024. 110

[173] Le Deng, Zhonghao Jiang, Jialun Cao, Michael Pradel, and Zhongxin Liu. NoCode-bench: A benchmark for evaluating natural language-driven feature addition. *arXiv preprint arXiv:2507.18130*, 2025.

[174] Changan Niu, Chuanyi Li, Vincent Ng, and Bin Luo. Crosscodebench: Benchmarking cross-task generalization of source code models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 537–549. IEEE, 2023.

[175] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[176] Yicheng Ouyang, Jun Yang, and Lingming Zhang. Benchmarking automated program repair: An extensive study on both real-world and artificial bugs. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 440–452, 2024.

[177] Xinyu Gao, Zhijie Wang, Yang Feng, Lei Ma, Zhenyu Chen, and Baowen Xu. Benchmarking robustness of ai-enabled multi-sensor fusion systems: Challenges and opportunities. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 871–882, 2023.

[178] Nan Jiang, Qi Li, Lin Tan, and Tianyi Zhang. Collu-bench: A benchmark for predicting language model hallucinations in code, 2024. URL `https://arxiv.org/abs/2410.09997`.

[179] Naman Jain, Manish Shetty, Tianjun Zhang, King Han, Koushik Sen, and Ion Stoica. R2e: Turning any GitHub repository into a programming agent environment. In *Forty-first International Conference on Machine Learning*, 2024.

[180] Niels Mündler, Mark Müller, Jingxuan He, and Martin Vechev. SWT-bench: Testing and validating real-world bug-fixes with code agents. *Advances in Neural Information Processing Systems*, 37:81857–81887, 2024.

[181] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.

[182] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents, 2025. URL `https://arxiv.org/abs/2504.21798`. 110

[183] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024. 110

[184] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.

[185] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1592–1604, 2024.

[186] Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. Magis: Llm-based multi-agent framework for GitHub issue resolution. *Advances in Neural Information Processing Systems*, 37:51963–51993, 2024.

[187] Chengxing Xie, Bowen Li, Chang Gao, He Du, Wai Lam, Difan Zou, and Kai Chen. SWE-fixer: Training open-source LLMs for effective and efficient GitHub issue resolution. In *ICLR 2025 Third Workshop on Deep Learning for Code*, 2025.

[188] Zexiong Ma, Chao Peng, Pengfei Gao, Xiangxin Meng, Yanzhen Zou, and Bing Xie. SoRFT: Issue resolving with subtask-oriented reinforced fine-tuning. *arXiv preprint arXiv:2502.20127*, 2025. URL `https://arxiv.org/abs/2502.20127`.

[189] Zexiong Ma, Chao Peng, Qunhong Zeng, Pengfei Gao, Yanzhen Zou, and Bing Xie. Tool-integrated reinforcement learning for repo deep search. *arXiv preprint arXiv:2508.03012*, 2025. URL `https://arxiv.org/abs/2508.03012`. 110

[190] Dejiao Zhang, Wasi Ahmad, Ming Tan, Hantian Ding, Ramesh Nallapati, Dan Roth, Xiaofei Ma, and Bing Xiang. Code representation learning at scale. *arXiv preprint arXiv:2402.01935*, 2024. 110

[191] Zhongming Yu, Hejia Zhang, Yujie Zhao, Hanxian Huang, Matrix Yao, Ke Ding, and Jishen Zhao. OrcaLoca: An LLM agent framework for software issue localization, 2025. URL `https://arxiv.org/abs/2502.00350`. 110

[192] Wei Liu, Chao Peng, Pengfei Gao, Aofan Liu, Wei Zhang, Haiyan Zhao, and Zhi Jin. GraphLocator: Graph-guided causal reasoning for issue localization. *arXiv preprint arXiv:2512.22469*, 2025. 110

[193] Ahmed Lekssays, Hamza Mouhcine, Khang Tran, Ting Yu, and Issa Khalil. LLMxCPG:context-aware vulnerability detection through code property graph-guided large language models. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 489–507, 2025. 110

[194] Anthropic. Models overview. `https://platform.claude.com/docs/en/models/overview`, 2026. 115

[195] OpenAI. Models overview. `https://developers.openai.com/api/docs/models`, 2026. 115

[196] Rachel Potvin and Josh Levenberg. Why google stores billions of lines of code in a single repository. *Communications of the ACM*, 59(7):78–87, 2016. 115

[197] Eric Christopher, Kevin Crossan, Wolff Dobson, Chris Kennelly, Drew Lewis, Kun Lin, Martin Maas, Parthasarathy Ranganathan, Emma Rapati, and Brian Yang. Instruction set migration at warehouse scale. arXiv preprint arXiv:2510.14928, 2025. URL `https://arxiv.org/abs/2510.14928`. 115

[198] Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A Rossi, Seunghyun Yoon, and Hinrich Schütze. Nolima: Long-context evaluation beyond literal matching. *arXiv preprint arXiv:2502.05167*, 2025. 115